

R Companion for

Sampling

Design and Analysis

Third Edition

R Companion for
Sampling
Design and Analysis
Third Edition

Yan Lu and Sharon L. Lohr



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business
A CHAPMAN & HALL BOOK

First edition published 2022
by CRC Press
6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

and by CRC Press
2 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

© 2022 Yan Lu and Sharon L. Lohr

CRC Press is an imprint of Taylor & Francis Group, LLC

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe. SAS® and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.

Library of Congress Cataloging-in-Publication Data

Names: Lu, Yan, author. | Lohr, Sharon L., 1960- author.

Title: R companion for sampling : design and analysis / Yan Lu and Sharon L. Lohr.

Description: First edition. | Boca Raton : CRC Press, 2022. | Includes bibliographical references and index. | Summary: "The R Companion for Sampling: Design and Analysis, designed to be read alongside Sampling: Design and Analysis, Third Edition by Sharon L. Lohr (SDA; 2022, CRC Press), shows how to use functions in base R and contributed packages to perform calculations for the examples in SDA. No prior experience with R is needed. Chapter 1 tells you how to obtain R and RStudio, introduces basic features of the R statistical software environment, and helps you get started with analyzing data. Each subsequent chapter provides step-by-step guidance for working through the data examples in the corresponding chapter of SDA, with code, output, and interpretation. Tips and warnings help you develop good programming practices and avoid common survey data analysis errors. R features and functions are introduced as they are needed so you can see how each type of sample is selected and analyzed. Each chapter builds on the knowledge developed earlier for simpler designs; after finishing the book, you will know how to use R to select and analyze almost any type of probability sample"-- Provided by publisher.

Identifiers: LCCN 2021039318 (print) | LCCN 2021039319 (ebook) | ISBN 9781032135946 (paperback) | ISBN 9781032132150 (hardback) | ISBN 9781003228196 (ebook)

Subjects: LCSH: R (Computer program language) | Sampling (Statistics)
Classification: LCC QA276.45.R3 L8 2022 (print) | LCC QA276.45.R3 (ebook)
| DDC 519.5/202855133--dc23

LC record available at <https://lccn.loc.gov/2021039318>

LC ebook record available at <https://lccn.loc.gov/2021039319>

ISBN: 978-1-032-13215-0 (hbk)

ISBN: 978-1-032-13594-6 (pbk)

ISBN: 978-1-003-22819-6 (ebk)

DOI: 10.1201/9781003228196

Access the Support Material: <https://www.routledge.com/9781032135946>

To Guoyi and Lynn, and to Doug

Contents

Preface	xi
1 Getting Started	1
1.1 Obtaining the Software	2
1.2 Installing R Packages	2
1.3 R Basics	4
1.4 Reading Data into R	5
1.5 Saving Output	7
1.6 Integrating R Output into L ^A T _E X Documents	10
1.7 Missing Data	12
1.8 Summary, Tips, and Warnings	13
2 Simple Random Sampling	15
2.1 Selecting a Simple Random Sample	15
2.2 Computing Statistics from a Simple Random Sample	18
2.3 Additional Code for Exercises	24
2.4 Summary, Tips, and Warnings	25
3 Stratified Sampling	27
3.1 Allocation Methods	27
3.2 Selecting a Stratified Random Sample	30
3.3 Computing Statistics from a Stratified Random Sample	32
3.4 Estimating Proportions from a Stratified Random Sample	36
3.5 Additional Code for Exercises	37
3.6 Summary, Tips, and Warnings	38
4 Ratio and Regression Estimation	41
4.1 Ratio Estimation	41
4.2 Regression Estimation	44
4.3 Domain Estimation	46
4.4 Poststratification	48
4.5 Ratio Estimation with Stratified Sampling	49
4.6 Model-Based Ratio and Regression Estimation	50
4.7 Summary, Tips, and Warnings	54
5 Cluster Sampling with Equal Probabilities	57
5.1 Estimates from One-Stage Cluster Samples	57
5.2 Estimates from Multi-Stage Cluster Samples	59
5.3 Model-Based Design and Analysis for Cluster Samples	63
5.4 Additional Code for Exercises	65
5.5 Summary, Tips, and Warnings	67

6	Sampling with Unequal Probabilities	69
6.1	Selecting a Sample with Unequal Probabilities	69
6.1.1	Sampling with Replacement	69
6.1.2	Sampling without Replacement	70
6.2	Selecting a Two-Stage Cluster Sample	71
6.3	Computing Estimates from an Unequal-Probability Sample	77
6.3.1	Estimates from with-Replacement Samples	77
6.3.2	Estimates from without-Replacement Samples	79
6.4	Summary, Tips, and Warnings	83
7	Complex Surveys	85
7.1	Selecting a Stratified Two-Stage Sample	85
7.2	Estimating Quantiles	88
7.3	Computing Estimates from Stratified Multistage Samples	89
7.4	Univariate Plots from Complex Surveys	92
7.5	Scatterplots from Complex Surveys	95
7.6	Additional Code for Exercises	103
7.7	Summary, Tips, and Warnings	105
8	Nonresponse	107
8.1	How R Functions Treat Missing Data	107
8.2	Poststratification and Raking	108
8.3	Imputation	110
8.4	Summary, Tips, and Warnings	112
9	Variance Estimation in Complex Surveys	113
9.1	Replicate Samples and Random Groups	113
9.2	Constructing Replicate Weights	116
9.2.1	Balanced Repeated Replication	117
9.2.2	Jackknife	120
9.2.3	Bootstrap	122
9.2.4	Replicate Weights and Nonresponse Adjustments	124
9.3	Using Replicate Weights from a Survey Data File	126
9.4	Summary, Tips, and Warnings	127
10	Categorical Data Analysis in Complex Surveys	129
10.1	Contingency Tables and Odds Ratios	129
10.2	Chi-Square Tests	131
10.3	Loglinear Models	133
10.4	Summary, Tips, and Warnings	137
11	Regression with Complex Survey Data	139
11.1	Straight Line Regression with a Simple Random Sample	139
11.2	Linear Regression for Complex Survey Data	142
11.3	Using Regression to Compare Domain Means	145
11.4	Logistic Regression	149
11.5	Additional Resources and Code	151
11.6	Summary, Tips, and Warnings	152
12	Additional Topics for Survey Data Analysis	155
12.1	Two-Phase Sampling	155
12.2	Estimating the Size of a Population	157

<i>Contents</i>	ix
12.2.1 Ratio Estimation of Population Size	157
12.2.2 Loglinear Models with Multiple Lists	159
12.3 Small Area Estimation	161
12.4 Summary	162
A Data Set Descriptions	163
Bibliography	197
Index	205

Preface

R Companion for Sampling: Design and Analysis, Third Edition shows how to use the R statistical software environment to perform the calculations in the textbook *Sampling: Design and Analysis, Third Edition* (SDA) by Sharon L. Lohr. It is intended to be read in conjunction with SDA and is not a standalone text. The parallel book by Lohr (2022) shows how to perform the computations for the examples using SAS[®] software, and could be read together with this book and SDA to learn how to perform the analyses in each software package.

All code and data sets can be downloaded from any of the following websites:

<https://math.unm.edu/~luyan/rbook.html>

<https://www.sharonlohr.com>

<https://www.routledge.com/9781032135946>

The first two websites also contain additional programs, not discussed in this book, that you can adapt for some of the SDA exercises. The data sets used in this book have also been saved in R format in the contributed R package `SDAResources` (Lu and Lohr, 2021).

In this book, we give step-by-step guidance for using functions from base R and contributed packages to select samples and analyze the data sets discussed in Chapters 1–13 of SDA. The software, however, can do much more than analyze the examples presented in this book. You can find information on advanced capabilities for the `survey` and `sampling` contributed packages in the documentation for those packages by Lumley (2020) and Tillé and Matei (2021); the books and articles by Lumley (2004, 2010) and Tillé and Matei (2010) provide additional information about the packages. Goga (2018) gives an overview of using R for survey sampling.

For easy reference, the index at the back of the book gives page numbers for the examples in SDA. To locate the code and output for Example 2.5, for example, look up the subentry “Example 02.05” under “Examples in SDA” in the index. The book also gives code and suggestions for some of the exercises in SDA, and these are listed under index entry “Exercises in SDA.”

Each chapter ends with a summary section containing tips and warnings for the analyses discussed in that chapter. These provide ways of avoiding common survey data analysis errors and checking whether you did the analysis correctly.

Although prior experience with R is helpful, it is not needed to read this book. Chapter 1 tells how to obtain the software and do basic operations in R. It also lists resources for learning more about programming in R and tells how to obtain help.

This book makes use of functions that exist in base R and contributed packages, and does not discuss how to write R functions. One of R’s most valuable features, however, is the capacity for writing functions to carry out new tasks. Advanced R users may want to write their own functions to select samples or analyze data from a complex survey. When teaching

survey sampling to students who have R programming experience, we have sometimes asked them to write their own functions to carry out various sampling tasks. This helps solidify their knowledge of the material and allows them to do computations not available in existing functions. For example, we have asked students to write R functions to perform allocation for and analyze data from a stratified random sample, select a with-replacement unequal-probability sample using Lahiri's method, compute the Sen–Yates–Grundy estimate of the variance, simulate the sampling distribution of a statistic, and find empirical estimates of the coverage probability of a confidence interval for a biased estimator.

All code, data sets, and output in this book are provided for educational purposes only and without warranty. Base R does not contain functions for survey data, and this book relies heavily on contributed packages that have been developed. These packages are in widespread use and have been quality-checked by their authors and other users. We have verified that the calculations from the R functions used for the examples in this book agree with calculations by the formulas and with calculations performed in other survey software packages.

Other R packages may not be checked as carefully, however. Although R contributed packages undergo some consistency and functionality tests when they are submitted (see Wickham, 2015, for a description of checks that are performed), no central authority reviews the packages to make sure that the functions do what they claim or that the algorithms perform computations accurately. Most R contributed packages are not peer-reviewed, and you should be aware that some may contain errors.

The code and output in this book were developed using version 4.0.4 of R for Windows (R Core Team, 2021) and the versions of the packages listed in their respective bibliography entries, and all code in the book works with those versions. But R is a dynamic language, and the R Core Team and authors of contributed packages can change or remove functions at any time. Although most authors who revise a package try to avoid changes that will affect previously written code, functions in R are not guaranteed to be backward compatible—it is possible that R code you write today may not work the same way with future versions of the software. If backward compatibility is important to you—for example, if you will be using the same code to produce estimates each year for an annual survey—you may want to perform or check your computations in a package that is backward compatible, such as SAS software. If a function changes in a subsequent version of an R package, you can either:

- Read the documentation and change your code so that it works with the modified function, or
- Download and use the older version of the package. You can find previous versions on the package's web page under the heading "Old sources."

Acknowledgments. Many thanks to John Kimmel, our editor at CRC Press, for encouraging us to write this book, and to the CRC Press production team for all their support and help. We are grateful to Yves Tillé and Thomas Lumley for answering questions about the **sampling** and **survey** packages. Students in Yan Lu's sampling class at the University of New Mexico provided helpful suggestions for clarifying the material. We also want to thank Lynn Zhang for helping with the preparation of the **SDAResources** package.

Complex Surveys

We have already seen most of the components needed for selecting and computing estimates from a stratified multistage sample. Now let's put them all together. Section 7.1 shows how to select a stratified two-stage sample, and Section 7.3 shows how to compute estimates using examples from the National Health and Nutrition Examination Survey (NHANES).

The new features considered in this chapter are how to estimate quantiles (Section 7.2) and how to graph survey data (Sections 7.4 and 7.5). The code is in file `ch07.R` on the book website.

7.1 Selecting a Stratified Two-Stage Sample

In the following example, we select a sample of classes from the small population considered in Section 6.2 of SDA, after first dividing the classes into three strata based on their sizes. Stratum 1 contains the two large classes, stratum 2 contains six medium-sized classes, and stratum 3 contains the seven smallest classes. The code specifies drawing two primary sampling units (psus) without replacement (`srswor`) from each stratum, and drawing three students without replacement from each selected class. Of course other allocations of psus to strata can be used, as described in Chapter 3. The psus are arranged in strata, so the only new feature here is to add the stratification information in the function `mstage` from the `sampling` package (Tillé and Matei, 2021).

We use the `classeslong` data frame that we created in Section 1.5, where variable `class` gives the psu number and variable `studentid` gives the student identifier (numbered from 1 to `class_size`) within each class.

```
# data(classeslong)
# define strata
classeslong$strat<-rep(3,nrow(classeslong))
classeslong$strat[classeslong$class_size > 40]<-2
classeslong$strat[classeslong$class_size > 70]<-1
# table(classeslong$class,classeslong$strat)
# order data by stratum
classeslong2<-classeslong[order(classeslong$strat),]
# check the stratum construction
table(classeslong2$strat,classeslong2$class_size)
##
##      15  20  22  24  26  33  34  44  46  54  63  76 100
##  1    0   0   0   0   0   0   0   0   0   0   0  76 100
##  2    0   0   0   0   0   0   0   88  92  54  63   0   0
##  3  15  20  22  24  26  33  34   0   0   0   0   0   0
nrow(classeslong2) # number of students in population
## [1] 647
```

To select a stratified cluster sample using *mstage*, we first define *numberselect* consisting of information on stratum sizes, number of psus selected from each stratum, and number of secondary sampling units (ssus; here, students) selected within each sampled psu as follows:

```
numberselect<-list(table(classeslong2$strat),rep(2,3),rep(3,6))
```

Next, use the *mstage* function to select a stratified two-stage cluster sample.

```
mstage(classeslong2,stage=list("stratified","cluster","stratified"),
        varnames=list("strat","class","studentid"),
        size=numberselect, method=list("", "srswor", "srswor"))
```

The *stage* argument for this example is a list with three components. The first component, "stratified", defines the stratification for the psus, but nothing is selected at this stage (the first component of *method* is blank). Then, an SRS of psus (variable *class*) is selected within each stratum (this is described by the second components of *stage*, *varnames*, and *method*). At this point, the sample consists of the selected psus from the 3 strata. We then use "stratified" again to select an SRS of students from each sampled psu.

The result from *mstage* is a list with three components corresponding to the components of *stage* in the function argument. The third component is the final sample, saved in *sample3*. The variable *Prob*, computed by function *mstage*, is the final selection probability, and we calculate *finalweight* as its reciprocal.

```
# select a stratified two stage cluster sample,
# stratum: strat
# psu: class
# ssu: studentid
# number of psus selected n =2, size=rep(n=2,3 strata) (srswor)
# number of students selected m_i =3 size=rep(m_i= 3,6 classes) (srswor)
numberselect<-list(table(classeslong2$strat),rep(2,3),rep(3,6))
numberselect
## [[1]]
##
## 1 2 3
## 176 297 174
##
## [[2]]
## [1] 2 2 2
##
## [[3]]
## [1] 3 3 3 3 3 3
# select a stratified two-stage cluster sample
set.seed(75745)
tempid<-mstage(classeslong2,stage=list("stratified","cluster","stratified"),
               varnames=list("strat","class","studentid"),
               size=numberselect, method=list("", "srswor", "srswor"))
# get data
sample3<-getdata(classeslong2,tempid)[[3]] #3rd stage
sample3$finalweight<-1/sample3$Prob
# check sum of weights, should be close to number of students in population
# (but not exactly equal, since psus not selected with prob proportional to M_i)
sum(sample3$finalweight)
## [1] 624
sample3 # print the sample
##      class class_size studentid strat ID_unit Prob_3 _stage      Prob
## 5.31      5         76        32     1      32    0.03947368 0.03947368
```

```
## 5.42      5      76      43      1      43      0.03947368 0.03947368
## 5.61      5      76      62      1      62      0.03947368 0.03947368
## 14.9     14     100     10      1      86      0.03000000 0.03000000
## 14.37    14     100     38      1     114      0.03000000 0.03000000
## 14.79    14     100     80      1     156      0.03000000 0.03000000
## 8.34      8      44      35      2     318      0.06818182 0.02272727
## 8.39      8      44      40      2     323      0.06818182 0.02272727
## 8.28      8      44      29      2     312      0.06818182 0.02272727
## 9.26      9      54      27      2     354      0.05555556 0.01851852
## 9.28      9      54      29      2     356      0.05555556 0.01851852
## 9.38      9      54      39      2     366      0.05555556 0.01851852
## 7.17      7      20      18      3     572      0.15000000 0.04285714
## 7.5       7      20       6      3     560      0.15000000 0.04285714
## 7.10      7      20      11      3     565      0.15000000 0.04285714
## 12.6     12      24       7      3     615      0.12500000 0.03571429
## 12.9     12      24      10      3     618      0.12500000 0.03571429
## 12.16    12      24      17      3     625      0.12500000 0.03571429
##          finalweight
## 5.31     25.33333
## 5.42     25.33333
## 5.61     25.33333
## 14.9     33.33333
## 14.37    33.33333
## 14.79    33.33333
## 8.34     44.00000
## 8.39     44.00000
## 8.28     44.00000
## 9.26     54.00000
## 9.28     54.00000
## 9.38     54.00000
## 7.17     23.33333
## 7.5      23.33333
## 7.10     23.33333
## 12.6     28.00000
## 12.9     28.00000
## 12.16    28.00000
```

You can also look at the selection probabilities for each stage if desired. The following extracts the other stages in *sample1* and *sample2*. The first stage defines the stratification, so *Prob_ 1 _stage* is 1. Stratum 1 contains two psus, so the function selects each with certainty. Classes are selected from stratum 2 with probability of 2/6, and classes are selected from stratum 3 with probability of 2/7. These are reported as *Prob_ 2 _stage* of *sample2*. At the third stage, *Prob_ 3 _stage* is calculated as $3/class_size$. The final probability *Prob* is calculated as $1*Prob_ 2_stage*Prob_ 3_stage$.

```
sample1<-getdata(classeslong2,tempid)[[1]] #1st stage
sample2<-getdata(classeslong2,tempid)[[2]] #2nd stage
names(sample1)
## [1] "class"      "class_size"  "studentid"   "strat"
## [5] "ID_unit"    "Prob_ 1 _stage" "Stratum"
table(sample1$`Prob_ 1 _stage`)
##
## 1
## 647
table(sample2$strat,sample2$`Prob_ 2 _stage`) # Selection probs for psus in strata
```

```
##
##      0.285714285714286 0.333333333333333 1
##    1      0      0 176
##    2      0      98 0
##    3     44      0 0
table(sample3$class,sample3$`Prob_ 3 _stage`) # Selection probs for ssus in psus
##
##      0.03 0.0394736842105263 0.0555555555555556 0.0681818181818182 0.125 0.15
##    5      0      3      0      0 0 0
##    7      0      0      0      0 0 3
##    8      0      0      0      3 0 0
##    9      0      0      3      0 0 0
##   12      0      0      0      0 3 0
##   14      3      0      0      0 0 0
```

For more complicated designs, you may want to select the sample at each stage separately, as illustrated in Section 6.2. For example, you can use function *UPsampford* to select a sample of psus from each stratum, then select the sample at the subsequent stages.

7.2 Estimating Quantiles

Quantiles are estimated using the empirical cumulative distribution function (cdf) $\hat{F}(y)$, which is the sum of the weights for the sample observations having $y_i \leq y$ divided by the sum of all of the weights. Because $\hat{F}(y)$ has jumps at the distinct values of y in the sample, however, for many values of q there is no value of y in the sample that has $\hat{F}(y)$ exactly equal to q . Multiple definitions for population and sample quantiles have been proposed (Hyndman and Fan, 1996; Wang, 2021).

The *svyquantile* function in the **survey** package (Lumley, 2020) calculates several estimates of quantiles and their standard errors. With **ties="discrete"**, the empirical cdf $\hat{F}$ is used directly, with jumps at the values of y in the sample. With **ties="rounded"** an interpolated cdf is used (see Exercise 7.19 of SDA). We usually prefer interpolated quantiles, as they smooth out an empirical cdf that has large jumps.

The quantiles $\tilde{\theta}_q$ are calculated by requesting the desired quantile values in the *svyquantile* function. Request the 0.25, 0.5, 0.75, and 0.90 quantiles, for example, by typing **quantiles=c(0.25, 0.50, 0.75, 0.90)**.

Example 7.5 of SDA. The following code requests quantiles and CIs for the height values in the SRS *htsrs*. Of course, for a complex design, one would include stratification and clustering information in the *svydesign* function.

```
data(htsrs)
dhtsrs<-svydesign(id = ~1,weights=rep(2000/200,200),fpc=rep(2000,200), data=htsrs)
# cdf treated as step function, gives values in Table 7.1 of SDA
svyquantile(~height, dhtsrs, quantiles=c(0.25,0.5,0.75,0.9), ties = "discrete")
##      0.25 0.5 0.75 0.9
## height 160 169 176 184
# interpolated quantiles (usually preferred method)
svyquantile(~height, dhtsrs, quantiles=c(0.25,0.5,0.75,0.9), ties = "rounded")
##      0.25 0.5 0.75 0.9
## height 159.7 168.75 176 183.4
```

The *svyquantile* function will also calculate confidence intervals for quantiles if you request `ci=TRUE`. The default method is `interval.type="Wald"`, which calculates the Woodruff (1952) interval presented in Section 9.5 of SDA.

Examples 7.6 and 9.12 of SDA. Here we calculate confidence intervals for the interpolated quantiles in the *htstrat* data.

```
data(htstrat)
popsize_recode <- c('F' = 1000, 'M' = 1000)
# create a new variable popsize for population size
htstrat$popsize<-popsize_recode[htstrat$gender]
head(as.data.frame(htstrat))
##      rn height gender popsize
## 1 201    166      F    1000
## 2 965    163      F    1000
## 3 490    166      F    1000
## 4 249    155      F    1000
## 5 260    154      F    1000
## 6 324    160      F    1000
# design object
# svydesign calculates the weights here from the fpc argument
dhtstrat<-svydesign(id = ~1, strata = ~gender, fpc = ~popsize,
  data = htstrat)
# ties = "discrete" gives values in Table 7.1 of SDA
svyquantile(~height, dhtstrat, c(0.25,0.5,0.75,0.9), ties = "discrete")
##      0.25 0.5 0.75 0.9
## height 161 168 177 182
# ties = "rounded" gives values in Example 9.12 of SDA
svyquantile(~height, dhtstrat, c(0.25,0.5,0.75,0.9), ties = "rounded",
  ci=TRUE, interval.type = "Wald")
## $quantiles
##      0.25      0.5      0.75      0.9
## height 160.7143 167.5556 176.625 181.5
##
## $CIs
## , , height
##
##      0.25      0.5      0.75      0.9
## (lower 159.3556 165.8078 173.3572 178.7176
## upper) 162.0247 170.0942 178.5439 190.1679
```

7.3 Computing Estimates from Stratified Multistage Samples

We have seen all the building blocks for computing the estimates from any survey. Now let's put them all together using the data from the National Health and Nutrition Examination Survey (NHANES, Centers for Disease Control and Prevention, 2017). The Centers for Disease Control and Prevention produce online tutorials for analyzing NHANES data. These, and sample code and tips for analyzing NHANES data using R, can be found at <https://wwwn.cdc.gov/nchs/nhanes/tutorials/>.

Example 7.9 of SDA. In this example, we look at statistics about body mass index (BMI, variable *bmxbmi*) for adults age 20 and over. We will compute these estimates using *svymean* and *svyquantile*.

One of the statistics to be calculated is the proportion of adults having BMI greater than 30, so we define categorical variable *bmi30* to equal 1 if the person's BMI is greater than 30 and 0 if it is less than or equal to 30. Note that the variable *bmxbmi* has missing values, so we set *bmi30* to be missing if *bmxbmi* is missing. In the data file *nhanes.csv*, missing values are coded by -9 . We coded all the missing values -9 as "NA" in the R data set *nhanes* in the *SDAResources* package.

We also need to define a variable giving the domain of interest to be analyzed: adults age 20 and over who have data for *bmxbmi*. We define *age20d*=1 if *ridageyr* ≥ 20 and *bmxbmi* is not missing, and 0 otherwise. This excludes the observations with missing values from the domain of interest, and ensures that standard errors for the domain are calculated correctly.

```
data(nhanes)
nrow(nhanes) #9971
## [1] 9971
names(nhanes)
## [1] "sdmvstra" "sdmvpsu" "wtint2yr" "wtmec2yr" "ridstatr" "ridageyr"
## [7] "ridagemn" "riagendr" "ridreth3" "dmdeduc2" "dmdfmsiz" "indfmpir"
## [13] "bmxwt" "bmxht" "bmxbmi" "bmxaist" "bmxlbg" "bmxaaml"
## [19] "bmxaarmc" "bmdavsad" "lbxtc" "bpxpls" "sbp" "dbp"
## [25] "bpread"
# count number of observations with missing value for ridageyr, bmxbmi
sum(is.na(nhanes$ridageyr)) # ridageyr gives age in years
## [1] 0
sum(is.na(nhanes$bmxbmi)) # bmxbmi gives BMI
## [1] 1215
# define age20d and bmi30
nhanes$age20d<-rep(0,nrow(nhanes))
nhanes$age20d[nhanes$ridageyr >=20 & !is.na(nhanes$bmxbmi)]<-1
nhanes$bmi30<-nhanes$bmxbmi
nhanes$bmi30[nhanes$bmxbmi>30]<-1
nhanes$bmi30[nhanes$bmxbmi<=30]<-0
nhanes$bmi30<-factor(nhanes$bmi30) # set bmi30 as a categorical variable
# check missing value counts for new variables
sum(is.na(nhanes$age20d))
## [1] 0
sum(is.na(nhanes$bmi30))
## [1] 1215
sum(nhanes$age20d) # how many records in domain?
## [1] 5406
head(nhanes)
## sdmvstra sdmvpsu wtint2yr wtmec2yr ridstatr ridageyr ridagemn riagendr
## 1 125 1 134671.37 135629.51 2 62 NA 1
## 2 125 1 24328.56 25282.43 2 53 NA 1
## 3 131 1 12400.01 12575.84 2 78 NA 1
## 4 131 1 102718.00 102078.63 2 56 NA 2
## 5 126 2 17627.67 18234.74 2 42 NA 2
## 6 128 1 11252.31 10878.68 2 72 NA 2
## ridreth3 dmdeduc2 dmdfmsiz indfmpir bmxwt bmxht bmxbmi bmxaist bmxleg
## 1 3 5 2 4.39 94.8 184.5 27.8 101.1 43.3
## 2 3 3 1 1.32 90.4 171.4 30.8 107.9 38.0
```

## 3	3	3	2	1.51	83.4	170.1	28.8	116.5	35.6	
## 4	3	5	1	5.00	109.8	160.9	42.4	110.1	38.5	
## 5	4	4	5	1.23	55.2	164.9	20.3	80.4	37.4	
## 6	1	2	5	2.82	64.4	150.0	28.6	92.9	34.4	
##	bmxarml	bmxarmc	bmdavsad	lbxtc	bpxpls	sbp	dbp	bpread	age20d	bmi30
## 1	43.6	35.9	22.8	173	76	120	63	2	1	0
## 2	40.0	33.2	27.3	265	72	137	85	2	1	1
## 3	37.0	31.0	26.6	229	56	134	45	2	1	0
## 4	37.7	38.3	25.1	174	78	135	69	2	1	1
## 5	36.0	27.2	NA	204	76	106	55	2	1	0
## 6	33.5	31.4	23.1	190	64	121	59	2	1	0

Next, we use *svydesign* to describe the design information and use *subset* to select the domain of adults with data for BMI for analysis.

```
# stratified cluster design
d0709 <- svydesign(id = ~sdmvpsu, strata=~sdmvstra, weights=~wtmec2yr,
                 nest=TRUE, data = nhanes)
# domain estimation, age20+
d0709sub<-subset(d0709, age20d ==1)
d0709sub
## Stratified 1 - level Cluster Sampling design (with replacement)
## With (30) clusters.
## subset(d0709, age20d == 1)
```

Let's look at the features used in the *svydesign* function to describe the design.

- The *weights* argument in *svydesign* specifies the variable containing the final weights. The data set contains two weight variables: *wtint2yr* gives the weight for the set of persons with interview data, and *wtmec2yr* gives the weight for the subset of interviewed persons who had a medical examination. BMI is measured in the medical examination, so the appropriate weight variable to use is *wtmec2yr*.
- The *strata* and *id* arguments are used exactly as in Chapters 3 and 5, except now we include both of them. The *strata=~sdmvstra* argument says that *sdmvstra* is the variable giving the stratum membership. The psus specified in *id=~sdmvpsu* are the first-stage sampling units.
- In the NHANES data, the two psus in each stratum are labeled as '1' and '2'. The *nest=TRUE* argument says that psu labels are nested within strata—that is, multiple strata have the same psu labels. Typing *nest=TRUE* ensures that psu 1 in stratum 1 is recognized as being a different psu than psu 1 in stratum 2.

When there is one stratum, the results are the same if you have *nest=TRUE*, *nest=FALSE*, or simply omit the *nest* argument.

- No *fpc* argument is included in *svydesign*. With complex samples such as NHANES, we usually want to calculate the with-replacement variance, which requires only psu-level information.

The *subset* function to define domains has been discussed in Chapter 4. This specifies that estimates are desired for the domain of persons age 20 and older having data for BMI (with *age20d=1*), and carries the stratification and clustering information from the full design over for analyzing the subset. If you just created a subset of the data consisting of the observations having *ridageyr* ≥ 20 , in some instances (for example, when some psus have

no members of the domain), the standard errors would be incorrect; by using the *subset* function, the correct standard errors are calculated.

Note that we exclude adults with missing values of *bmx bmi* from the domain of interest with *age20d*=1. The estimates are computed from the adults who have data. If the domain contained missing values, we would need to include *na.rm=TRUE* in the *svymean* function to be able to calculate statistics.

Functions *svymean* and *svyquantile* are then applied to calculate the estimated mean and quantiles of BMI and the proportion in each category of *bmi30*. The confidence intervals for quantiles differ slightly from those in SDA, which were calculated using SAS software under a slightly different algorithm. Adding *deff=TRUE* to *svymean* requests the design effect (*deff*) for each statistic.

```
# Request means and design effects
nhmeans<-svymean(~bmx bmi+bmi30, d0709sub, deff=TRUE)
degf(d0709sub)
## [1] 15
nhmeans
##           mean           SE    DEff
## bmx bmi 29.389101  0.253197  7.1248
## bmi300  0.607775  0.015856  5.7003
## bmi301  0.392225  0.015856  5.7003
confint(nhmeans,df=degf(d0709sub))
##           2.5 %           97.5 %
## bmx bmi 28.8494243 29.9287768
## bmi300  0.5739798  0.6415707
## bmi301  0.3584293  0.4260202
# Find quantiles
svyquantile(~bmx bmi, d0709sub, quantiles=c(0.05,0.25,0.5,0.75,0.95),
            ties = "rounded",ci=TRUE, interval.type="Wald")
## $quantiles
##           0.05           0.25           0.5           0.75           0.95
## bmx bmi 20.29893 24.35349 28.2349 33.06615 42.64092
##
## $CIs
## , , bmx bmi
##
##           0.05           0.25           0.5           0.75           0.95
## (lower 19.83403 23.92667 27.55465 32.35400 41.91584
## upper) 20.70609 24.84391 28.91359 33.64129 43.47766
```

The mean BMI for adults age 20 and over is 29.389 (using design object *d0709sub*), with 95% confidence interval [28.849, 29.929]. The estimated proportion of adults age 20 and over who have BMI > 30 (*bmi30*=1) is 0.392225 with 95% confidence interval [0.3584293, 0.4260202]. The confidence interval is calculated using a *t* distribution with 15 degrees of freedom (number of psus minus number of strata).

7.4 Univariate Plots from Complex Surveys

The *svyhist* and *svyboxplot* functions, along with *svydesign*, produce histograms and box-plots that incorporate the weights, as described in Chapter 7 of SDA.

Examples 7.10, 7.11, and 7.12 of SDA. These examples consider data in *htstrat*, a disproportional stratified sample of 160 women and 40 men.

Histograms and smoothed density estimates. Figure 7.1 shows the difference between a histogram constructed without the weights (left panel) and one constructed with the weights (right panel). Each histogram is overlaid with a smoothed density estimate.

```
data(htstrat)
# set graphics parameters, 1*2 plots, axis labels horizontal
par(mfrow=c(1,2),las=1,mar=c(2.1,4.1,2.1,0.3))
# Histogram overlaid with kernel density curve (without weight information)
# Displays the sample values, but does not estimate population histogram
# freq=FALSE changes the vertical axis to density
# breaks tell how many breakpoints to use
hist(htstrat$height,main="Without weights", xlab = "Height (cm)",
      breaks = 10, col="gray90", freq=FALSE, xlim=c(140,200), ylim=c(0,0.045))
# overlaid with kernel density curve
lines(density(htstrat$height),lty=1,lwd=2)

# Histogram (with weight information)
# create survey design object, weights calculated from fpc here
d0710 <- svydesign(id = ~1, strata = ~gender, fpc = c(rep(1000,160),rep(1000,40)),
                  data = htstrat)
d0710
## Stratified Independent Sampling design
## svydesign(id = ~1, strata = ~gender, fpc = c(rep(1000, 160),
##      rep(1000, 40)), data = htstrat)
svyhist(~height,d0710, main="With weights",xlab = "Height (cm)",
        breaks = 10, col="gray90", freq=FALSE,xlim=c(140,200), ylim=c(0,0.045))
dens1<-svysmooth(~height,d0710,bandwidth=5)
lines(dens1,lwd=2) # draw the density line
```

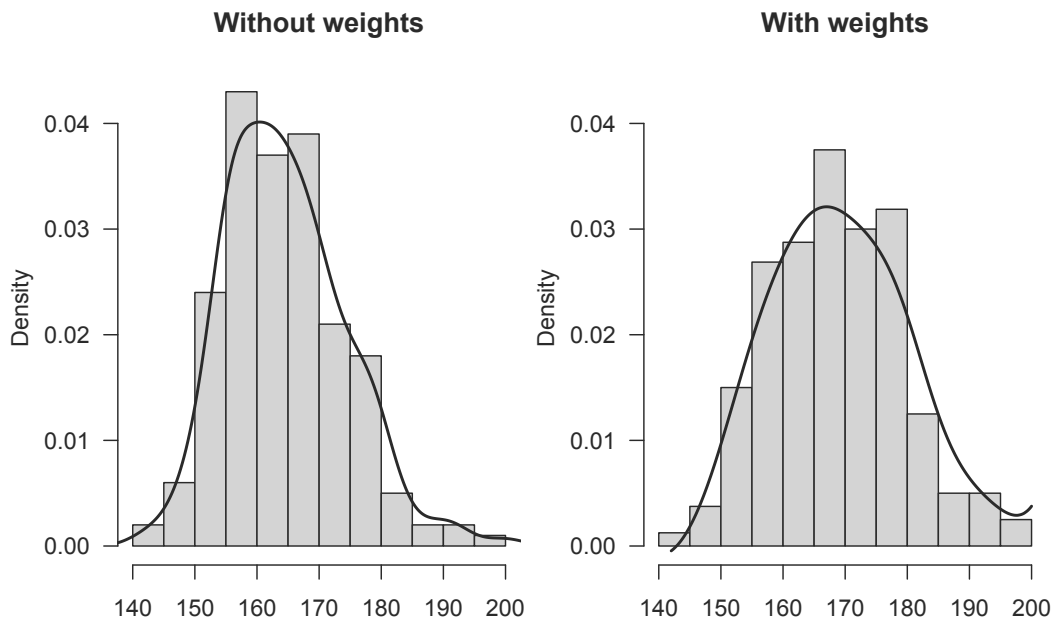


FIGURE 7.1: Histograms constructed without and with weights.

We call *svydesign* and *svyhist* to create a histogram that uses the survey information. The only difference between *svyhist* and *hist* is that *svyhist* includes the design object so that the histogram accounts for the weights.

```
svyhist(~height, design object, main=" ", xlab = " ", ylab= "")
```

In *hist* and *svyhist*, the *breaks* argument controls the number of bins, and *freq=FALSE* changes the vertical axis to density instead of frequency. The *density* function calculates a kernel density curve for unweighted data, and *svysmooth* performs density estimation using the weighted data. We allowed the bandwidth to be chosen automatically by the *density* function and specified a bandwidth of 5 in the *svysmooth* function (if you omit the *bandwidth* argument, it will be chosen automatically).

In Figure 7.1, the unweighted plot on the left displays the values in the sample but, because this is a disproportionally allocated sample, it does not estimate the histogram that would be obtained if we measured everyone in the population. The distribution appears skewed, reflecting the underrepresentation of men (who have greater average height) in the sample. The histogram and density estimate in the plot on the right incorporate the survey weights and thus can be interpreted as estimates of the histogram and density that would be obtained if the entire population were measured.

Boxplots. The *svyboxplot* function creates boxplots that include the weights; it is the survey analog of the *boxplot* function. Figure 7.2 displays boxplots for the full sample and separately for each gender.

```
par(mfrow=c(1,2),las=1,mar=c(2.1,4.1,2.1,0.3))
# boxplot (with weight information)
svyboxplot(height~1,d0710,ylab="Height",xlab=" ", main="Full sample")
svyboxplot(height~gender,d0710,ylab="Height",xlab="Gender",
           main="Separately by gender")
```

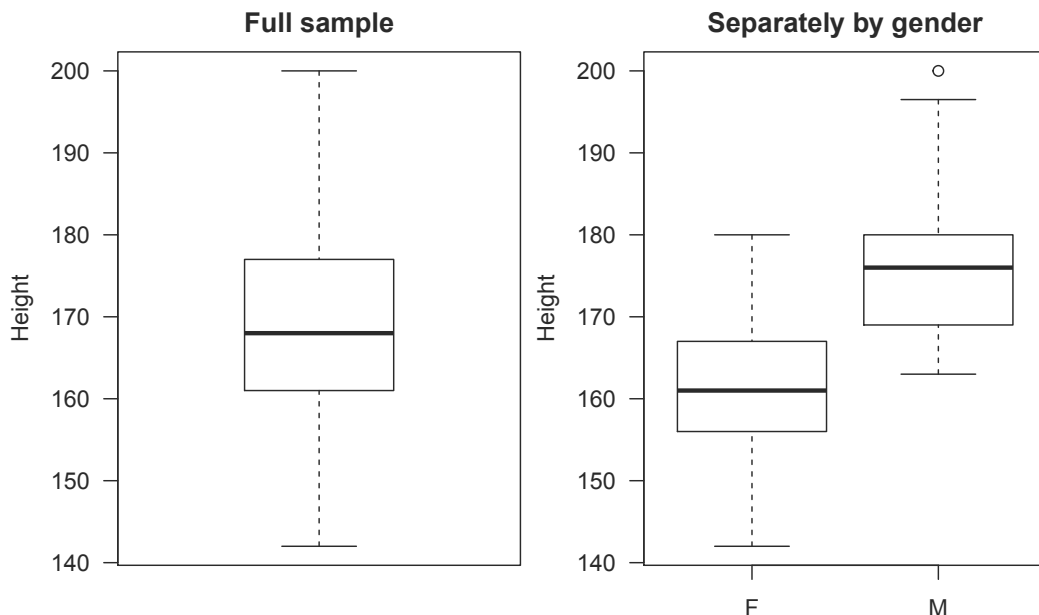


FIGURE 7.2: Boxplots of height for the full sample, and separately by gender.

Histograms for domains. What if you want to draw a histogram for just one domain from a complex survey? You can do that by using the *subset* function to redefine the object from *svydesign*. Figure 7.3 gives a histogram of BMI for adults age 20 and over, using the survey weights in the *nhanes* data. The smoothed density function is superimposed.

```
# Restore graphics settings
par(mfrow=c(1,1),las=1,mar=c(5.1, 4.1, 4.1, 2.1))
svyhist(~bmx bmi,d0709sub, main="Histogram of body mass index for adults",
       breaks = 30, col="gray90",xlab = "Body Mass Index (kg/m^2)")
dens2<-svysmooth(~bmx bmi,d0709sub)
lines(dens2,lwd=2)
```

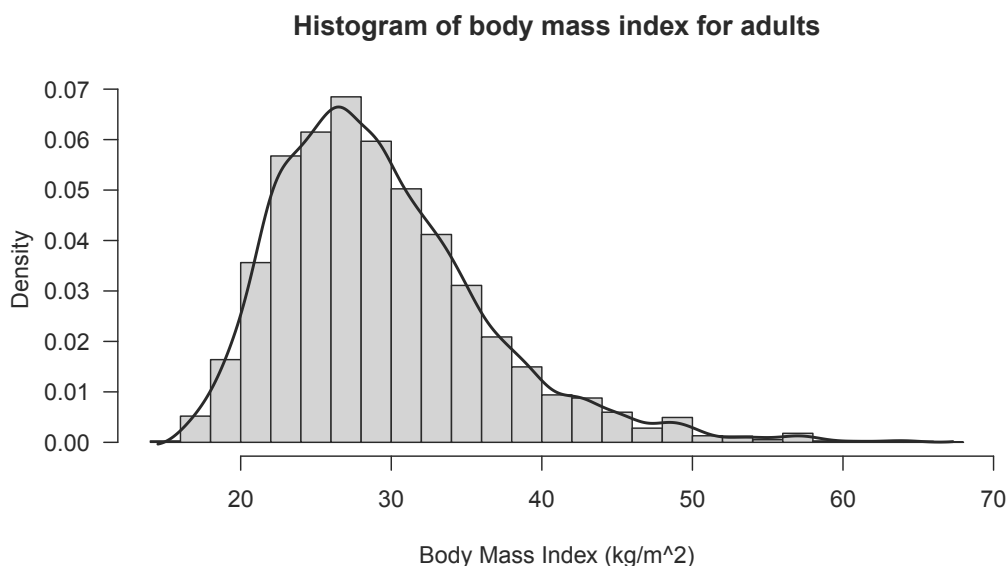


FIGURE 7.3: Histogram and density estimate for adults age 20 and over.

7.5 Scatterplots from Complex Surveys

Most of the plot types described in Section 7.6 of SDA can be drawn using functions *plot*, *svyplot*, *svyboxplot*, or *svycdf*.

Unweighted plots. The R functions *plot*, *hist*, and *boxplot* draw scatterplots, histograms, boxplots, and more. They do not incorporate the survey weights. In general, extra data preparation is needed to persuade these functions to draw graphs that estimate the population.

In some instances, however, you may want to examine the unweighted data. You may want to see how the weights affect the regression relationship between x and y , or to identify unusual observations in the data. For that reason, we include an unweighted scatterplot of the NHANES data.

The following shows the code to produce the unweighted scatterplot of y variable body mass index (*bmx bmi*) versus x variable age (*ridageyr*) shown in Figure 7.4. This plot is for all ages, not just the subset of adults for which we computed summary statistics in Section 7.3. Numerous options are available for customizing the plot; we chose to use the default setting `type="p"` for the points and used plotting symbol '+' (`pch=3`), making the symbols small so more of them are displayed on the plot (`cex=0.5`). The optional `xlab` and `ylab` arguments allow customizing the axis labels. You can also set the minimum and maximum values for each axis using `xlim` and `ylim`.

```
# scatterplot without weights
par(las=1) # make tick mark labels horizontal
plot(nhanes$ridageyr,nhanes$bmx bmi,xlab="Age (years)",ylab="Body Mass Index",
     main="Scatterplot without weights",pch=3,cex=0.5,
     ylim=c(10,70),xlim=c(0,80))
```

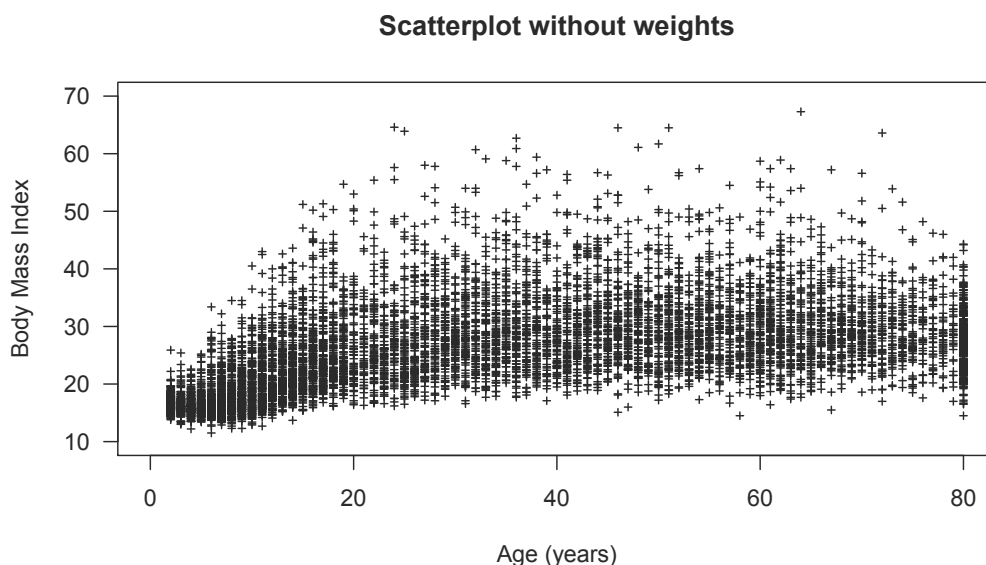


FIGURE 7.4: Scatterplot of BMI versus age (unweighted).

The unweighted scatterplot shows the relationship between x and y in the sample. If the sample is self-weighting, this scatterplot also estimates the relationship between x and y in the population. If the sample is not self-weighting, however, the relationship between x and y in the population may be different, and one of the other plots in this section should be used if you want to visualize the relationship between x and y in the population.

Although we do not recommend unweighted plots for non-self-weighting samples in general, sometimes, for small data sets, they are useful for seeing whether the relationship between x and y is the same with and without the weights (see Section 11.4 of SDA). The NHANES data, however, have so many data points that it is difficult to see patterns from the unweighted scatterplot in Figure 7.4.

Plot subsample of data. We can also select a subsample of the data that is approximately self-weighting, and draw the scatterplot of that subsample using the *plot* function. The plot of the subsample, displayed in Figure 7.5, then estimates the scatterplot that would be drawn from the population.

This subsample was selected with probability proportional to the weights and with replacement, using the *sample* function. Any method that will select a sample with probability proportional to the weights can be used, however, including the *UP* functions described in Table 6.1. If desired, you can draw multiple plots with different subsamples.

```
# select subsample with probability proportional to weights
set.seed(2847654)
subsamp<-sample(1:nrow(nhanes),500,replace=TRUE,prob=nhanes$wtmec2yr)
par(las=1) # make tick mark labels horizontal
plot(nhanes$ridageyr[subsamp],nhanes$bmxbmi[subsamp],
     xlab="Age (years)",ylab="Body Mass Index",
     main="Scatterplot of pps subsample",pch=3,cex=0.5,
     ylim=c(10,70),xlim=c(0,80))
```

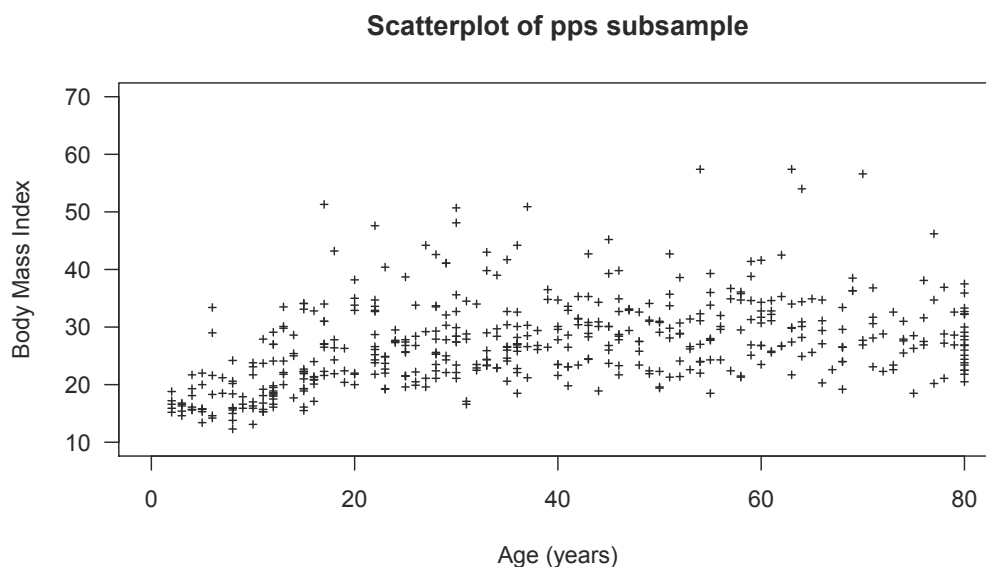


FIGURE 7.5: Scatterplot of BMI versus age (self-weighting subsample).

Bubble plots. If the sample is not self-weighting, a bubble plot displays the estimated shape of the population data. Bubble plots can be constructed using individual (x, y) values, or using bins. We will show how to use *svyplot* to create bubble plots of the individual (x, y) values.

The *svyplot* function is essentially the same as the *plot* function, except it adds the survey design object so that the weights are incorporated into the plots. You can use the same arguments to label the axes (*xlab*, *ylab*), set the extent of the plotting region (*xlim*, *ylim*), title the plot (*main*), and perform other formatting as in the *plot* function.

The *svyplot* function will construct a variety of types of plots that incorporate the weights, including bubble plots, “transparent” plots where the opacity of points is proportional to their weights, and binned hexagonal scatterplots.

Figure 7.6 shows a bubble plot for the NHANES data. The *inches* argument scales the bubbles; you may need to try several values until you find one that looks nice.

```
par(las=1) # make tick mark labels horizontal
svyplot(bmxbmi~ridageyr, design=d0709, style="bubble", inches=0.03,
        xlab="Age(years)", ylab="Body Mass Index", xlim=c(0,80), ylim=c(10,70),
        main="Weighted bubble plot of BMI versus age")
```

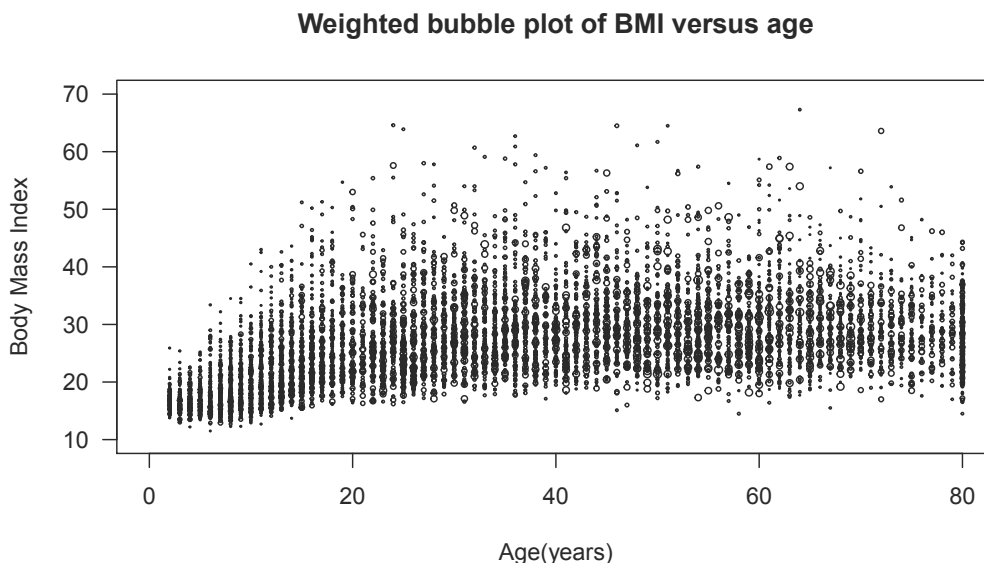


FIGURE 7.6: Bubble plot of BMI versus age (weighted).

Plot data for a domain. If we want to draw a plot for sample members in a specific domain, we can use the *subset* function to define a design object for the domain and then use *svyplot* with the subset design object.

Figure 7.7 shows a bubble plot of BMI versus age for the domain of non-Hispanic Asian Americans (having *ridreth3*=6).

```
# define subset
d0709subA<-subset(d0709, ridreth3==6)
par(las=1) # make tick mark labels horizontal
svyplot(bmxbmi~ridageyr, design=d0709subA, style="bubble", inches = 0.03,
        xlab="Age(years)", ylab="Body Mass Index", xlim=c(0,80), ylim=c(10,70),
        main="Weighted bubble plot of BMI versus age for Asian Americans")
```

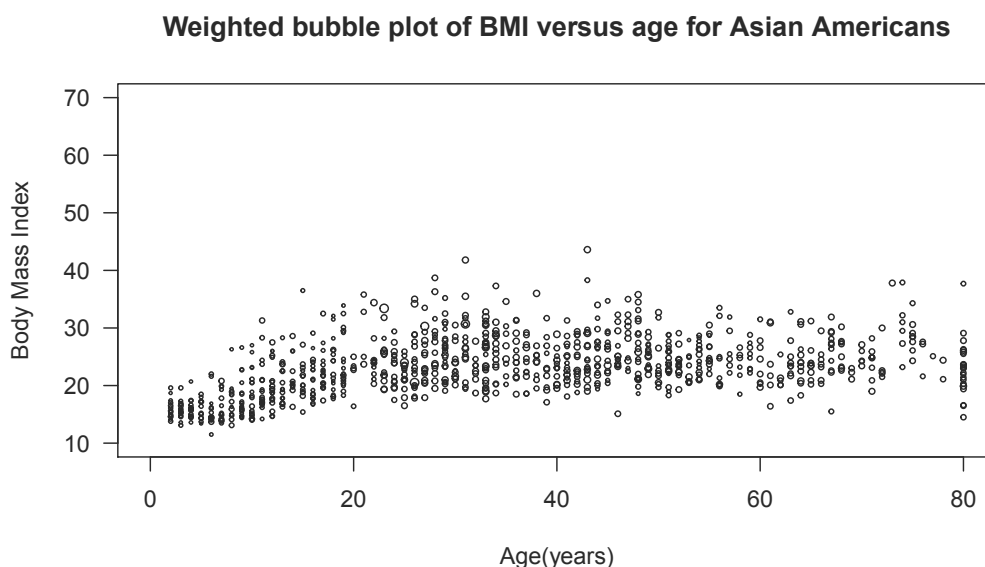


FIGURE 7.7: Bubble plot of BMI versus age for Asian Americans.

Note that for purposes of graphing data, it is also acceptable to consider just the subset of observations being graphed. This is because the scatterplots in this section incorporate the weights, but they do not use any other features of the survey design. Thus, if producing a subsampled graph for a domain that is similar to that in Figure 7.5, you could first create a subset of data consisting of the domain of interest, then select points with probabilities proportional to the weights, and then use the *plot* function to draw the scatterplot. With the *svyplot* function, however, it is easiest to create a subset design object for the domain being graphed.

Side-by-side boxplots. Boxplots will display the distributions of subgroups of the data. Figure 7.2 showed their use to display the distribution of height, separately for males and females.

They can also be used to display the bivariate relationship between two continuous variables. Simply partition the x variable into a categorical variable that defines different ranges of the variable. You can use the *round* function to round the x variable to the nearest multiple of a number. Alternatively, you can use the *cut* function to divide the range of x into intervals with user-supplied cutpoints and code each value x by the interval that contains it.

The easiest way to obtain side-by-side boxplots for survey data is through the *svyboxplot* function. Figure 7.8 shows boxplots of BMI by age groups that are formed by rounding the values of *ridageyr* to the nearest multiple of 5.

```
# include agegroup in the data frame
nhanes$agegroup<-5*round(nhanes$ridageyr/5)
d0709 <- svydesign(id = ~sdmvpsu, strata = ~ sdmvstra, nest=TRUE,
                  weights=~wtmec2yr, data = nhanes)
par(las=1) # make tick mark labels horizontal
svyboxplot(bmxbmi~factor(agegroup),d0709,ylab="Body mass index",xlab="Age Group",
           ylim=c(10,70),main="Side-by-side boxplots of BMI for age groups")
```

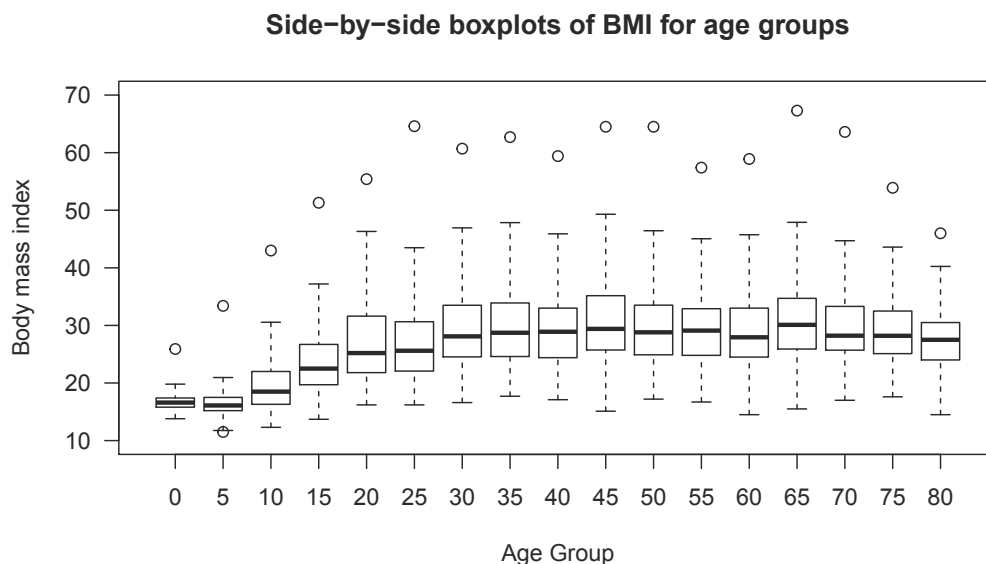


FIGURE 7.8: Boxplots of BMI by age groups.

Smoothed trend line for mean. The *svysmooth* function can carry out scatterplot smoothing and density estimation for weighted data. For the default method "locpoly", the extra arguments are passed to *locpoly* from the *KernSmooth* package (Wand et al., 2020), which implements the smoothing methods described in Wand and Jones (1995). The default is local linear smoothing for the mean. If desired, the *bandwidth* argument can be included for a user-specified bandwidth.

Figure 7.9 shows the smoothed curve superimposed on the bubble plot in Figure 7.6. The color of the bubbles is changed to light gray for better visibility of the trend line.

```
# plot data bmxbbmi~ridageyr
par(las=1) # make tick mark labels horizontal
svyplot(bmxbbmi~ridageyr, design=d0709, style="bubble", basecol="gray", inches=0.03,
        xlab="Age(years)", ylab="Body Mass Index", xlim=c(0,80), ylim=c(10,70),
        main="Smoothed trend line with bubble plot of BMI versus age")
# plot smoothing trend line
# library(KernSmooth) # install and load the package if not already done
smth<-svysmooth(bmxbbmi~ridageyr,d0709)
lines(smth,lwd=2)
```

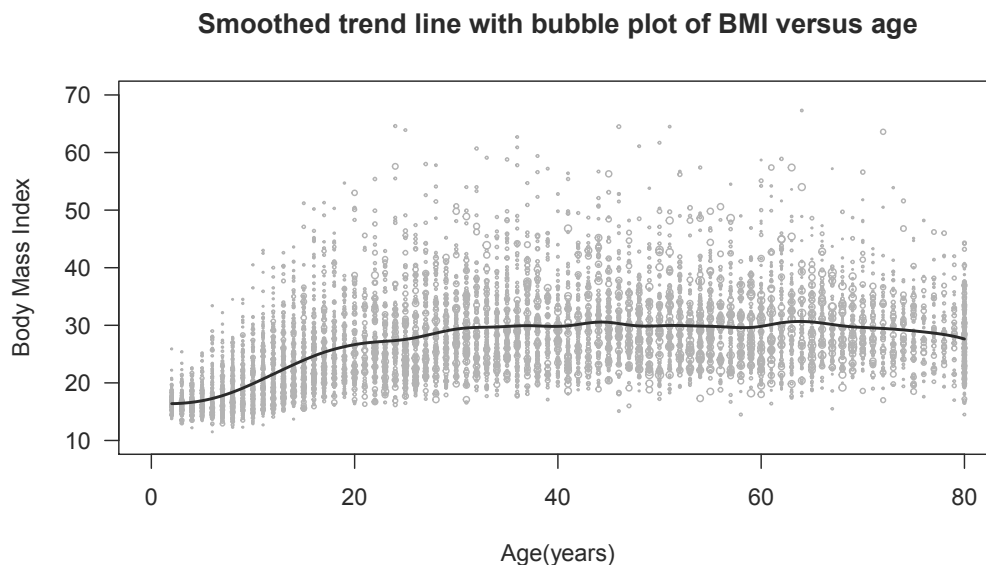


FIGURE 7.9: Smoothed trend line with bubble plot of BMI versus age.

Other smoothing methods may also be used to estimate trend lines. Figure 7.14 in SDA was created by calculating survey-weighted spline estimates (Zhang et al., 2015) of the trend line for a grid of x points and then connecting the dots with the *lines* function.

Smoothed trend lines for quantiles. To fit smoothed lines for quantiles, change to `method="quantreg"` in `svysmooth`. This smooths the regression quantiles from package `quantreg` (Koenker et al., 2021). Instead of fitting smoothed lines to the means of different groups, separate lines are drawn that estimate each conditional quantile (Koenker, 2005) specified in the `quantile` argument. You can think of this as “connecting the dots” of the quantiles shown in the side-by-side boxplots in Figure 7.8. We requested the quantiles corresponding to probabilities 0.05, 0.25, 0.5, 0.75, and 0.95 with the argument `taus=c(.05,.25,.5,.75,.95)`. This set of quantiles gives a good picture of the center (median) of the data, as well as the large and small values. Figure 7.10 includes the bubble plot and the smoothed line for each quantile.

```
# library(quantreg)) # install and load the package if not already done
# plot data bmxbmi~ridageyr
par(las=1) # make tick mark labels horizontal
svyplot(bmxbmi~ridageyr, design=d0709, style="bubble", basecol="gray", inches=0.03,
        xlab="Age (years)", ylab="Body Mass Index", xlim=c(0,80), ylim=c(10,70),
        main="Smoothed quantile trend lines")
# plot smoothed trend lines for quantiles
taus<-c(.05,.25,.5,.75,.95)
for (i in 1:length(taus)) {
  qsmth<-svysmooth(bmxbmi~ridageyr,d0709, quantile=taus[i],method="quantreg")
  lines(qsmth,lwd=1.2)
}
```

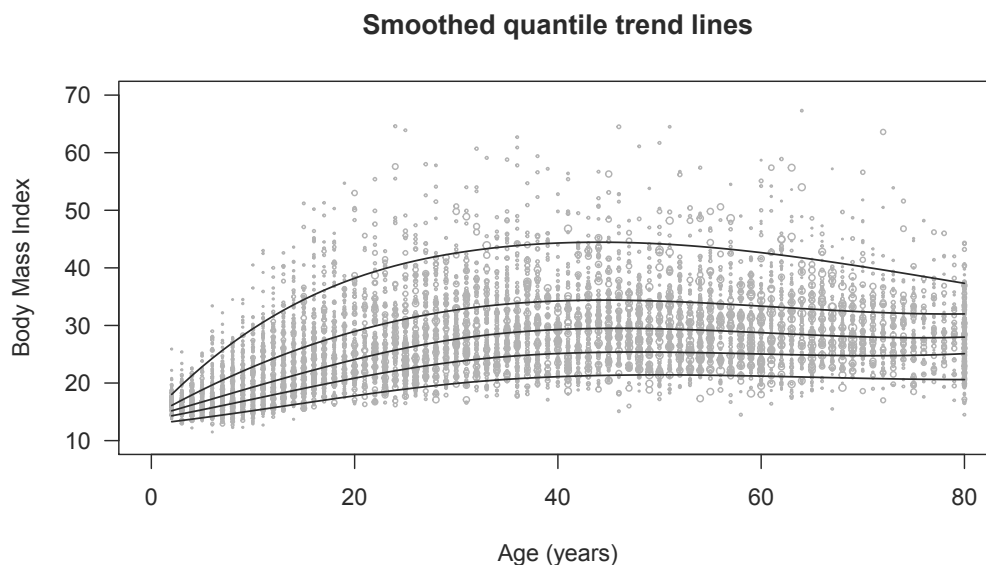


FIGURE 7.10: Smoothed quantile trend lines with bubble plot of BMI versus age.

Customizing graphs. Users who want to be able to do more customization of the graphs may want to write their own code. The graphs in SDA were produced with custom-written code (not using the `svyplot` function), and the file `ch07.R` includes code for using the `ggplot2` graphics visualization package (Wickham et al., 2020) to create more types of scatterplots with survey data.

7.6 Additional Code for Exercises

Some of the exercises in Chapter 7 of SDA ask you to construct an empirical cumulative distribution function (ecdf) or an empirical probability mass function (epmf). Any population characteristic can be estimated using these functions. There are functions in R that calculate commonly requested statistics such as means, totals, quantiles, and regression coefficients from survey data, so the survey analyst typically does not calculate or graph the ecdf. For continuous variables, it is usually more informative to create a histogram or a smoothed density estimate than to view the epmf, which may contain a large number of spikes.

The ecdf and epmf are useful concepts for learning about how survey weights work, however, because they illustrate how the sample is used to create a reconstruction of the population. And both are easy to calculate in R.

Example 7.5 of SDA. Function *emppmf* from package **SDAResources** calculates the empirical probability mass function for a variable with associated weights. Call the function as

```
emppmf(y,sampling.weight)
```

to return a list with component vectors *vals*, the distinct values of *y*, and *epmf*, the value of the epmf corresponding to each *y* value in *vals*.

Here we look at the stratified sample of heights in *htstrat*, where each female has a sampling weight of 1000/160 and each male has a sampling weight of 1000/40. It produces the estimated population proportion for each of the distinct values of *height*; the estimated proportion for value *y* is the sum of the weights for observations having *height* = *y* divided by the sum of all the weights in the sample. Figure 7.11 plots the epmf for *height*, using *type*="h" to draw vertical lines.

```
# Empirical pmf for stratified sample of heights
# define sampling weight
htstrat$sampwt <- 1000/sum(htstrat$gender=="F")
htstrat$sampwt[htstrat$gender=="M"] <- 1000/sum(htstrat$gender=="M")
# use function emppmf to calculate pmf
strresult <- emppmf(htstrat$height,htstrat$sampwt)
# plot
par(las=1)
plot(strresult$vals, strresult$epmf,type="h",xlab="Height Value, y (cm)",
     ylab="Empirical pdf",lwd=1.2,
     main="Empirical pdf for stratified sample of heights (weighted)")
```

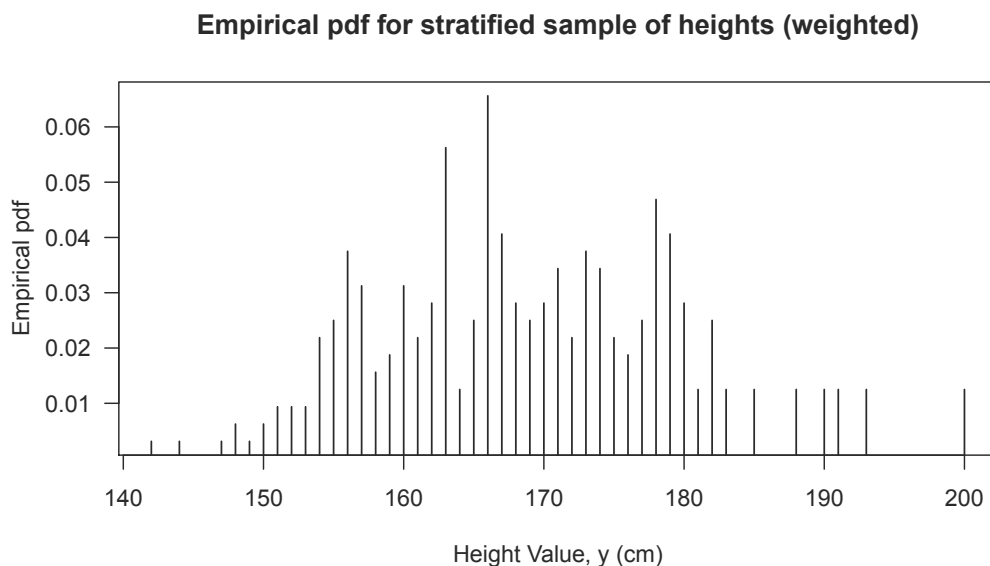


FIGURE 7.11: Empirical pmf for stratified sample of heights, using weights.

Function *ecdf* in base R will estimate the empirical cdf without weights, and function *svycdf* from the **survey** package will estimate the empirical cdf by incorporating survey weights. In contrast to *svyquantile*, *svycdf* does not interpolate but instead produces a right-continuous step function. Figure 7.12 shows the cdf of the population *htpop* (thick black) with the ecdf of sample *htstrat* without incorporating weights (medium red), and the ecdf of sample *htstrat* with weights (thin purple). We see that the medium red line is quite far off from the population cdf, while the thin purple line with weights is close to the population cdf.

```
# data(htstrat)
# Recall that
# d0710 <- svydesign(id = ~1, strata = ~gender, fpc = c(rep(1000,160),rep(1000,40)),
#                   data = htstrat)
cdf.weighted<-svycdf(~height, d0710)
cdf.weighted
## Weighted ECDFs: svycdf(~height, d0710)
## evaluate the function for height 144
cdf.weighted[[1]](144)
## [1] 0.00625
## compare to population and unweighted sample ecdfs.
cdf.pop<-ecdf(htpop$height) # ecdf for population
cdf.samp<-ecdf(htstrat$height) # unweighted ecdf of sample
par(las=1,mar=c(5.1,4.1,2.1,2.1))
plot(cdf.pop, do.points = FALSE,
     xlab="Height value y",ylab="Empirical cdf",xlim=c(135,205),lwd=2.5,
     main="Empirical cdfs for population and sample")
lines(cdf.samp, col="red", do.points = FALSE, lwd=1.8)
lines(cdf.weighted[[1]], do.points = FALSE, col = "purple",lwd=1)
legend("topleft", legend=c("Population", "Sample unweighted", "Sample weighted"),
     col=c("black", "red", "purple"),lwd=c(2.5,1.8,1),cex=0.8,bty="n")
```

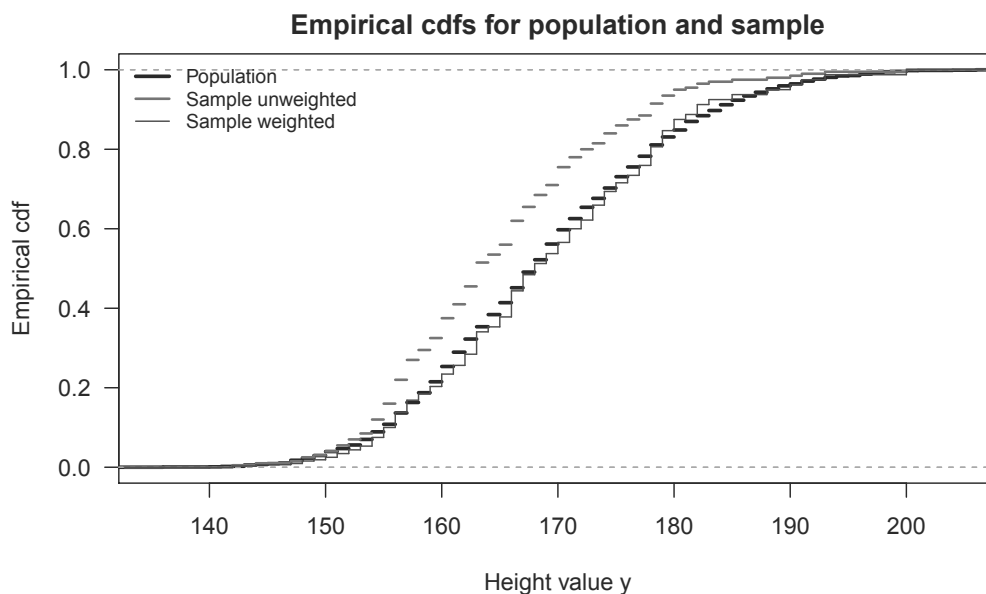


FIGURE 7.12: Empirical cdf of height for data *htpop*, and for data *htstrat* with and without weights.

7.7 Summary, Tips, and Warnings

Table 7.1 lists the major R functions used in this chapter.

Tips and Warnings

- For stratified multistage surveys, put the stratification variable in the *strata* argument of *svydesign*, and put the first-stage clustering variable in the *id* argument. Include the *weights* argument and do not include *fpc*. This will calculate the with-replacement variance approximations.
- If you want separate statistics for domains, first create the design object for the entire sample using the *svydesign* function, then use the *subset* or *svyby* function to calculate statistics for the domain of interest. This will ensure that standard errors for the domain statistics are calculated correctly. Do not subset the data first and then run *svydesign*—this can lead to incorrect standard error calculations.
- Incorporate the survey weights when constructing graphs if you want them to estimate the graphs that would be constructed if you had data from the entire population.

TABLE 7.1

Functions used for Chapter 7.

Function	Package	Usage
sample	base	Select a with-replacement sample with unequal probabilities
subset	base	Work with a subset of a vector, matrix, or data frame
confint	stats	Calculate confidence intervals, add df for t confidence interval
density	stats	Compute a kernel density estimate for self-weighting data
ecdf	stats	Calculate empirical cdf from self-weighting sample
par	graphics	Set graphics parameters
hist	graphics	Draw a histogram without weights
boxplot	graphics	Draw a boxplot without weights
plot	graphics	Draw a scatterplot without weights
mstage	sampling	Select a stratified multistage sample
svydesign	survey	Specify the survey design
svymean	survey	Calculate mean and standard error of mean
svyquantile	survey	Calculate quantiles and their confidence intervals
svyhist	survey	Draw a histogram of survey data, incorporating the weights
svyboxplot	survey	Draw boxplot of survey data, incorporating the weights
svyplot	survey	Draw scatterplot of survey data, incorporating the weights
svsmooth	survey	Estimate a smoothed density estimate or trend line from survey data
svycdf	survey	Calculate the empirical cdf from survey data
emppmf	SDAResources	Calculate the empirical probability mass function from survey data

Nonresponse

We have already seen most of the features needed to select survey samples and to compute estimates of means, totals, and quantiles using the sampling weights and survey design.

Nonresponse affects almost all surveys, however. Even samples of inanimate objects such as audit records may have missing accounts or missing items for accounts. This chapter looks at how R deals with missing data and points you to resources for calibrating survey data and performing imputations in R. The code is in file `ch08.R` on the book website.

8.1 How R Functions Treat Missing Data

We saw in Section 1.7 that R uses the value `NA` to denote missing data. If a data set uses a different code for missing values, such as `-99`, these must be converted to `NA` before analyzing the data so that R recognizes the values as missing. Otherwise, functions will treat these observations as though $y = -99$ and include them in calculations, sometimes with embarrassing results as when the estimated mean age is a negative number.

Data set `impute.csv` contains the data from Table 8.4 of SDA; it codes the missing values by `-99`. In the R data set `impute` in the `SDAResources` package, these missing values have been converted to `NA`. Variables *education*, *crime*, and *violcrime* all have missing values.

Most functions in R have a default method for how missing values are treated. The *mean* function, for example, returns `NA` if you attempt to find the mean of a vector that has missing values; if you want it to calculate the mean of the non-missing values, include argument `na.rm=TRUE`. The following shows how some of the base R functions and the *svy*mean function treat the missing data in variables *crime* and *violcrime*, which take on values 1, 0, and `NA`.

```
data(impute)
impute$crime
## [1] 0 1 0 1 1 0 1 0 0 NA 0 0 1 1 0 0 0 0 NA 0
is.na(impute$crime) # vector with TRUE for missing values
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
# identify the rows with no missing values in columns 5-6
impute$cc<-complete.cases(impute[,5:6])
impute$cc
## [1] TRUE TRUE TRUE TRUE TRUE TRUE FALSE TRUE FALSE FALSE TRUE TRUE
## [13] FALSE TRUE TRUE TRUE TRUE TRUE FALSE TRUE
mean(impute$crime) # returns NA
## [1] NA
mean(impute$crime,na.rm=TRUE) # calculates mean of non-missing values
## [1] 0.3333333
```

```

table(impute$crime,impute$violcrime) # excludes values missing in either variable
##
##      0  1
##  0 11  0
##  1  1  3
table(impute$crime,impute$violcrime,useNA="ifany") # counts NAs as category in table
##
##      0  1 <NA>
##  0   11  0    1
##  1    1  3    2
## <NA>  1  0    1
# input design information, use relative weights of 1 for comparison with above
dimpute <- svydesign(id = ~1, weights = rep(1,20), data = impute)
dimpute
## Independent Sampling design (with replacement)
## svydesign(id = ~1, weights = rep(1, 20), data = impute)
# calculate survey mean and se
svymean(~crime,dimpute) # returns NA
##      mean SE
## crime   NA NA
svymean(~crime, dimpute, na.rm=TRUE)
##      mean    SE
## crime 0.33333 0.114
svytable(~violcrime+crime,dimpute)
##      crime
## violcrime 0  1
##           0 11  1
##           1  0  3

```

When `na.rm=TRUE` is included, the *mean* and *svymean* functions estimate the mean using the observations having non-missing values.

In a complex survey, however, omitting the missing values can affect the standard error calculations. A better option, for complex surveys where the missing values are not imputed, is to use the *subset* function to define the subset of observations with non-missing values on which the analysis is to be performed—treating the non-missing values as a domain, as seen in Section 7.3. Remember, though, that an analysis of complete cases alone can be subject to nonresponse bias. You may want to explore the sensitivity of your estimates to the assumptions about the missing values.

8.2 Poststratification and Raking

Unit nonresponse occurs when a unit selected for the sample provides no data. In some data sets, the nonrespondent unit may be represented in the data set, but with missing values for all survey responses. In others, nonrespondent units are missing entirely from the data set.

The most common method for trying to compensate for potential effects of nonresponse is to weight the data. The sampling frame may contain information that can be used in weighting class adjustments, or information known about the population from an external source may be used to poststratify or rake the data as described in Section 8.6 of SDA.

We have already seen the *postStratify* function from the *survey* package (Lumley, 2020) in Chapter 4. The package also has a function to rake the weights to marginal counts in a table. It takes the form

```
rake(design.object, sample.margins, population.margins)
```

where *sample.margins* is a list of formulas describing the sample margins and *population.margins* is a list giving the population counts for each raking variable.

Here is a simple example, using the data in Section 8.6.2 of SDA. The entries in the data frame *rakewtsum* are the sums of weights for persons in the sample falling into each cross-classification of gender by race. For this example, we assume that the data are from a sample of size 500 where each person has weight 6 and construct data frame *rakedf* that has 500 records listing the race and gender of each person (in most applications of raking, you will already have the data frame and will not need this step).

Function *rake* is then used to do raking adjustment on design object *drake* according to the population marginal totals of gender (*pop.gender*) and race (*pop.race*).

```
rakewtsum <- data.frame(gender=rep(c("F","M"),each=5),
                        race=rep(c("Black","White","Asian","NatAm","Other"),times=2),
                        wtsum=c(300,1200,60,30,30,150,1080,90,30,30))

rakewtsum # check data entry
##      gender  race wtsum
## 1      F Black   300
## 2      F White 1200
## 3      F Asian   60
## 4      F NatAm   30
## 5      F Other   30
## 6      M Black   150
## 7      M White 1080
## 8      M Asian   90
## 9      M NatAm   30
## 10     M Other   30

# Need data frame with individual records to use rake function
rakedf <- rakewtsum[rep(row.names(rakewtsum), rakewtsum[,3]/6), 1:2]
dim(rakedf)
## [1] 500    2
rakedf$wt <- rep(6,nrow(rakedf))
# Create the survey design object
drake <- svydesign(id=~1, weights=~wt, data=rakedf)
# Create data frames containing the marginal counts
pop.gender <- data.frame(gender=c("F","M"), Freq=c(1510,1490))
pop.race <- data.frame(race=c("Black","White","Asian","NatAm","Other"),
                      Freq=c(600,2120,150,100,30))

# Now create survey design object with raked weights
drake2 <- rake(drake, list(~gender,~race), list(pop.gender, pop.race))
drake2 # describes SRS with replacement
## Independent Sampling design (with replacement)
## rake(drake, list(~gender, ~race), list(pop.gender, pop.race))
# Look at first 10 entries in vector of raked weights
weights(drake2)[1:10]
##      1      1.1      1.2      1.3      1.4      1.5      1.6      1.7
## 7.511886 7.511886 7.511886 7.511886 7.511886 7.511886 7.511886 7.511886
##      1.8      1.9
## 7.511886 7.511886
```

```
# Look at sum of raked weights for raking cells
svytable(~gender+race, drake2)
##           race
## gender      Asian      Black      NatAm      Other      White
## F    53.71714  375.59431  45.55940  13.66782 1021.46870
## M    96.28286  224.40569  54.44060  16.33218 1098.53130
# Look at sum of raked weights for margins
svytotal(~factor(gender),drake2)
##           total      SE
## factor(gender)F    1510 0.0028
## factor(gender)M    1490 0.0028
svytotal(~factor(race),drake2)
##           total SE
## factor(race)Asian    150  0
## factor(race)Black    600  0
## factor(race)NatAm    100  0
## factor(race)Other     30  0
## factor(race)White   2120  0
```

After raking, the standard errors for the population counts for the raking categories equal 0 (approximately 0 for gender since the raking procedure stops when a preset tolerance is reached; you can change that tolerance if desired). After raking, there is no longer any sampling variability for the estimated population counts for each race category—the raking process has forced them to equal the population counts from the external source, which are assumed to be known exactly.

You can also see the raking iterations, if desired, by typing

```
rake(drake, list(~gender,~race), list(pop.gender, pop.race),
     control=list(verbose=TRUE)).
```

Other R functions for nonresponse weight adjustments. The *survey* package has several other functions for nonresponse weight adjustments. The *calibrate* function will perform calibration with general auxiliary information, as described in Section 11.6 of SDA. The *trimWeights* function will trim weights that are outside of user-defined bounds for the weights, and redistribute the trimmings among the untrimmed observations (this keeps the sum of the weights the same). Valliant et al. (2018) go through the detailed steps for constructing weights and describe additional resources for nonresponse weight adjustments using R.

8.3 Imputation

Many data sets have item nonresponse in addition to unit nonresponse. Item nonresponse occurs when a unit has responses for some of the items on the survey but missing values for other items. A person may answer questions about age, race, gender, and health outcomes but decline to answer questions about income. Or the data editing process may uncover logical inconsistencies in the data, such as a 6-year-old who is listed as being married. If the discrepancy cannot be resolved, both age and marital status may be recoded as missing for that person. In this section, we assume that the imputation is limited to missing items.

Example 8.9 of SDA. It is easy to perform simple imputation methods such as cell mean or regression imputation in R. We do not recommend these methods for most applications, since they do not preserve multivariate relationships and do not provide a means of accounting for the imputation when calculating standard errors. But they may be useful when there are, say, a handful of missing items in one of the weighting variables or if it is desired to have an initial imputation before moving on to one of the more advanced imputation methods.

Cell mean imputation is a special case of regression imputation, where the explanatory variables are the factors defining the cells. For cell mean imputation, we first define the cells that cross-classify the observations by gender and age group, then use the *tapply* function to calculate the mean age for the non-missing values in each cell. Note that we also calculate a matrix *impute.flag* that tells which observations were imputed.

```
##### cell mean imputation #####
impute$education
## [1] 16 NA 11 NA 12 NA 20 12 13 10 12 12 11 16 14 11 14 10 12 10
impute.cm<-impute
# define matrix giving imputation flags, TRUE for each missing value
impute.flag<-is.na(impute)
# fit two-way model with interaction, omit NAs from model-fitting
edmodel<-lm(education~factor(gender)*factor(age>=35),
            data=impute.cm,na.action=na.omit)
# replace missing values with imputations from model
newdata<- impute[is.na(impute$education),]
impute.cm$education[is.na(impute$education)] <- predict(edmodel,newdata)
impute.cm$education
## [1] 16.00 12.25 11.00 11.25 12.00 12.25 20.00 12.00 13.00 10.00 12.00 12.00
## [13] 11.00 16.00 14.00 11.00 14.00 10.00 12.00 10.00
```

The other variables can be imputed similarly. Although this simple method might suffice if there are only a handful of missing items to be imputed, it will not preserve multivariate relationships. If you want to perform multivariate analyses of imputed data, the imputation model must include the relationships you will be studying in the analysis model. This requires a much more complicated procedure than a simple cell mean or regression model. The imputer must decide on an imputation method and model(s), and decide how to incorporate survey design features into the imputation (see Little and Vartivarian, 2003; Reiter et al., 2006; Kott, 2012, for discussions of this issue).

For most applications, we recommend using one of the many contributed packages for R that perform imputation for survey data. We briefly describe four of them here; Yadav and Roychoudhury (2018) review additional packages.

- The **Hmisc** package (Harrell, 2021) contains functions for single and multiple imputation using additive regression and predictive mean matching.
- The **VIM** package (Kowarik and Templ, 2016; Templ et al., 2021) performs model-based, hot-deck, and nearest-neighbor imputations. The package also provides tools for exploring the missing data patterns and imputations through visualization.
- The **FHDI** package (Im et al., 2018; Cho et al., 2020) imputes multivariate missing data using the fractional hot deck imputation method described by Kim and Fuller (2004) and Kim (2011). In this method, multiple donors contribute to each imputation.
- The **mice** package (van Buuren et al., 2021) performs multivariate imputation by chained equations. This method relies on a sequence of regression models that predict the missing values for each response variable in turn. This is the package that we usually use when

imputing data. Azur et al. (2011) and van Buuren (2018) describe how to perform imputations using this method.

8.4 Summary, Tips, and Warnings

The **survey** package has functions for raking and poststratification that can be used to perform simple nonresponse adjustments of the weights.

Table 8.1 lists the major R functions used in this chapter.

TABLE 8.1

Functions used for Chapter 8.

Function	Package	Usage
is.na	base	Indicate which values are missing; the function returns TRUE if the value is missing and FALSE otherwise
subset	base	Work with a subset of a vector, matrix, or data frame
complete.cases	stats	Indicate which records have complete data; the function returns a vector with the value TRUE if the record has no missing values and FALSE if at least one item is missing
lm	stats	Fit a linear model to a data set (not using survey methods)
predict	stats	Obtain predicted values from a model object
svydesign	survey	Specify the survey design
svymean	survey	Calculate mean and standard error of mean
svytotal	survey	Calculate total and standard error of total
postStratify	survey	Adjust the sampling weights using poststratification
rake	survey	Carry out poststratification to table margins using raking

Tips and Warnings

- Check how missing values are coded before analyzing your data set, and recode the missing values to NA.
- The functions *postStratify* and *rake* in the **survey** package will perform poststratification adjustments to the weights.
- If the survey has item nonresponse and imputation is not used to fill in missing values, treat the observations with non-missing values as a domain using the *subset* function.
- Several R packages are available that perform imputation. Be aware, though, that imputed values are only as good as the model that produces them. Performing a good imputation requires a lot of expertise; the references in the For Further Reading section of Chapter 8 of SDA can help you get started. See Haziza (2009) for a summary of approaches.

Variance Estimation in Complex Surveys

In this chapter, we discuss variance estimation in complex surveys. We have already seen the use of linearization methods to calculate variances in functions such as *svymean* and *svytotal* in the **survey** package (Lumley, 2020). This chapter will focus on the other methods for calculating variances—random groups, balanced repeated replication (BRR), jackknife, and bootstrap. The code is in file `ch09.R` on the book website.

9.1 Replicate Samples and Random Groups

The random group methods in Section 9.2 of SDA are seldom used in practice. But they are of interest because they motivate how replication methods work in general, and they can be useful for providing quick-and-easy variance estimates for a complex survey design. The methods require three steps:

1. Select the replicate samples from the population, or divide the probability sample among the random groups.
2. Calculate the statistic of interest from each replicate, using the survey weights.
3. Use the estimated statistics from the replicates to calculate the standard error.

Example 9.3 of SDA. In this example, we consider estimating the variance of the ratio of out-of-state tuition fee (*tuitionfee_out*) and in-state tuition fee (*tuitionfee_in*) for a population of public colleges using replicate samples. Data *public_college*, created below, consists of 500 public colleges and universities from the *college* data. This data set serves as the population from which we draw replicate samples.

For Step 1, we use a loop and the *srswor* function (package **sampling**, Tillé and Matei, 2021) to select five independent replicate SRSs, each of size 10, from *public_college* (one could also write a function to do this). We print the values of the selected variables for the colleges in the fifth replicate sample. Note that the weights for each replicate sample sum to the population size, 500. The replicate samples in this book differ from those in SDA, which were selected using SAS software.

For Step 2, compute the statistic of interest from each replicate sample. We calculate the ratio of average out-of-state tuition $ybar[i]$ to average in-state tuition $xbar[i]$ for each replicate sample i by calling the *svymean* function.

Variable *thetahat* contains the values of $\hat{\theta}_i$ for the five replicate samples. These values are the only part of the output used in Step 3; The standard errors given by the *svymean* function are ignored—only the point estimates of the means are used.

For Step 3, treat the 5 estimated ratios in *thetahat* as independent and identically distributed observations, and calculate their mean $\tilde{\theta}$. We also calculate a 95% confidence interval, which can be done either by formula or with the *t.test* function.

```
data(college)
# define population with public colleges and universities
public_college<-college[college$control==1,]
N<-nrow(public_college) #500
# select five SRSs and calculate means
xbar<-rep(NA,5)
ybar<-rep(NA,5)
set.seed(8126834)
for(i in 1:5){
  index <- srswor(10,N)
  replicate <- public_college[(1:N)[index==1],]
  # save replicate in a data frame if you want to keep it for later analyses
  # define design object (since SRS, weights are computed from fpc)
  dcollege<-svydesign(id = ~1, fpc = ~rep(500,10), data = replicate)
  # calculate mean of in-state and out-of-state tuition fees
  xbar[i]<-coef(svymean(~tuitionfee_in, dcollege))
  ybar[i]<-coef(svymean(~tuitionfee_out,dcollege))
}
# print the 5th replicate sample
replicate[,c(2,24:25)]
##
##          instnm tuitionfee_in tuitionfee_out
## 459          Coppin State University          8873          15144
## 474          Towson University          9940          23208
## 556      University of Michigan-Flint          11304          22065
## 674      University of Nevada-Reno          7599          22236
## 735          CUNY Brooklyn College          7240          14910
## 853  University of North Carolina at Greensboro          7331          22490
## 1024  Millersville University of Pennsylvania          12226          22196
## 1030  Pennsylvania State University-Main Campus          18454          34858
## 1359          Texas A&M University-San Antonio          8656          21159
## 1368      University of North Texas at Dallas          9139          21589
# calculate and print the five ratio estimates
thetahat<-ybar/xbar
thetahat
## [1] 2.172545 2.055528 2.107828 2.213799 2.181924
# calculate mean of the five ratio estimates, and SE
thetatilde<-mean(thetahat)
thetatilde
## [1] 2.146325
setheta<-sqrt(var(thetahat)/5)
# calculate confidence interval by direct formula using t distribution
c( thetatilde- qt(.975, 4)*setheta, thetatilde+ qt(.975, 4)*setheta)
## [1] 2.067224 2.225426
# easier: use t.test function to calculate mean and confidence interval
t.test(thetahat)
##
##  One Sample t-test
##
## data:  thetahat
## t = 75.336, df = 4, p-value = 1.861e-07
## alternative hypothesis: true mean is not equal to 0
```

```
## 95 percent confidence interval:
## 2.067224 2.225426
## sample estimates:
## mean of x
## 2.146325
```

The output shows that the mean of the ratios estimated from the five replicate samples is 2.146 and the 95% confidence interval for the population ratio is [2.067, 2.225]. The 95% confidence interval is calculated as $\tilde{\theta} \pm t \text{SE}(\tilde{\theta})$, where t is the critical value from a t distribution with 4 (number of replicates minus one) degrees of freedom (df). This critical value is 2.78, giving a wider interval than would be obtained from a variance estimate with more df.

In Example 9.3, the survey weights are the only design feature used for the calculations. Even if a replicate sample has stratification or clustering, that information is not needed to calculate the point estimate of the parameter of interest θ for the replicates. The effect of the stratification or clustering on the variance is incorporated in the variability among the $\hat{\theta}_i$'s. For example, if clustering decreases the precision for $\hat{\theta}_i$, then the estimates $\hat{\theta}_i$ will vary more from replicate to replicate, and the decreased precision will show up in a large value of the sample variance for the replicate values $\hat{\theta}_i$.

We illustrated the method for estimating a ratio, but the same method can be used for any statistic you would like to estimate. The method can also be applied to statistics that are not calculated by the survey analysis functions. All you need to do is to calculate the statistic of interest for each replicate using the survey weights, then apply t confidence interval methods to the statistics calculated from the replicates.

Random groups. After dividing the survey data into R random groups, the procedure for calculating the estimator and variance is exactly the same as Steps 1 to 3 for replicate sample methods. Note that if you are estimating population totals, you need to scale the weights for each random group so they sum to the population size.

Example 9.4 of SDA. This example illustrates the random group method with the Survey of Youth in Custody (*syc*) data. The whole sample is divided into seven random groups by variable *randgrp*. The *svyby* function computes the mean of variable *age* using *svymean* separately for each random group. Each is calculated using the survey weight *finalwt*.

Only the point estimates for *age*, printed in *repmean*, are needed to calculate the mean and confidence interval. The standard errors are not used. Thus, although the survey has stratification and clustering, we do not need to include that information in the *svydesign* function when calculating variances with the random groups method—the estimated means of each group depend only on the weights in *finalwt*.

```
data(syc)
dsyc<-svydesign(id = ~1, weights = ~finalwt, data = syc)
repmean<-svyby(~age, ~randgrp, dsyc, svymean)
repmean # we use only the means, not the SEs
##   randgrp   age      se
## 1      1 16.54947 0.1171541
## 2      2 16.66331 0.1133751
## 3      3 16.82544 0.1242695
## 4      4 16.05688 0.1240046
## 5      5 16.31776 0.1160307
## 6      6 17.02798 0.1181861
## 7      7 17.26605 0.1110258
```

Let $\hat{\theta}_r$ be the estimate from the r th replicate sample, $r = 1, 2, \dots, 7$, $\tilde{\theta}$ be the mean of the seven random group means, and $\hat{\theta}$ be the estimated mean age using the whole data set *sync*. We can calculate two variance estimates: *SEthetatilde* calculates the square root of

$$\hat{V}_1(\tilde{\theta}) = \frac{1}{R} \frac{1}{R-1} \sum_{r=1}^R (\hat{\theta}_r - \tilde{\theta})^2 \quad (9.1)$$

and *SEthetahat* calculates the square root of

$$\hat{V}_2(\hat{\theta}) = \frac{1}{R} \frac{1}{R-1} \sum_{r=1}^R (\hat{\theta}_r - \hat{\theta})^2. \quad (9.2)$$

```
# Estimate and SE 1 (could also use t.test function)
thetatilde<-mean(repmean$age)
SEthetatilde<- sqrt( (1/7)*var(repmean$age) )
# Estimate and SE 2
thetahat<-coef(svymean(~age,dsync))
SEthetahat<- sqrt((1/7)*(1/6)*sum((repmean$age-thetahat)^2))

#calculate confidence interval by direct formula using t distribution
Mean_CI1 <- c(thetatilde, SEthetatilde, thetatilde- qt(.975, 7-1)*SEthetatilde,
              thetatilde+ qt(.975, 7-1)*SEthetatilde)
names(Mean_CI1) <- c("thetatilde","SE","lower CL", "upper CL")
Mean_CI1
## thetatilde          SE    lower CL    upper CL
## 16.6724103  0.1559995 16.2906932 17.0541274
Mean_CI2 <- c(thetahat,SEthetahat, thetahat- qt(.975, 7-1)*SEthetahat,
              thetahat+ qt(.975, 7-1)*SEthetahat)
names(Mean_CI2) <- c("thetahat","SE","lower CL", "upper CL")
Mean_CI2
## thetahat          SE    lower CL    upper CL
## 16.6392931  0.1565843 16.2561452 17.0224411
```

The estimated mean of age using *thetatilde* is 16.67241 with a confidence interval of [16.29069, 17.05413]. The estimated mean of age using *thetahat* is 16.63929 with a confidence interval of [16.25615, 17.02244]. A t distribution with 6 (the number of random groups minus 1) df is used. Again, though, with few random groups, the confidence interval is wider than it would be if a variance estimation method having more df were used. We discuss such methods in the next section.

9.2 Constructing Replicate Weights

Replicate weights for balanced repeated replication (BRR), jackknife, and bootstrap methods can be created with the *as.svrepdesign* function in the *survey* package (Lumley, 2020). It creates a replicate-weights survey design object from a survey design object that contains stratification and clustering information.

The basic function call, shown here for BRR, is

```
as.svrepdesign(design.object, type = "BRR")
```

where *design.object* is a design object that has been created using *svydesign* (containing the stratification and clustering information) and *type* specifies the replication method to be used.

9.2.1 Balanced Repeated Replication

Example 9.5 of SDA. This example shows how to use BRR to calculate variances for the small data set in Table 9.2 of SDA. Here, we assume that $N = 10,000$. Variable *wt* contains the sampling weight, which is $N_h/2$ for this sample with $n_h = 2$ sampled observations per stratum and is obtained by $N(N_h/N)/n_h = (N)(strfrac)/2$. We request *type*="BRR" in the *as.svrepdesign* function.

The function creates 8 replicate weights because that is the smallest multiple of 4 that is larger than 7, the number of strata.

```
brrex<-data.frame(strat = c(1,1,2,2,3,3,4,4,5,5,6,6,7,7),
  strfrac =c(0.3,0.3,0.1,0.1,0.05,0.05,0.1,0.1,0.2,0.2,0.05,0.05,0.2,0.2),
  y =c(2000,1792,4525,4735,9550,14060,800,1250,9300,7264,13286,12840,2106,2070)
)

brrex$wt <- 10000*brrex$strfrac/2
brrex
##      strat strfrac      y   wt
## 1         1    0.30  2000 1500
## 2         1    0.30  1792 1500
## 3         2    0.10  4525   500
## 4         2    0.10  4735   500
## 5         3    0.05  9550   250
## 6         3    0.05 14060   250
## 7         4    0.10   800   500
## 8         4    0.10  1250   500
## 9         5    0.20  9300  1000
## 10        5    0.20  7264  1000
## 11        6    0.05 13286   250
## 12        6    0.05 12840   250
## 13        7    0.20  2106  1000
## 14        7    0.20  2070  1000
dbrrex<-svydesign(id=~1, strata=~strat,weights=~wt,data=brrex)
dbrrex # stratified random sample
## Stratified Independent Sampling design (with replacement)
## svydesign(id = ~1, strata = ~strat, weights = ~wt, data = brrex)
# convert to BRR replicate weights
dbrrexbrr <- as.svrepdesign(dbrrex, type="BRR")
dbrrexbrr # identifies as BRR
## Call: as.svrepdesign(dbrrex, type = "BRR")
## Balanced Repeated Replicates with 8 replicates.
# now use the replicate weights to calculate the mean and confidence interval
svymean(~y,dbrrexbrr)
##      mean      SE
## y 4451.7 236.42
degf(dbrrexbrr)
## [1] 7
confint(svymean(~y,dbrrexbrr),df=7)
##      2.5 %    97.5 %
## y 3892.664 5010.736
```

The estimated mean value of y is 4451.7 with a standard error of 236.42, and a confidence interval of [3892.664, 5010.736].

You can also look at the replicate weights multiplier if desired. Note that the two observations in each stratum have complementary patterns.

```
# can look at replicate weight multiplier if desired
# this is to illustrate weight structure; it is not needed for data analysis
dbrrexbr$repweights$weights
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
## [1,]    2    0    2    0    2    0    2    0
## [2,]    0    2    0    2    0    2    0    2
## [3,]    2    2    0    0    2    2    0    0
## [4,]    0    0    2    2    0    0    2    2
## [5,]    2    0    0    2    2    0    0    2
## [6,]    0    2    2    0    0    2    2    0
## [7,]    2    2    2    2    0    0    0    0
## [8,]    0    0    0    0    2    2    2    2
## [9,]    2    0    2    0    0    2    0    2
## [10,]   0    2    0    2    2    0    2    0
## [11,]   2    2    0    0    0    0    2    2
## [12,]   0    0    2    2    2    2    0    0
## [13,]   2    0    0    2    0    2    2    0
## [14,]   0    2    2    0    2    0    0    2
```

Fay's method for BRR. The *as.svrepdesign* function will also construct replicate weights using Fay's variation of BRR (Dippo et al., 1984; Judkins, 1990). Simply specify `type="Fay"` instead of `type="BRR"`. To set ϵ for Fay's method equal to 0.5, add the argument `fay.rho = 0.5`. This results in weight multipliers 1.5 and 0.5 instead of the multipliers 0 and 2 for the BRR example above.

```
dbrrex Fay <- as.svrepdesign(dbrrex, type="Fay", fay.rho=0.5)
svymean(~y, dbrrex Fay)
##      mean      SE
## y 4451.7 236.42
confint(svymean(~y, dbrrex Fay), df=7)
##      2.5 %   97.5 %
## y 3892.664 5010.736
# look at replicate weights for contrast with regular BRR
# note values for replicate weight multiplier are now 1.5 and 0.5
dbrrex Fay$repweights$weights
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
## [1,] 1.5 0.5 1.5 0.5 1.5 0.5 1.5 0.5
## [2,] 0.5 1.5 0.5 1.5 0.5 1.5 0.5 1.5
## [3,] 1.5 1.5 0.5 0.5 1.5 1.5 0.5 0.5
## [4,] 0.5 0.5 1.5 1.5 0.5 0.5 1.5 1.5
## [5,] 1.5 0.5 0.5 1.5 1.5 0.5 0.5 1.5
## [6,] 0.5 1.5 1.5 0.5 0.5 1.5 1.5 0.5
## [7,] 1.5 1.5 1.5 1.5 0.5 0.5 0.5 0.5
## [8,] 0.5 0.5 0.5 0.5 1.5 1.5 1.5 1.5
## [9,] 1.5 0.5 1.5 0.5 0.5 1.5 0.5 1.5
## [10,] 0.5 1.5 0.5 1.5 1.5 0.5 1.5 0.5
## [11,] 1.5 1.5 0.5 0.5 0.5 0.5 1.5 1.5
## [12,] 0.5 0.5 1.5 1.5 1.5 1.5 0.5 0.5
## [13,] 1.5 0.5 0.5 1.5 0.5 1.5 1.5 0.5
## [14,] 0.5 1.5 1.5 0.5 1.5 0.5 0.5 1.5
```

Example 9.6 of SDA. Now let's create BRR weights for a data set with stratification and clustering: the NHANES data. First, use *svydesign* to include primary sampling unit (psu) *sdmvpstu*, stratum *sdmvstra*, and weight *wtmec2yr* information, and then use the *as.svrepdesign* function to construct replicate weights using BRR by adding `type="BRR"`. By specifying the stratification and clustering information through *svydesign*, you ensure that observations in the same psu are kept together during the replicate weight construction.

This analysis finds summary statistics of body mass index (BMI, in variable *bmxbmi*) for adults age 20 and over, so the *subset* function is also used. Note that the df for the subset is the same as for the full sample—there are members from this domain in each psu, so there are (number of psus – number of strata) df.

```
data(nhanes)
nhanes$age20d<-rep(0,nrow(nhanes))
nhanes$age20d[nhanes$ridageyr >=20 & !is.na(nhanes$bmxbmi)]<-1
dnhanes<-svydesign(id=~sdmvpstu, strata=~sdmvstra,nest=TRUE,
                  weights=~wtmec2yr,data=nhanes)
dnhanesbrr <- as.svrepdesign(dnhanes, type="BRR")
# look at subset of adults age 20+
dnhanesbrrsub<-subset(dnhanesbrr, age20d == '1')
degf(dnhanes)
## [1] 15
degf(dnhanesbrrsub) # same df
## [1] 15
# calculate mean
bmimean<-svymean(~bmxbmi, dnhanesbrrsub)
bmimean
##           mean      SE
## bmxbmi 29.389 0.261
confint(bmimean,df=15)
##           2.5 %   97.5 %
## bmxbmi 28.83279 29.94541
# calculate quantiles
svyquantile(~bmxbmi, dnhanesbrrsub, quantiles=c(0.25,0.5,0.75,0.95),
            ties = "rounded")
## Statistic:
##           bmxbmi
## q0.25 24.35349
## q0.5  28.23490
## q0.75 33.06615
## q0.95 42.64092
## SE:
##           bmxbmi
## q0.25 0.2215986
## q0.5  0.3241246
## q0.75 0.3139102
## q0.95 0.3436826
```

The mean and quantiles are estimated using the full sample weights, and the replicate weights are used to calculate the standard errors. The estimated mean BMI for adults is 29.389 with a standard error of 0.261 and a 95% confidence interval of [28.833, 29.945]. You can also include optional argument `return.replicates=TRUE` in the *svymean* statement in case you want to look at the statistics calculated for each replicate.

Note that these replicate weights, constructed from the NHANES final weights available on the public-use data file, do not account for the effects of poststratification on the variances.

See Section 9.2.4 for how to calculate replicate weights so that the variance estimates include the effects of weighting adjustments.

Domain estimates with replicate weights. We carried out this analysis on the subset of observations with *age20d*=1. The *subset* function is used exactly the same way with a replicate-weight design object as with a strata/cluster design object. Make sure, though, that you define the replicate-weight design object on the full data set first, and then use the *subset* function to define the domain. This ensures that the full design information is used to calculate standard errors.

9.2.2 Jackknife

The **survey** package will create replicate weights for two types of jackknife: JK1 and JK_n. These are described in detail by Brick et al. (2000) and Chapter 15 of Valliant et al. (2018). Briefly, JK_n is the jackknife for stratified multistage sampling described in Chapter 9 of SDA. If there are n_h psus in stratum h and observation i has weight w_i , then the JK_n jackknife weights are defined as follows:

$$w_{i(hj)} = \begin{cases} w_i & \text{if observation unit } i \text{ is not in stratum } h \\ 0 & \text{if observation unit } i \text{ is in psu } j \text{ of stratum } h \\ \frac{n_h}{n_h - 1} w_i & \text{if observation unit } i \text{ is in stratum } h \text{ but not in psu } j. \end{cases} \quad (9.3)$$

The weights $w_{i(hj)}$ are used to calculate $\hat{\theta}_{(hj)}$ for each replicate, and

$$\hat{V}_{\text{JK}_n}(\hat{\theta}) = \sum_{h=1}^H \frac{n_h - 1}{n_h} \sum_{j=1}^{n_h} \left(\hat{\theta}_{(hj)} - \hat{\theta} \right)^2. \quad (9.4)$$

Jackknife JK1 is a special case of JK_n where there is one stratum.

Jackknife weights for an SRS. Let's start by looking at how the *as.svrepdesign* function works using jackknife variance estimation for an SRS, then move on to complex sample designs.

Example 9.7 of SDA. Data *collegerg* shows the values of in-state and out-of-state tuition for five replicate samples, each of size 10 (these samples were selected with SAS software, and differ from the replicate samples selected earlier in this chapter). We define data **collegerg1** to be replicate sample 1 (having *repgroup* = 1).

For comparison purposes, we first look at the linearization (Taylor series) variance estimate of the mean in-state tuition, mean out-of-state tuition, and ratio of mean out-of-state tuition to mean in-state tuition. Note that we did not include the *fpc* argument in *svydesign*, so the with-replacement variance is calculated.

```
data(collegerg)
collegerg1<-collegerg[collegerg$repgroup==1,]
collegerg1$sampwt<-rep(500/10,10)
# calculate SEs of means and ratio using linearization
dcollegerg1<-svydesign(id=~1, weights=~sampwt,data=collegerg1)
means.lin<-svymean(~tuitionfee_in+tuitionfee_out, dcollegerg1)
means.lin
##               mean      SE
## tuitionfee_in  8913.3  454.46
## tuitionfee_out 21614.7 2325.15
```

```

confint(means.lin,df=degf(dcollegerg1))
##                2.5 %    97.5 %
## tuitionfee_in   7885.247  9941.353
## tuitionfee_out 16354.843 26874.557
ratio.lin<-svyratio(~tuitionfee_out,~tuitionfee_in,dcollegerg1)
ratio.lin
## Ratio estimator: svyratio.survey.design2(~tuitionfee_out, ~tuitionfee_in,
##      dcollegerg1)
## Ratios=
##          tuitionfee_in
## tuitionfee_out      2.424994
## SEs=
##          tuitionfee_in
## tuitionfee_out      0.2311776
confint(ratio.lin,df=degf(dcollegerg1))
##                2.5 %    97.5 %
## tuitionfee_out/tuitionfee_in 1.902034 2.947954

```

Now let's calculate the jackknife variance by omitting observation j in replicate j . We use function *as.svrepdesign* with `type="JK1"` to create jackknife weights for this SRS of size 10. We then use *svymean* and *svyratio* to calculate the standard errors of the estimated means and ratio with the jackknife weights.

```

## define jackknife replicate weights design object
dcollegerg1jk <- as.svrepdesign(dcollegerg1, type="JK1")
dcollegerg1jk
## Call: as.svrepdesign(dcollegerg1, type = "JK1")
## Unstratified cluster jackknife (JK1) with 10 replicates.
# now look at jackknife SE for means
# these are same as linearization since SRS and statistic = mean
svymean(~tuitionfee_in + tuitionfee_out, dcollegerg1jk)
##          mean          SE
## tuitionfee_in   8913.3  454.46
## tuitionfee_out 21614.7 2325.15
# jackknife SE for ratio
svyratio(~tuitionfee_out, ~tuitionfee_in, design = dcollegerg1jk)
## Ratio estimator: svyratio.svyprep.design(~tuitionfee_out, ~tuitionfee_in,
##      design = dcollegerg1jk)
## Ratios=
##          tuitionfee_in
## tuitionfee_out      2.424994
## SEs=
##          [,1]
## [1,] 0.2314828

```

The above output shows the statistics produced by the *svymean* and *svyratio* functions using linearization and jackknife methods. Typing `dcollegerg1jk` shows that this unstratified jackknife (JK1) has 10 replicates. The jackknife standard errors for the estimated means of *tuitionfee_in* and *tuitionfee_out* are 454.46 and 2325.15, respectively. These are the same as the linearization standard errors because $\hat{V}_{JK}(\bar{y}) = s_y^2/n$ for an SRS, as shown in Section 9.3 of SDA. The jackknife standard error for the nonlinear statistic of the ratio, 0.231483, differs slightly from the linearization standard error of 0.231178. These values are extremely close—after all, the linearization variance and the jackknife variance are both consistent estimators for $V(\hat{B})$ —but are not exactly the same. A t distribution with $n - 1 = 9$ df is used for the confidence interval.

For an SRS such as this, the jackknife weights are calculated by setting the weight of observation j to zero and assigning weight $w_i \times n/(n-1) = (500/10) * (10/9) = 55.555$ to each of the other observations. You can print the replicate weight multipliers for each observation if you want (although you typically do not need to do this when carrying out a data analysis).

```
# can look at replicate weight multipliers if desired
# note that observation being omitted for replicate has weight 0
# weight multiplier for other observations is 10/9 = 1.11111
round(dcollegerg1jk$repweights$weights,digits=4)
##           [,1]  [,2]  [,3]  [,4]  [,5]  [,6]  [,7]  [,8]  [,9]  [,10]
## [1,] 0.0000 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111
## [2,] 1.1111 0.0000 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111
## [3,] 1.1111 1.1111 0.0000 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111
## [4,] 1.1111 1.1111 1.1111 0.0000 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111
## [5,] 1.1111 1.1111 1.1111 1.1111 0.0000 1.1111 1.1111 1.1111 1.1111 1.1111
## [6,] 1.1111 1.1111 1.1111 1.1111 1.1111 0.0000 1.1111 1.1111 1.1111 1.1111
## [7,] 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 0.0000 1.1111 1.1111 1.1111
## [8,] 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 0.0000 1.1111 1.1111
## [9,] 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 0.0000 1.1111
## [10,] 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 1.1111 0.0000
```

Jackknife weights for a complex survey. For a survey with stratification and clustering, you need to include the stratum and cluster information in the *svydesign* function. The replicate weights are then constructed by deleting one psu at a time from each stratum.

Example 9.8 of SDA. In this example, we consider a two-stage cluster sample. Because *clutch* is the primary sampling unit (psu), the procedure deletes one clutch at a time rather than one observation at a time.

The sampling weight *relwt* is defined as $M_i/m_i = \text{csize}/2$, the number of eggs in the clutch divided by the number of eggs selected from the clutch for measurement. The *as.svrepdesign* function sets the replicate weight variable equal to 0 for all observations from the psu being deleted in that replicate. This design has one stratum, so we use `type="JK1"`; if your survey has stratification, use `type="JKn"`.

```
data(coots)
coots$relwt<-coots$csize/2
dcoots<-svydesign(id=~clutch,weights=~relwt,data=coots)
dcootsjk <- as.svrepdesign(dcoots, type="JK1")
svymean(~volume,dcootsjk)
##           mean      SE
## volume 2.4908 0.061
confint(svymean(~volume,dcootsjk),df=degf(dcootsjk))
##           2.5 %    97.5 %
## volume 2.370354 2.611203
```

The estimated mean volume is 2.49 with a standard error of 0.061 by the jackknife method. The confidence interval for the mean volume is [2.370354, 2.611203], calculated using 183 (number of psus minus 1) df.

9.2.3 Bootstrap

Example 9.9 of SDA. This example looks at creating bootstrap weights using the method in Rao et al. (1992) to estimate the population distribution of height from the SRS in data set *htsrs*. The number of replicates used is specified by `replicates = 1000` in the function

as.svrepdesign with `type = "subbootstrap"`. You will need to use *set.seed* to be able to reconstruct the same set of bootstrap weights later.

The weight variable *wt* equals 10 for each observation. In each replicate, if observation *i* is selected *m* times; the weight in the replicate for that observation is, using the formula in Section 9.3 of SDA, $10 \times (200/199) \times m$. The *df* with the bootstrap method should be set equal to the sample size minus one, which is $200 - 1 = 199$, not the number of bootstrap replicates, which can be set to any value.

```
data(htsrs)
nrow(htsrs)
## [1] 200
wt<-rep(10,nrow(htsrs))
dhtsrs<-svydesign(id=~1, weights=~wt,data=htsrs)
set.seed(9231)
dhtsrsboot <- as.svrepdesign(dhtsrs, type="subbootstrap",replicates=1000)
svymean(~height,dhtsrsboot)
##          mean          SE
## height 168.94 0.7978
degf(dhtsrsboot) # 199 = n - 1
## [1] 199
confint(svymean(~height,dhtsrsboot),df=degf(dhtsrsboot))
##          2.5 %    97.5 %
## height 167.3667 170.5133
svyquantile(~height, dhtsrsboot, quantile=c(0.25,0.5,0.75), ties=c("rounded"))
## Statistic:
##          height
## q0.25 159.70
## q0.5  168.75
## q0.75 176.00
## SE:
##          height
## q0.25 0.8878743
## q0.5  0.9930391
## q0.75 1.1339674
```

The output above shows the summary statistics of the mean, selected quantiles, and associated standard errors calculated using the bootstrap method with seed 9231.

Example 9.10 of SDA. The code for creating bootstrap weights in a complex survey design is similar to that for an SRS. The survey design is specified by *svydesign*, and the *as.svrepdesign* function is used to create the replicate weights. Data set *htstrat* is a stratified random sample, so *svydesign* requires only the stratum information. When the *weights* argument is not supplied, the function calculates the weights from the information in the *fpc* argument.

```
data(htstrat)
nrow(htstrat)
## [1] 200
dhtstrat <- svydesign(id = ~1, strata = ~gender, fpc = c(rep(1000,160),rep(1000,40)),
                    data = htstrat)
set.seed(982537455)
dhtstratboot <- as.svrepdesign(dhtstrat, type="subbootstrap",replicates=1000)
svymean(~height,dhtstratboot)
##          mean          SE
## height 169.02 0.7296
degf(dhtstratboot)
```

```
## [1] 198
confint(svymean(~height,dhtstratboot),df=degf(dhtstratboot))
##          2.5 %    97.5 %
## height 167.5769 170.4543
```

The estimated mean height is 169.02 with a standard error of 0.7296 by the bootstrap method. The 95% confidence interval for mean height is [167.5769, 170.4543], calculated using a t distribution with 198 df (sample size minus number of strata, $200 - 2$). If you specify the bootstrap design object with a different random number seed, you will obtain a slightly different value for the standard error because a different set of bootstrap samples will be used for the variance calculations.

The *as.svrepdesign* function will also construct bootstrap replicate weights for other forms of bootstrap: `type="bootstrap"` uses the method of Canty and Davison (1999) and `type="mrbbootstrap"` uses the multistage rescaled bootstrap of Preston (2009).

9.2.4 Replicate Weights and Nonresponse Adjustments

The code given so far in this section constructs replicate sampling weights. When nonresponse adjustments are made to the final weights, as described in Chapter 8 of SDA, the steps of weighting class adjustments, poststratification, raking, and other adjustments that are used on the final weights need to be repeated for each replicate weight column.

For example, the combination of the *as.svrepdesign* and *postStratify* functions will create replicate weights that reflect poststratification weight adjustments. Let's look at that for the poststratified weights in Example 4.9 of SDA, which we discussed in Section 4.4.

Example 4.9 of SDA. Let's look at creating poststratified replicate weights for an SRS. Here are the steps:

1. Define a design object with the sampling weights, stratification, and clustering. Here, the object *dsrs* is an SRS with weights 3078/300. Because most replicate weight methods calculate the with-replacement variance, only the first-stage strata and psu information need to be supplied.
2. Choose the variance estimation method (here, JK1), and create a replicate-weights design object.
3. Apply poststratification (function *postStratify*) to the replicate-weights design object. This will poststratify the sampling weights and each variable of replicate weights.

```
data(agsrs)
# define design object for sample
dsrs <- svydesign(id = ~1, weights=rep(3078/300,300), data = agsrs)
# define replicate weights design object
dsrsjk<-as.svrepdesign(dsrs,type="JK1")
# poststratify on region
pop.region <- data.frame(region=c("NC","NE","S","W"), Freq=c(1054,220,1382,422))
dsrsjk<-postStratify(dsrsjk, ~region, pop.region)
svymean(~acres92, dsrsjk)
##          mean      SE
## acres92 299778 18653
confint(svymean(~acres92, dsrsjk),df=degf(dsrsjk))
##          2.5 %    97.5 %
## acres92 263069.2 336487
```

```
svytotal(~acres92, dsrspjk)
##              total      SE
## acres92 922717031 57413300
# Check: estimates of counts in poststrata = pop.region counts with SE = 0
svytotal(~factor(region), dsrspjk)
##              total SE
## factor(region)NC  1054  0
## factor(region)NE   220  0
## factor(region)S   1382  0
## factor(region)W    422  0
```

The estimated mean value of *acres92* is 299,778 with a standard error of 18,653, where the jackknife replicate weights are poststratified. Recall that in example 4.9, the poststratified standard error of the mean of *acres92* is 17,513—this value is smaller because it included a finite population correction (fpc), while the replicate weight methods calculate the with-replacement variance.

Note that the final poststratified weights, and each poststratified replicate weight variable, sum to 1054 for the NC region, to 220 for the NE region, to 1382 for the S region, and to 422 for the W region—exactly the poststratification totals defined in the data set *pop.region*. After poststratification, there is no sampling variability for the variables used in the poststratification. Variables associated with the poststratification variables are expected to have reduced variance as well.

Performing multiple steps of nonresponse adjustments. Many surveys have several steps of weighting class adjustments followed by calibration; sometimes intermediate or final weights are trimmed or smoothed so that the weight adjustments do not have “spikes” for some observations. Each step must be repeated for each replicate sampling weight.

1. Start with the sampling weight vector $\mathbf{w}$, which is calculated as the inverse of the probability of selection for each member of the selected sample, whether respondent or nonrespondent.
2. Create R replicate sampling weights $\mathbf{w}_1, \dots, \mathbf{w}_R$ using the desired replication variance estimation method; if desired, this can be done using the *as.svyrepdesign* function.
3. Now carry out each step of the weight adjustments—weighting class adjustments, weight smoothing or trimming, propensity score adjustments, calibration, or other methods that may be used—on the sampling weight. The final weight variable results from this process.
4. Repeat the operations in Step 3 for each column of replicate weights. Each additional step in the weighting adjustments needs to be carried out separately on each replicate weight.

Some of the functions in the *survey* package, such as *postStratify*, will create replicate weights for the nonresponse adjustments they carry out. For a complicated weighting procedure, we prefer constructing the weights and replicate weights in a custom-written program so that we can see the adjustments at each step.

9.3 Using Replicate Weights from a Survey Data File

Section 9.2 gave examples of how to create your own replicate weights for a survey from the sampling weights, stratification, and clustering information.

Many survey organizations supply data files for which the replicate weights have already been created. These replicate weights have usually already accounted for the nonresponse weighting adjustments. You can analyze these files with the `survey` package, too.

Example 9.5 of SDA. Here, let's suppose that the BRR weights we created for the simple data set in Table 9.2 of SDA have been stored in an external data set and then imported into R. We create `brrdf` using the replicate weights from the design object `dbrrrexbr` that we formed in Section 9.2.1, but then use `brrdf` alone to illustrate how the function `svrepdesign` will create a design object from an imported data set that has replicate weights.

```
# Create data frame containing final and replicate weights, and y
repwts<- dbrrrexbr$repweights$weights * matrix(brrex$wt,nrow=14,ncol=8,byrow=FALSE)
brrdf<-data.frame(y=brrex$y,wt=brrex$wt,repwts)
colnames(brrdf)<-c("y","wt",paste("repwt",1:8,sep=""))
brrdf # contains weight, repwt1-repwt8, and y but no stratum info
##          y      wt repwt1 repwt2 repwt3 repwt4 repwt5 repwt6 repwt7 repwt8
## 1    2000  1500    3000      0   3000      0   3000      0   3000      0
## 2    1792  1500      0   3000      0   3000      0   3000      0   3000
## 3    4525   500   1000   1000      0      0   1000   1000      0      0
## 4    4735   500      0      0   1000   1000      0      0   1000   1000
## 5    9550   250   500      0      0   500   500      0      0   500
## 6   14060   250      0   500   500      0      0   500   500      0
## 7     800   500  1000  1000  1000  1000      0      0      0      0
## 8    1250   500      0      0      0      0  1000  1000  1000  1000
## 9    9300  1000  2000      0  2000      0      0  2000      0  2000
## 10   7264  1000      0  2000      0  2000  2000      0  2000      0
## 11  13286   250   500   500      0      0      0      0   500   500
## 12  12840   250      0      0   500   500   500   500      0      0
## 13   2106  1000  2000      0      0  2000      0  2000  2000      0
## 14   2070  1000      0  2000  2000      0  2000      0      0  2000
# create design object
dbrrdf<-svrepdesign(weights=~wt,repweights="repwt[1-9]",data=brrdf,type="BRR")
dbrrdf
## Call: svrepdesign.default(weights = ~wt, repweights = "repwt[1-9]",
##      data = brrdf, type = "BRR")
## Balanced Repeated Replicates with 8 replicates.
svymean(~y,dbrrdf) # same as before!
##      mean      SE
## y 4451.7 236.42
```

In the `svrepdesign` function, we specify the weight variable and tell the function that the replicate weights are in variables whose names are of the form “`repwt`” followed by a number. The argument `type="BRR"` indicates that the replicate weights were formed using the BRR method. The function requires only `weights=` and `repweights=` arguments to calculate point estimates and standard errors. The point estimates are calculated using the full sample weights (in the `weights=` argument), and the replicate weights are used to calculate standard errors. No stratification or clustering information is supplied to the function.

The *svrepdesign* function can also accommodate replicate weights with types JK1, JK2 (a version of jackknife for two-psu-per-stratum designs; see Brick et al., 2000), JKn, bootstrap, the Fay variant of BRR, and other structures. The producer of the survey you are analyzing will include information about the type of replicate weights that were produced and any special considerations that are needed for analysis.

9.4 Summary, Tips, and Warnings

Table 9.1 lists the major R functions used in this chapter.

TABLE 9.1

Functions used for Chapter 9.

Function	Package	Usage
<code>subset</code>	base	Work with a subset of a vector, matrix, or data frame
<code>srswor</code>	sampling	Select an SRS without replacement
<code>svydesign</code>	survey	Specify the survey design
<code>svymean</code>	survey	Calculate mean and standard error of mean
<code>svyratio</code>	survey	Calculate ratio and standard error of ratio
<code>svyquantile</code>	survey	Calculate quantiles and their standard errors
<code>as.svrepdesign</code>	survey	Creates a replicate-weights survey design object from a design object that includes weighting, stratification, and clustering information
<code>svrepdesign</code>	survey	Creates a replicate-weights survey design object from a data frame that contains columns for final and replicate weights

Tips and Warnings

- To create replicate weights for a complex survey for which stratification and clustering information is available, first create a design object with the *svydesign* function, then convert it to a replicate-weights survey design object with the *as.svrepdesign* function.
- If performing nonresponse adjustments, do the adjustments on the sampling weight variable and then on each replicated sampling weight. Then standard errors calculated using the replicate weights will account for the weighting adjustments.
- To analyze data for which replicate weights have already been supplied, create the survey design object with the *svrepdesign* function. You do not need stratification or clustering information to analyze survey data when you have the replicate weights.
- Use the *subset* function to analyze data for a domain. When using replication variance estimation, create the replicate-weights design object first, then apply the *subset* function.

Categorical Data Analysis in Complex Surveys

The functions *svytable*, *svychisq*, and *svyloglin* from the **survey** package (Lumley, 2020) perform categorical data analyses on survey data. In this chapter, we present several examples to illustrate the usage of these functions. The code is in file **ch10.R** on the book website.

10.1 Contingency Tables and Odds Ratios

First, let's look at the contingency table and odds ratio for a simple random sample (SRS).

Example 10.1 of SDA. Data set *cablecomp* is created from the category counts in Example 10.1 of SDA. Each household in the sample gives the computer (Yes or No) and cable (Yes or No) status.

```
# create the categorical table (Table 10.1)
cablecomp<-matrix(c(119,188,88,105), ncol=2, byrow=2)
dimnames(cablecomp)<-list(Cable=c("yes", "no"),
                           Computer=c("yes","no"))

cablecomp
##      Computer
## Cable yes  no
##   yes 119 188
##   no   88 105
```

Are households with a computer more likely to subscribe to cable? A chi-square test for independence is often used for such questions. Using the function *chisq.test* (without the continuity correction), the Pearson's chi-square test statistic X^2 is 2.281. For large samples, X^2 approximately follows a chi-square (χ^2) distribution with $(r - 1) * (c - 1)$ degrees of freedom (df) under the null hypothesis, where r and c are the number of rows and columns in the contingency table. In this case, $df = 1$. The p -value for the X^2 statistic is 0.13, giving no reason to doubt the null hypothesis that owning a computer and subscribing to cable television are independent.

```
# Pearson's chi-square test under multinomial sampling, obtain X^2
cablechisq<-chisq.test(cablecomp,correct=F)
cablechisq
##
##  Pearson's Chi-squared test
##
## data:  cablecomp
## X-squared = 2.281, df = 1, p-value = 0.131
## Expected values under null hypothesis
cablechisq$expected
##      Computer
## Cable   yes   no
```

```
##   yes 127.098 179.902
##   no  79.902 113.098
# odds ratio
(cablecomp[1,1]/cablecomp[1,2])/(cablecomp[2,1]/cablecomp[2,2])
## [1] 0.7552587
```

We estimate the odds of owning a computer if the household subscribes to cable as 119/188 and estimate the odds of owning a computer if the household does not subscribe to cable as 88/105. The odds ratio is therefore estimated as

$$(119/188)/(88/105) = 0.755.$$

Contingency tables for data from a complex survey. The only differences between constructing contingency tables and computing odds ratios for an SRS and doing so for a complex sample are that for the complex sample, we include the design information in function *svydesign* and use *svytable* to calculate the weighted counts.

Example 10.5 of SDA. This example shows how to use functions *svydesign* and *svytable* to produce statistics for a two-factor contingency table when observations are from a stratified multistage sample—in this case, from the Survey of Youth in Custody (*syc*) data.

The following two variables are of interest. Variable *famtime* denotes “Was anyone in your family ever incarcerated?” with 2 corresponding to No and 1 corresponding to Yes; and variable *everviol* refers to the question “Have you ever been put on probation or sent to a correctional institution for a violent offense?” with 0 corresponding to No and 1 corresponding to Yes. Next, we specify the survey design using *svydesign*, and use *svytable* to create the contingency table. Finally, we obtain a Wald chi-square test statistic with the *summary* or *svychisq* function.

```
dsyc<-svydesign(ids=~psu,weights=~finalwt,strata=~stratum,nest=TRUE,data=syc)
dsyc # Verify this is a stratified cluster sample
## Stratified 1 - level Cluster Sampling design (with replacement)
## With (861) clusters.
## svydesign(ids = ~psu, weights = ~finalwt, strata = ~stratum,
##       nest = TRUE, data = syc)
# Create contingency table by incorporating weights
example1005 <- svytable(~famtime+everviol,design=dsyc)
example1005
##       everviol
## famtime    0    1
##       1 4838 7946
##       2 4761 7154
# Perform the Wald chi-square test
summary(example1005,statistic="Wald")
##       everviol
## famtime    0    1
##       1 4838 7946
##       2 4761 7154
##
## Design-based Wald test of association
##
## data:  svychisq(~famtime + everviol, design = dsyc, statistic = "Wald")
## F = 0.99514, ndf = 1, ddf = 845, p-value = 0.3188
# Alternatively, can calculate the Wald statistic directly using svychisq
# without forming the table first
```

```
svychisq(~famtime+everviol,design=dsyc,statistic="Wald")
##
## Design-based Wald test of association
##
## data:  svychisq(~famtime + everviol, design = dsyc, statistic = "Wald")
## F = 0.99514, ndf = 1, ddf = 845, p-value = 0.3188
```

The Wald chi-square test statistic is $X_W^2 = 0.995$ with p -value of 0.32, which indicates that there is no evidence of an association between the two factors *famtime* and *everviol*.

10.2 Chi-Square Tests

The *svychisq* function performs all of the chi-square tests discussed in Chapter 10 of SDA. The general form of the function is:

```
svychisq(~variable1 + variable2, design.object, statistic="")
```

Table 10.1 lists some of the test statistics and measures of association produced.

TABLE 10.1

Chi-square test statistics calculated by function *svychisq*.

statistic=	Statistic or Test
Wald	Wald test (Koch et al., 1975).
Chisq	First-order Rao–Scott test, based on Pearson’s chi-square test statistic (Rao and Scott, 1981, 1984).
F	Second-order Rao–Scott test.
adjWald	Adjusted Wald test (Thomas and Rao, 1987).
lincom	Use exact asymptotic distribution for the linear combination of chi-square variables in the Rao–Scott statistic.
saddlepoint	Use saddlepoint approximation for the linear combination of chi-square variables in the Rao–Scott statistic.

In general, we do not recommend using a finite population correction (fpc) when conducting a chi-square test. Often, the purpose of the test is to explore whether there is a general association between the factors in the superpopulation, not merely in the finite population from which the data are drawn. Conducting the test without the fpc allows generalization to the superpopulation (under some superpopulation models) while still accounting for the clustering, stratification, and unequal weights in the sampling design.

Example 10.6 of SDA. For this example, we define the variable *currviol* as 1 if *crimtype*=1 and 0 otherwise. This means that the “0” category of *currviol* consists of the persons with *crimtype* $\in \{2, 3, 4, 5\}$ as well as the 12 persons with missing values for *crimtype*, and can be thought of as the persons not known to have committed a violent offense. The analysis results are almost the same when the 12 missing values are excluded. We also define the variable *ageclass* as 1 if *age* is less than or equal to 15, as 2 if *age* is equal to 16 or 17, and as 3 if *age* is greater than or equal to 18.

```
# Create variables currviol and ageclass for 10.6
syc$currviol <- syc$crimtype
```

```

syc$currviol[syc$scrimtype != 1 | is.na(syc$scrimtype)] <- 0

syc$ageclass <- syc$age
syc$ageclass[syc$age <= 15] <- 1
syc$ageclass[syc$age == 16 | syc$age == 17] <- 2
syc$ageclass[18 <= syc$age] <- 3

# Specify the survey design
dsyc<-svydesign(ids=~psu,weights=~finalwt,strata=~stratum,nest=TRUE,data=syc)
# estimate the contingency table
svytable(~currviol+ageclass,design=dsyc)
##      ageclass
## currviol   1   2   3
##      0 4247 6542 3190
##      1 2770 4630 3633
# First-order Rao-Scott test
svychisq(~currviol+ageclass,design=dsyc,statistic="Chisq")
##
## Pearson's X^2: Rao & Scott adjustment
##
## data:  svychisq(~currviol + ageclass, design = dsyc, statistic = "Chisq")
## X-squared = 33.993, df = 2, p-value = 0.001909
# Second-order Rao-Scott test
# (this is the default, can also request with "statistic=F")
svychisq(~currviol+ageclass, design=dsyc)
##
## Pearson's X^2: Rao & Scott adjustment
##
## data:  svychisq(~currviol + ageclass, design = dsyc)
## F = 6.2614, ndf = 1.7258, ddf = 1458.2973, p-value = 0.003245

```

Pearson's X^2 statistic is 33.993. The Rao-Scott first-order test statistic adjusts X^2 by the design correction $\hat{E}[X^2]/2$ and is compared to a χ^2 distribution with $(3 - 1) * (2 - 1) = 2$ df. The p -value from the Rao-Scott first-order test is 0.002.

The Rao-Scott F statistic for the second-order correction is $F = 6.2614$, which is obtained by dividing the Pearson chi-square statistic ($X^2 = 33.99$) by the design correction and the adjusted numerator df. The second-order Rao-Scott test is the default for *svychisq*, or you can add **statistic = "F"** to obtain it. You may also use an exact distribution to calculate p -values by setting **statistic = "lincom"**; this uses a linear combination of chi-square distributions.

We do not need to calculate the design effects (deffs) for the table cells and margins in order to conduct a chi-square test because the *svychisq* function automatically adjusts for the design effects when calculating the statistics. If you would like to see the deffs, however, they can be calculated with the *svymean* function. Here, we obtain estimates of the deffs compared with simple random sampling with replacement (**deff="replace"**). The design effects for the table cells and margins are large, indicating that persons within the same psu tend to be relatively homogeneous.

```

# deffs for table cells
svymean(~interaction(factor(ageclass), factor(currviol)),design=dsyc,deff="replace")
##              mean      SE  DEff
## interaction(factor(ageclass), factor(currviol))1.0 0.169798 0.028312 14.8978
## interaction(factor(ageclass), factor(currviol))2.0 0.261554 0.017127  3.9791

```

```
## interaction(factor(ageclass), factor(currviol))3.0 0.127539 0.012152 3.4771
## interaction(factor(ageclass), factor(currviol))1.1 0.110747 0.013269 4.6840
## interaction(factor(ageclass), factor(currviol))2.1 0.185111 0.019301 6.4706
## interaction(factor(ageclass), factor(currviol))3.1 0.145250 0.013478 3.8335
# deffs for table margins
svymean(~factor(ageclass)+ factor(currviol),design=dsyc,deff="replace")
##              mean      SE    DEff
## factor(ageclass)1 0.280545 0.033395 14.4762
## factor(ageclass)2 0.446666 0.026528  7.4598
## factor(ageclass)3 0.272789 0.022366  6.6068
## factor(currviol)0 0.558892 0.025337  6.8223
## factor(currviol)1 0.441108 0.025337  6.8223
```

10.3 Loglinear Models

This section will discuss how to fit a loglinear model with categorical data using function *svyloglin*.

Example 10.8 of SDA. Recall the computer and cable data from Example 10.1 of SDA. To analyze this in the *survey* package, let's first create data set *cabledf* with 500 records—one record per observation with a sampling weight of 1, and create the survey design object.

```
cabletable <- matrix(c(
  "no","no",105,
  "no","yes",88,
  "yes", "no",188,
  "yes", "yes",119),byrow=T,nrow=4,ncol=3)
colnames(cabletable) <- c("cable","computer","count")
cabletable <- data.frame(cabletable)
cabledf <- cabletable[rep(row.names(cabletable), cabletable[,3]), 1:2]
dim(cabledf)
## [1] 500  2
cabledf$wt <- rep(1,500)
head(cabledf)
##      cable computer wt
## 1      no        no  1
## 1.1    no        no  1
## 1.2    no        no  1
## 1.3    no        no  1
## 1.4    no        no  1
## 1.5    no        no  1
dcable <- svydesign(id=~1, weights=~wt, data=cabledf)
```

Now let's do a chi-square test for independence with the *svychisq* function. Since this is an SRS with weights of 1, *chisq.test* and *svychisq* both give the same result. The value of the chi-square statistic from *svychisq* is 2.281 with *p*-value = 0.1314, just as was found in the chi-square test from Example 10.1.

```
# chi-squared test for independent data, no continuity correction
chisq.test(cabledf$computer,cabledf$cable,correct=F)
##
##  Pearson's Chi-squared test
```

```
##
## data:  cabledf$computer and cabledf$cable
## X-squared = 2.281, df = 1, p-value = 0.131
svychisq(~computer+cable,design=dcable,statistic="Chisq")
##
## Pearson's X^2: Rao & Scott adjustment
##
## data:  svychisq(~computer + cable, design = dcable, statistic = "Chisq")
## X-squared = 2.281, df = 1, p-value = 0.1314
```

We can also use the *svyloglin* function to fit an additive loglinear model with variables *cable* and *computer*.

```
# Fit loglinear model for independence, with additive factors
cableindep <- svyloglin(~factor(cable)+factor(computer),design=dcable)
summary(cableindep)
## Loglinear model: svyloglin(~factor(cable) + factor(computer), design = dcable)
##               coef           se           p
## factor(cable)1  -0.2320788  0.04597713  4.471605e-07
## factor(computer)1  0.1737269  0.04544339  1.318753e-04
# obtain coefficients including intercept, deviance
cableindep$model
##
## Call:  glm(formula = ff, family = quasipoisson, data = dat)
##
## Coefficients:
##      (Intercept)      factor(cable)1  factor(computer)1
##           4.7866          -0.2321           0.1737
##
## Degrees of Freedom: 3 Total (i.e. Null);  1 Residual
## Null Deviance:      43.36
## Residual Deviance: 2.275  AIC: NA
# obtain the predicted counts under the independence model
cableindep$model$fitted.values
##      1      2      3      4
## 113.098 179.902  79.902 127.098
# obtain the fitted probabilities under the model
cableindep$model$fitted.values/500
##      1      2      3      4
## 0.226196 0.359804 0.159804 0.254196
```

The estimates from *summary(cableindep)* are the coefficients for *cable* = “no” and *computer* = “no”. The output prints *factor(cable)1*, where 1 means “no” according to the alphabetical order of “no” and “yes”. Note that the intercept here is equal to 4.786607, which is the intercept for the expected count. To calculate the fitted probabilities using the loglinear model, we need to convert it to the intercept for probabilities, which is equal to $4.786607 - \ln(500) = -1.428$. We can calculate the fitted probabilities using the formula, or by requesting the *fitted.values*, which give the predicted counts under the additive loglinear model, from the function output.

The *deviance* component of *cableindep\$model* contains the deviance of 2.275. This is the value of G^2 from the likelihood ratio test for independence, corresponding to a *p*-value of 0.132. Alternatively, you can obtain this *p*-value from the interaction term of a saturated model or by comparing the two nested models.

```
# Fit saturated loglinear model
cablesat <- svyloglin(~factor(cable)*factor(computer),design=dcable)
summary(cablesat)
## Loglinear model: svyloglin(~factor(cable) * factor(computer), design = dcable)
##
##               coef          se          p
## factor(cable)1      -0.22106707  0.04655596  2.050156e-06
## factor(computer)1      0.15848550  0.04655596  6.635966e-04
## factor(cable)1:factor(computer)1 -0.07017373  0.04655596  1.317341e-01
# this can also be obtained by comparing the two models
anova(cablesat,cableindep)
## Analysis of Deviance Table
## Model 1: y ~ factor(cable) + factor(computer)
## Model 2: y ~ factor(cable) + factor(computer) + factor(cable):factor(computer)
## Deviance= 2.274961 p= 0.1324977
## Score= 2.281035 p= 0.1319832
```

Example 10.9 of SDA. In this example, we use *svyloglin* to analyze data set *sysc*. Let's first take a look at the three-way table of weighted counts for the variables *ageclass*, *everviol*, and *famtime*. We use the design object *dsyc*, defined earlier for Example 10.5 of SDA, to incorporate the weights.

```
syctable3way <- svytable(~ageclass+everviol+famtime,design=dsyc)
syctable3way
## , , famtime = 1
##
##      everviol
## ageclass    0    1
##      1 1628 2115
##      2 2332 3395
##      3  878 2436
##
## , , famtime = 2
##
##      everviol
## ageclass    0    1
##      1 1453 1723
##      2 2234 3056
##      3 1074 2375
# Estimate probabilities in table
syctable3way/sum(syctable3way)
## , , famtime = 1
##
##      everviol
## ageclass      0      1
##      1 0.06591360 0.08563100
##      2 0.09441678 0.13745496
##      3 0.03554800 0.09862747
##
## , , famtime = 2
##
##      everviol
## ageclass      0      1
##      1 0.05882829 0.06975991
##      2 0.09044901 0.12372971
##      3 0.04348354 0.09615774
```

Next, we fit a saturated model. This produces the parameter estimates, their standard errors, and the p -values for the null hypotheses that the individual parameters equal 0. This survey has large design effects because the facilities have a high degree of clustering with respect to ages served and severity of offenses.

The following code requests the saturated model.

```
svyloglin(~ageclass+everviol+famtime+ageclass*everviol
+ ageclass*famtime+everviol*famtime+ageclass*everviol*famtime,design=dsyc)
```

Alternatively, you could type

```
svyloglin(~ageclass*everviol*famtime,design=dsyc)
```

to include all the factors and interactions among the factors.

```
# Fit saturated loglinear model
sycsat <- svyloglin(~ageclass*everviol*famtime,design=dsyc)
summary(sycsat)
## Loglinear model: svyloglin(~ageclass * everviol * famtime, design = dsyc)
##
##               coef          se          p
## ageclass1      -0.114863155  0.11589240  3.216275e-01
## ageclass2       0.344093390  0.07257131  2.121916e-06
## everviol1      -0.244591938  0.04529927  6.683913e-08
## famtime1       0.024225036  0.03268196  4.585506e-01
## ageclass1:everviol1  0.136557159  0.05820665  1.897229e-02
## ageclass2:everviol1  0.072369272  0.03217396  2.449266e-02
## ageclass1:famtime1  0.055452399  0.03677297  1.315632e-01
## ageclass2:famtime1  0.012807329  0.02838298  6.518218e-01
## everviol1:famtime1 -0.031699074  0.02297100  1.675988e-01
## ageclass1:everviol1:famtime1  0.008882581  0.02890809  7.586381e-01
## ageclass2:everviol1:famtime1  0.016132994  0.02780612  5.617826e-01
```

The output shows the parameter estimates for the 11 terms in the saturated model, along with standard errors and p -values for the Wald tests that the individual parameters equal zero. There are two parameters for each term involving the three-category factor *ageclass*. The p -values can be used to test individual terms.

You can also compare sets of nested models. Let's fit the independent-factor model, and compare it with the saturated model.

```
# Fit additive loglinear model for independent factors
sycind <- svyloglin(~ageclass+everviol+famtime,design=dsyc)
summary(sycind)
## Loglinear model: svyloglin(~ageclass + everviol + famtime, design = dsyc)
##
##               coef          se          p
## ageclass1 -0.14745454  0.11162655  1.865137e-01
## ageclass2  0.31771375  0.07232164  1.117632e-05
## everviol1 -0.22651791  0.04817065  2.571154e-06
## famtime1  0.03519814  0.03202768  2.717719e-01
# compare independent and saturated models
anova(sycsat, sycind)
## Analysis of Deviance Table
## Model 1: y ~ ageclass + everviol + famtime
## Model 2: y ~ ageclass + everviol + famtime + ageclass:everviol + ageclass:famtime +
##          everviol:famtime + ageclass:everviol:famtime
## Deviance= 50.82273 p= 0.0007817253
## Score= 48.67106 p= 0.001132184
```

From the output, the p -value is 0.00113, indicating that the independence model exhibits lack of fit when compared with the saturated model.

10.4 Summary, Tips, and Warnings

Table 10.2 lists the major functions used in this chapter to perform categorical data analyses.

TABLE 10.2

Functions used for Chapter 10.

Function	Package	Usage
summary	base	Summarize the results from fitting a model (here used for loglinear models)
chisq.test	stats	Perform a chi-square test (not using survey methods)
anova	stats	Compute an analysis of variance table from a model object
predict	stats	Obtain predicted values from a model object
svydesign	survey	Specify the survey design
svymean	survey	Calculate mean and standard error of mean; also calculate design effects
svytable	survey	Estimate the population contingency table from survey data
svychisq	survey	Calculate chi-square test statistics and p -values for survey data
svyloglin	survey	Fit a loglinear model to survey data; may also be used to compare nested models

Tips and Warnings

- Always look at the estimated contingency table before conducting a chi-square test or fitting a loglinear model.
- In general, we recommend conducting chi-square tests and fitting loglinear models without an `fpc`. It is often desired to explore the association between variables in a context more general than the particular finite population.
- Check for empty or sparse cells by applying the `table` function before analyzing the data, particularly if you are fitting a loglinear model with many terms. See Fienberg and Rinaldo (2007) for a discussion of what can go wrong when fitting loglinear models.

Regression with Complex Survey Data

We have already used the *svyglm* function from the **survey** package (Lumley, 2020) in Chapter 4 to perform ratio and regression estimation. In this chapter, we use the function to calculate regression coefficients and provide other summary statistics for regression analyses with complex survey data. The code is in file `ch11.R` on the book website.

11.1 Straight Line Regression with a Simple Random Sample

For many analyses carried out on a simple random sample (SRS), results from a model-based analysis in a function designed for independent and identically distributed data (such as the *mean* function) are the same as the results from the corresponding survey analysis function (such as the *svymean* function) used with weights set equal to 1. For regression, however, the standard errors for an SRS calculated using functions *lm* or *glm*, which perform model-based regression analyses (see Section 4.6), differ from those calculated by the *svyglm* function from the **survey** package. This is because, as explained in Section 11.2 of SDA, the standard errors for the SRS calculated using linearization account for the errors in estimating the population totals of both the x and y variables; the model-based standard error calculated in the *glm* or *lm* function is conditional on the values of x in the sample and is calculated under the model assumptions.

We can see a slight difference for the estimates calculated in Examples 11.2 and 11.4 of SDA using the *lm* and *svyglm* functions. For this example, the difference is small because the model fits well and the sample is an SRS; for other surveys and models, the difference can be greater and we recommend using the *svyglm* function to fit the model.

Example 11.2 of SDA. The following shows an analysis conducted using the function *lm* for the data in *anthrsrs*. This conducts a model-based analysis under assumptions (A1)–(A4) given in Section 11.1 of SDA. The *summary* function shows the regression parameter estimates, standard errors, t statistic and p -value for the null hypothesis that each parameter equals 0, value of R^2 , and more. The function *glm*, which fits generalized linear models for non-survey data, will give the same results.

```
data(anthrsrs)
fit<-lm(anthrsrs$height~anthrsrs$finger)
summary(fit)
##
## Call:
## lm(formula = anthrsrs$height ~ anthrsrs$finger)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.9045 -1.1638  0.0543  1.1407  5.0543
```

```
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)   30.3162     2.5668   11.81  <2e-16 ***
## anthsrs$finger  3.0453     0.2217   13.73  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.75 on 198 degrees of freedom
## Multiple R-squared:  0.4879, Adjusted R-squared:  0.4853
## F-statistic: 188.6 on 1 and 198 DF, p-value: < 2.2e-16
```

Example 11.4 of SDA. Now let's perform the regression analysis using the *svyglm* function. The major difference is to include the design object created by *svydesign*, where the variable *wt* is set equal to 3000/200 for each observation.

```
anthsrs$wt<-rep(3000/200,200)
danthrsrs<- svydesign(id = ~1, weight = ~wt, data = anthsrs)
degf(danthrsrs) # here, 199
## [1] 199
fit2 <- svyglm(height~finger, design=danthrsrs)
summary(fit2)
##
## Call:
## svyglm(formula = height ~ finger, design = danthrsrs)
##
## Survey design:
## svydesign(id = ~1, weight = ~wt, data = anthsrs)
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)   30.3162     2.5436   11.92  <2e-16 ***
## finger         3.0453     0.2201   13.84  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 3.046384)
##
## Number of Fisher Scoring iterations: 2
confint(fit2) # here calculated using normal distribution
##           2.5 %    97.5 %
## (Intercept) 25.330821 35.301675
## finger      2.613917  3.476583
fit2$coefficients # contains coefficients
## (Intercept)      finger
##    30.31625     3.04525
fit2$deviance # residual sum of squares (for this SRS example)
## [1] 606.2304
```

The values of the estimated slope and intercept are the same as in the analysis with the *lm* function, but the standard errors of the regression coefficients, here calculated using linearization, are different. In this example, where the straight-line model fits the data well, the difference in the standard errors is small. In other examples, the two sets of standard errors may exhibit wider disparities.

Degrees of freedom (df) for regression analyses. One other difference between the two analyses is the df to be used for the confidence intervals and hypothesis tests. For the model-based analysis in Example 11.2, a t distribution with $n - (\text{number of model parameters})$ df is used (here 198). We usually set the df equal to $(\text{number of psus}) - (\text{number of strata})$ for regression analyses with complex survey data, regardless of the number of model parameters. One exception to that guideline is when an analysis is being done on a domain that does not appear in some of the psus (although in this situation a model-based analysis might be preferred; see Section 11.5). Valliant and Rust (2010) discuss df in survey data analyses.

Finite population corrections for regression analyses. If desired, you can include an *fpc* argument in the *svydesign* function to calculate standard errors that incorporate a finite population correction (fpc) and hence will be slightly smaller. We suggest omitting the *fpc* argument when performing regression analyses because we often want to learn about the relationships among variables in a universal sense (including potential populations that are similar to the finite population), not just in the particular finite population that was studied. Ask yourself: If I were estimating regression relationships for data from a population census, would I want the standard errors of the coefficients to be zero (as they would be if a census were taken because there is no sampling variability)? If the answer is no, then omit the fpc.

Example 11.6 of SDA. Instead of using the linearization method to calculate standard errors of the regression coefficients, we can calculate jackknife weights for the survey and use those with the *svyglm* function to compute the standard errors. The jackknife weights are $3000/199 = 15.0754$ for the observations not deleted for the replicate and 0 for the observation that is deleted.

```
danthrsrj <- as.svrepdesign(danthrsr, type="JK1")
fit3 <- svyglm(height~finger, design=danthrsrj)
summary(fit3)
##
## Call:
## svyglm(formula = height ~ finger, design = danthrsrj)
##
## Survey design:
## as.svrepdesign(danthrsr, type = "JK1")
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  30.3162    2.5805   11.75  <2e-16 ***
## finger       3.0453     0.2233   13.64  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 606.2304)
##
## Number of Fisher Scoring iterations: 2
confint(fit3)
##              2.5 %      97.5 %
## (Intercept) 25.258652 35.373844
## finger      2.607591  3.482909
```

The output for the jackknife is similar to that from Example 11.4, where standard errors are calculated using linearization, but the standard errors are slightly larger. This is not a matter for concern; the two methods of variance estimation are asymptotically equivalent but often give slightly different numbers for real data sets, which are of finite size.

For comparison, Table 11.1 lists the point estimates and standard errors using *lm* and *svyglm*.

TABLE 11.1

Comparison of standard error estimates using linearization or jackknife for the regression coefficients produced by *glm* and *svyglm* (data *anthsr*s).

	Estimate	SE(lm)	SE(svyglm, linearization)	SE(svyglm, JK)
Intercept	30.3162	2.5668	2.5436	2.5805
Slope	3.0453	0.2217	0.2201	0.2233

11.2 Linear Regression for Complex Survey Data

The National Health and Nutrition Examination Survey (NHANES) data (data set *nhanes* in the *SDAResources* package) will be used for examples in the remaining sections. This survey has unequal weights, stratification, and clustering.

Example 11.7 of SDA. In this example, we fit the multiple linear regression model

$$bmxbmi = ridageyr + ridageyr^2$$

to the NHANES data; *ridageyr* is the variable in the data that gives each person's age in years. This analysis fits the model to the entire range of ages; Section 11.3 will show an example of a regression model fit to the observations in the domain of adults. To enter the design information, recall that *sdmvp*su and *sdmv*stra are the primary sampling unit (psu) and strata variables, respectively, and *wtmec2yr* is the weight variable. We create a subset design object, *dnhanescc*, for the domain that has no missing data for the response and explanatory variables in the regression.

```
data(nhanes)
nhanes$ridageyr2<-nhanes$ridageyr^2
dnhanes<-svydesign(id=~sdmvp$psu, strata=~sdmv$stra,nest=TRUE,
  weights=~wtmec2yr,data=nhanes)
dnhanescc <- subset(dnhanes,complete.cases(cbind(bmxbmi,ridageyr)))
fit4<-svyglm(bmxbmi~ridageyr + ridageyr2, design=dnhanescc)
summary(fit4)
##
## Call:
## svyglm(formula = bmxbmi ~ ridageyr + ridageyr2, design = dnhanescc)
##
## Survey design:
## subset(dnhanes, complete.cases(cbind(bmxbmi, ridageyr)))
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 15.2981488  0.2293566   66.70  < 2e-16 ***
## ridageyr      0.6031084  0.0188377   32.02 9.43e-14 ***
## ridageyr2    -0.0057488  0.0002311  -24.88 2.38e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 42.25111)
```

```
##
## Number of Fisher Scoring iterations: 2
# can also extract separate elements
nobs(fit4) # number of observations used in regression
## [1] 8756
fit4$coefficients # extract regression parameters
## (Intercept)      ridageyr      ridageyr2
## 15.298148792  0.603108429 -0.005748801
1 - fit4$deviance/fit4$null.deviance # R-squared
## [1] 0.2833541
# test linear hypotheses about model terms
regTermTest(fit4, ~ridageyr + ridageyr2, df=15)
## Wald test for ridageyr ridageyr2
## in svyglm(formula = bmx bmi ~ ridageyr + ridageyr2, design = dnhanescc)
## F = 746.9868 on 2 and 15 df: p= 9.5625e-16
```

The `svyglm` function tells R to fit a model with y variable *bmx bmi* and x variables *ridageyr* and *ridageyr2*. The estimated regression coefficients are accompanied by their standard errors. The fitted values are given by the equation

$$\hat{y} = 15.298 + 0.6031084 * \text{ridageyr} - 0.0057488 * \text{ridageyr}^2,$$

and the fitted values for the data observations are stored in `fit4$fitted.values`. Residuals can be found in `fit4$residuals`.

We can estimate the population value of R^2 using the estimated deviance from the fitted model and the null model (model fit with just an intercept term). For linear regression, R^2 is the proportion of variability about the mean that is explained by the regression model, which is one minus (deviance from the regression model)/(deviance from the null model). Here we estimate $R^2 = 0.2834$.

Tests of linear hypotheses. The `regTermTest` function gives the Wald F statistic for testing the hypothesis that the terms given in the second argument are all zero. For a linear hypothesis $H_0 : \mathbf{L}\boldsymbol{\beta} = 0$, the Wald F statistic equals

$$F = \frac{(\mathbf{L}\hat{\mathbf{B}})^T [\mathbf{L} \hat{\mathbf{V}}(\hat{\mathbf{B}}) \mathbf{L}^T]^{-1} (\mathbf{L}\hat{\mathbf{B}})}{\text{rank} [\mathbf{L} \hat{\mathbf{V}}(\hat{\mathbf{B}}) \mathbf{L}^T]}$$

(a generalized inverse may be used when the inverse of $\mathbf{L} \hat{\mathbf{V}}(\hat{\mathbf{B}}) \mathbf{L}^T$ does not exist), and is compared to an F distribution with numerator df equal to the rank of $\mathbf{L} \hat{\mathbf{V}}(\hat{\mathbf{B}}) \mathbf{L}^T$. We use (number of psus minus number of strata) as the denominator df. Here, the Wald F statistic of 746.99, with p -value much less than 0.0001, is for the null hypothesis that the coefficients of *ridageyr* and *ridageyr2* are both zero, and is compared to an $F_{2,15}$ distribution. The `regTermTest` function will perform a Rao–Scott test based on the estimated log-likelihood ratio if `method="LRT"` is specified.

Regression analyses and missing data. Many data sets, including the *nhanes* data, have missing values for the y variable or one or more of the x variables. In this case, only 8756 (`=nobs(fit4)`) of the 9971 observations in the data set had values for *bmx bmi*.

We performed the regression analysis on the subset of data with non-missing values. For this example, the same results would be obtained by running the regression model with design object *dnhanes* since the default method is to omit cases with missing data. For some analyses, however, entire psus might be missing the value of at least one of the model variables;

using the *subset* function ensures that the standard error calculations use information from all of the psus.

Another option is to fit the regression model with a replicate-weights design object. If you are using replication methods to estimate variances, you can analyze subsets of the data as long as the replicate weights have been created using the full sample. This is because the replicate weight construction already accounts for the full design structure. Replicate weight variance estimates are of the form $\sum_{r=1}^R c_r (\hat{\theta}_r - \hat{\theta})^2$, where $\hat{\theta}$ is the estimate calculated using the full-sample weight and $\hat{\theta}_r$ is the estimate calculated using the r th replicate weight. When you calculate domain estimates, $\hat{\theta}$ and the replicate values $\hat{\theta}_r$ are calculated using the same subset of observations, so the domain of non-missing values does not need to be defined separately.

Here we fit the model with balanced repeated replication (BRR) weights, excluding the observations with missing data. The standard errors differ slightly from those calculated earlier using linearization.

```
dnhanesbrr <- as.svrepdesign(dnhanes, type="BRR")
fit5<-svyglm(bmxbmi~ridageyr + ridageyr2, design=dnhanesbrr,na.action=na.omit)
summary(fit5)
##
## Call:
## svyglm(formula = bmxbmi ~ ridageyr + ridageyr2, design = dnhanesbrr,
##      na.action = na.omit)
##
## Survey design:
## as.svrepdesign(dnhanes, type = "BRR")
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 15.2981488  0.2337378   65.45 < 2e-16 ***
## ridageyr     0.6031084  0.0190054   31.73 1.06e-13 ***
## ridageyr2    -0.0057488  0.0002321  -24.76 2.53e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 405852.8)
##
## Number of Fisher Scoring iterations: 2
```

But be aware that item nonresponse can distort estimates of regression relationships in the population. It is also possible that when a model has many explanatory variables, each with some item nonresponse, the missing data patterns can mesh in such a way that the model is fit on relatively few observations. Or the model might be fit on data from only a few of the psus. We recommend exploring the amount and patterns of missing data before performing analyses.

Graphing the regression equation. As we saw in Section 7.5, the *svyplot* function will produce scatterplots that account for the survey weights. Figure 11.1 displays a bubble plot with the fitted quadratic regression line.

```
# plot data bmxbmi~ridageyr
svyplot(bmxbmi~ridageyr, design=dnhanes, style="bubble",basecol="gray",
        inches=0.03,xlab="Age (years)",ylab="Body Mass Index",
        xlim=c(0,80),ylim=c(10,70))
# plot fitted quadratic regression line
```

```
timevalues <- seq(2, 80, 0.02)
length(timevalues)
## [1] 3901
predicted <- predict(fit4, data.frame(ridageyr=timevalues, ridageyr2=timevalues^2))
lines(timevalues, predicted, col = "black", lwd = 3)
```

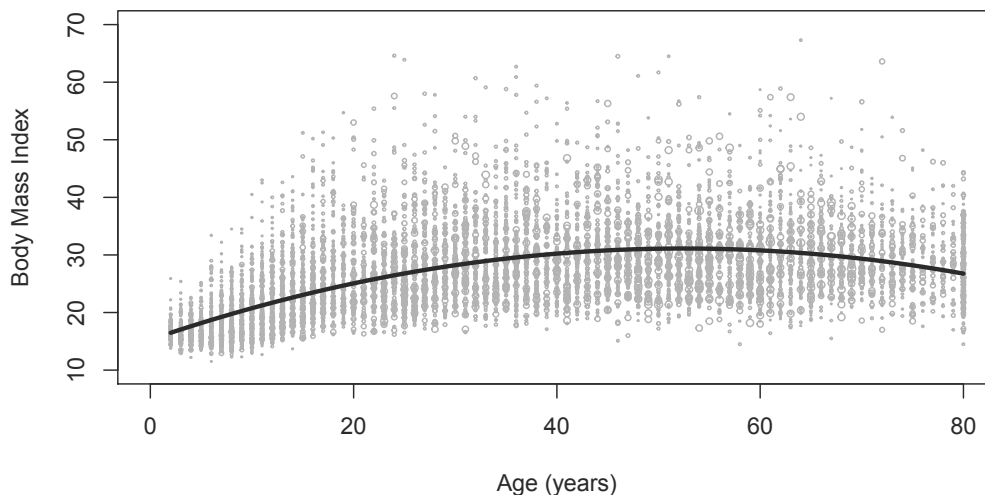


FIGURE 11.1: Scatterplot with fitted quadratic model.

11.3 Using Regression to Compare Domain Means

We computed domain means using the *svyby* function in Chapter 4. We can also compute—and compare—domain means using the *svyglm* function, by fitting a regression model with a categorical predictor variable that defines the domains.

To show how this works, let's define a few new variables and design object `dnhanes` for the remaining examples in this chapter.

- *female*: define *female* = 1 if *riagendr* = 2, and *female* = 0 if *riagendr* = 1
- *age20d*: define *age20d* = 1 if *ridageyr* ≥ 20 and *bmx bmi* is not missing, and 0 otherwise
- *raceeth*: define *raceeth* = Hispanic if *ridreth3* = 1 or 2; *raceeth* = White if *ridreth3* = 3; *raceeth* = Black if *ridreth3* = 4; *raceeth* = Asian if *ridreth3* = 6; *raceeth* = Other if *ridreth3* = 7
- *bmi30*: define *bmi30* = 1 if *bmx bmi* > 30, *bmi30* = 0 if *bmx bmi* ≤ 30

```
nhanes$female <- nhanes$riagendr-1
nhanes$age20d<-rep(0,nrow(nhanes))
nhanes$age20d[nhanes$ridageyr >=20 & !is.na(nhanes$bmx bmi)]<-1
```

```

nhanes$bmi30<-nhanes$bmxbmi
nhanes$bmi30[nhanes$bmxbmi>30]<-1
nhanes$bmi30[nhanes$bmxbmi<=30]<-0
nhanes$raceeth <- rep(NA,nrow(nhanes))
nhanes$raceeth[nhanes$ridreth3==1 | nhanes$ridreth3==2] <- "Hispanic"
nhanes$raceeth[nhanes$ridreth3==3] <- "White"
nhanes$raceeth[nhanes$ridreth3==4] <- "Black"
nhanes$raceeth[nhanes$ridreth3==6] <- "Asian"
nhanes$raceeth[nhanes$ridreth3==7] <- "Other"
# check variable construction; display missing values
table(nhanes$age20d,nhanes$bmi30,useNA="ifany")
##
##           0      1 <NA>
##  0 3130  220 1215
##  1 3248  2158    0
# no missing data for female, raceeth
table(nhanes$female,nhanes$riagendr,useNA="ifany")
##
##           1      2
##  0 4892    0
##  1    0 5079
table(nhanes$raceeth,nhanes$ridreth3,useNA="ifany")
##
##           1      2      3      4      6      7
##   Asian      0      0      0      0 1042    0
##   Black      0      0      0 2129    0    0
##   Hispanic 1921 1308    0      0    0    0
##   Other      0      0      0      0    0 505
##   White      0      0 3066    0    0    0
dnhanes <- svydesign(id = ~sdmvpstu, strata = ~ sdmvstra, nest=TRUE,
                    weights=~wtmec2yr, data = nhanes)

```

Example 11.8 of SDA. Section 4.3 used the *svyby* function to calculate separate estimates for domains. The easiest way to compare domain means in the **survey** package is to fit a regression model predicting the response from one or more variables defining the domains. When there are only two domains, an indicator variable may be used. Here we fit a model on the domain of adults predicting BMI from the indicator variable *female*.

```

dnhanescc <- subset(dnhanes,!is.na(bmxbmi))
fit6<-svyglm(bmxbmi~female, design=dnhanescc)
summary(fit6)
##
## Call:
## svyglm(formula = bmxbmi ~ female, design = dnhanescc)
##
## Survey design:
## subset(dnhanes, !is.na(bmxbmi))
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  26.9276      0.2070 130.076  <2e-16 ***
## female        0.6897      0.2531   2.725   0.0164 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##

```

```
## (Dispersion parameter for gaussian family taken to be 58.8379)
##
## Number of Fisher Scoring iterations: 2
2*(1-pt(2.725,15)) # calculate p-value for female with 15 df
## [1] 0.01565339
```

The regression coefficient corresponding to *female* is the estimated difference between the mean BMI of females and the mean BMI of males. From the output, this value is 0.6897. The test statistic for the null hypothesis that the mean BMI is the same for both genders is $2.725 = 0.6897/0.2531$; comparing this to a t distribution with 15 df (= number of psus minus number of strata) results in a p -value of 0.0157, indicating that the mean BMI is significantly different for the two genders. The estimated mean BMI for the male group (having *female*=0) is 26.9276, and the estimated mean BMI for the female group is $26.9276 + 0.6897 = 27.6173$.

You can also see the individual domain means by fitting a regression model without an intercept with the explanatory variable *factor(female)*. The regression parameters are the estimated mean BMI for males and females.

```
fit7<-svyglm(bmxbmi~factor(female)-1, design=dnhanescc)
summary(fit7)
##
## Call:
## svyglm(formula = bmxbmi ~ factor(female) - 1, design = dnhanescc)
##
## Survey design:
## subset(dnhanes, !is.na(bmxbmi))
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## factor(female)0    26.928      0.207   130.1   <2e-16 ***
## factor(female)1    27.617      0.260   106.2   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 58.8379)
##
## Number of Fisher Scoring iterations: 2
```

Comparing more than two domain means. With more than two categories, it is more convenient to declare the variable defining the domains as a categorical variable using function *factor*, and use that as the explanatory variable in the model statement of function *svyglm*. Bretz et al. (2016) discuss how to use multiple comparison methods in R to compare group means.

Example 11.9 of SDA. This example shows a comparison of BMI for adults in five race/ethnicity groups measured in NHANES. We defined the categories in variable *raceeth* above. We first create the subset design object to restrict the analysis to adults age 20 and over, and then fit the model with the factor variable.

```
# subset with age20d = 1 has no missing data for modeling
dnhanesadult <- subset(dnhanes,age20d==1)
fit8<-svyglm(bmxbmi~factor(raceeth), design=dnhanesadult)
summary(fit8)
##
## Call:
```

```
## svyglm(formula = bmx bmi ~ factor(raceeth), design = dnhanesadult)
##
## Survey design:
## subset(dnhanes, age20d == 1)
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)      24.9707    0.1411  176.92 < 2e-16 ***
## factor(raceeth)Black    5.6205    0.3380   16.63 3.83e-09 ***
## factor(raceeth)Hispanic  5.6266    0.3368   16.71 3.65e-09 ***
## factor(raceeth)Other    5.4714    0.5337   10.25 5.76e-07 ***
## factor(raceeth)White    4.2599    0.2372   17.96 1.69e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for gaussian family taken to be 47.05606)
##
## Number of Fisher Scoring iterations: 2
1 - fit8$deviance/fit8$null.deviance # R-squared
## [1] 0.03264132
# test statistic for H0: all domain means are equal
regTermTest(fit8,~factor(raceeth),df=15)
## Wald test for factor(raceeth)
## in svyglm(formula = bmx bmi ~ factor(raceeth), design = dnhanesadult)
## F = 131.6208 on 4 and 15 df: p= 1.7277e-11
# can draw side-by-side boxplot, see Figure 11.6 of SDA for plot
# svyboxplot(bmx bmi~raceeth,dnhanesadult)
```

The above code produces estimates of the regression parameters together with standard errors and test statistics. These are the coefficients for the domain of adults ($age20d = 1$) that are shown in Table 11.5 of SDA. Note that the coefficient for the reference category, Asian, is the intercept 24.9707. The other regression coefficients estimate the difference between the mean for the category listed and the reference category, and you can calculate each estimated domain mean from those. For example, the estimated BMI for Black adults is $24.9707 + 5.620 = 30.5907$.

Alternatively, you can fit the model without an intercept. Then the estimated regression coefficients are the domain means.

```
fit9<-svyglm(bmx bmi~factor(raceeth)-1, design=dnhanesadult)
fit9$coefficients
##      factor(raceeth)Asian      factor(raceeth)Black factor(raceeth)Hispanic
##              24.97068              30.59121              30.59725
##      factor(raceeth)Other      factor(raceeth)White
##              30.44209              29.23055
```

Note that the estimated value of R^2 for this analysis is small—only about three percent of the variability is explained by the variable *raceeth*, but the Wald test shows the group means to be highly significantly different ($F = 131.6$ with p -value < 0.0001). Remember that statistical significance, which depends on the effective sample sizes for the domains, is not necessarily the same as practical importance.

11.4 Logistic Regression

The *svyglm* function can perform logistic regression by adding the argument `family = quasibinomial`. Call the function as:

```
svyglm(y ~ x1 + x2 + ...,family=quasibinomial,design=design object)
```

where y is a (binary) response variables and $x_1, x_2, \dots$ are explanatory variables. You can specify the variance estimation methods for the *svyglm* function through the *svydesign* or *svrepdesign* functions, as discussed in Chapter 9.

Example 11.12 of SDA. Recall that variable *female* is defined to equal 1 if the person is a female and 0 if the person is a male. Here, *female* is treated as a numeric variable; it could also be analyzed as a categorical variable using function *factor*. Variable *bmi30* is defined as 1 if *bmx bmi* > 30, and defined as 0 if *bmx bmi* ≤ 30.

We use a subset design object to estimate parameters for adults age 20 and over who have data for all variables in the regression equation. The model fit in this example, which does not include an interaction term, has the same rate of increase for the logit of the predicted probability for males and females, even though the two genders may have different base probabilities.

```
dnhanessub2<-subset(dnhanes,age20d==1 & !is.na(bmxwaist))
lrout<-svyglm(bmi30 ~ bmxwaist+ female,family=quasibinomial,
              design=dnhanessub2)
summary(lrout)
##
## Call:
## svyglm(formula = bmi30 ~ bmxwaist + female, design = dnhanessub2,
##        family = quasibinomial)
##
## Survey design:
## subset(dnhanes, age20d == 1 & !is.na(bmxwaist))
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -29.9560      1.1920 -25.130 2.09e-12 ***
## bmxwaist      0.2809      0.0115  24.434 3.00e-12 ***
## female       1.5786      0.1666   9.478 3.34e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasibinomial family taken to be 0.7048114)
##
## Number of Fisher Scoring iterations: 7
# calculate odds ratios
exp(lrout$coefficient)
## (Intercept)      bmxwaist      female
## 9.778316e-14 1.324332e+00 4.848338e+00
# Wald test for all parameters in model
regTermTest(lrout, ~bmxwaist+female,df=15)
## Wald test for bmxwaist female
## in svyglm(formula = bmi30 ~ bmxwaist + female, design = dnhanessub2,
##            family = quasibinomial)
```

```
## F = 312.2534 on 2 and 15 df: p= 5.982e-13
```

The output shows the parameter estimates for the model. The odds ratio is calculated as $\exp(B_j)$ for the corresponding regression coefficient. Thus the odds ratio for *bmxwaist* equals $\exp(0.2809) = 1.324$ and is interpreted as follows. Suppose that person 1 has a waist circumference that is 1 cm larger than the waist circumference of person 2, and the two persons have the same values for all of the other covariates in the model (in this example, that means they have the same gender). Then the model predicts the odds that person 1 has BMI > 30 to be 1.324 times as large as the odds that person 2 has BMI > 30.

The *regTermTest* function computes a Wald-type *F* statistic for comparing nested models. Here we compute the statistic for the null hypothesis that the effects of *bmxwaist* and *female* are both 0. The Wald test statistic is $F = 312.25$ with a *p*-value much less than 0.0001, suggesting rejection of the null hypothesis.

Graphing predicted probabilities from logistic regression. In the following, we show a graph of the predicted probability that BMI > 30 for men and women. The predicted values from the model are stored in *lrout\$fittd.values*, but we use the *predict* function to give an even distribution of points for drawing the lines.

```
waist <- seq(50,175,0.1)
xfemale <- data.frame(bmxwaist=waist,female=rep(1,length(waist)))
linfemalepred <- predict(lrout,xfemale)
xmale <- data.frame(bmxwaist=waist,female=rep(0,length(waist)))
linmalepred <- predict(lrout,xmale)
# predicted probability for female
femalepred <- exp(linfemalepred)/(1 + exp(linfemalepred))
# predicted probability for male
malepred <- exp(linmalepred)/(1 + exp(linmalepred))

# draw the graph
par(las=1,mar=c(4,4,1,2))
plot(waist,femalepred,type="n",xlab="Waist Circumference (cm)",
     ylab="Estimated Probability",axes=F,xlim=c(70,140))
lines(waist,femalepred,lty=1,lwd=2)
lines(waist,malepred,lty=2,lwd=2)
legend("topleft",c("Female","Male"),lty=c(1,2),bty="n")
axis(2)
axis(1)
box(bty="l")
```

The graph in Figure 11.2 shows the predicted probabilities from the model. This does not show the original data, however, and a scatterplot of a binary variable *y* versus the explanatory variables typically does not provide much information about the relationship because it usually displays an indistinguishable mass of observations at *y* = 1 and another mass at *y* = 0. A more helpful option is to construct graphs showing the distribution of continuous covariates for each level of the response variable (here, *bmi30*). If there is a single continuous covariate, you may want to construct a histogram of the values of the covariate at each value of *y*.

Many of the diagnostics and graphs described by Allison (2012) for model-based logistic regression can also be applied to survey data.

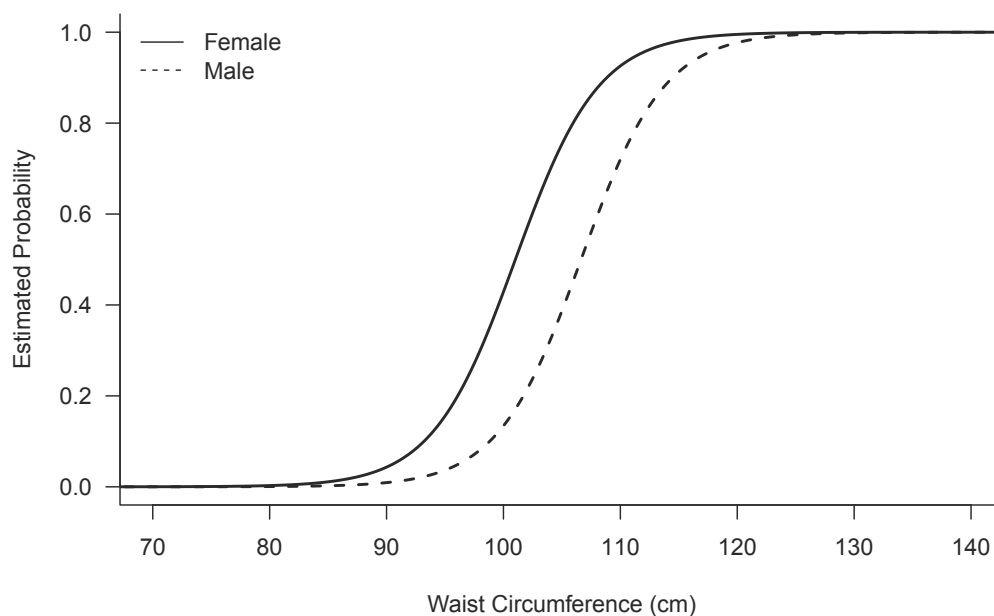


FIGURE 11.2: Predicted probability that BMI > 30.

11.5 Additional Resources and Code

Balanced sampling: Exercise 11.37 of SDA. There are several methods that can be used for selecting a balanced sample in R. The rejective method described in the exercise involves generating repeated samples and then selecting a sample at random from those that meet the balancing constraints within a predetermined tolerance.

The rejective method can require a great deal of computation, however. If the population and sample are large, it may not be feasible to generate repeated samples and then reject those that fail to meet the balancing criteria. The R packages `sampling` (Tillé and Matei, 2021) and `BalancedSampling` (Grafström and Lisic, 2019) use computationally efficient algorithms to select balanced samples. See also Chauvet and Tillé (2006) and Tillé and Wilhelm (2017).

Model-based regression for survey data. Section 11.4 of SDA discussed model-based regression analyses for survey data. You may want to see if the regression coefficients vary from cluster to cluster, or compare the results of models fit with and without weights to see if the weights have information about the regression relationships.

Packages `nlme` (Pinheiro et al., 2021) and `lme4` (Bates et al., 2015, 2020) are commonly used to fit mixed models in R. Functions from these packages can be used to fit linear and generalized mixed models to survey data, using a model-based approach in which stratification variables can be included as covariates and clusters can be included as random effects. As of this writing, these packages do not provide options for fitting mixed models from a design-based perspective.

11.6 Summary, Tips, and Warnings

The *svyglm* function performs linear and logistic regression analyses with numeric and categorical explanatory variables. It will also fit other generalized linear models to survey data. When called without the *family* argument or with **family=gaussian**, it will fit a linear regression model; when called with **family=quasibinomial**, it will fit a logistic regression model; when called with **family=quasipoisson**, it will fit a Poisson regression model.

Table 11.2 lists *svyglm* and the other major functions used in this chapter to perform regression analyses.

TABLE 11.2

Functions used for Chapter 11.

Function	Package	Usage
summary	base	Summarize the results from fitting a model
subset	base	Work with a subset of a vector, matrix, or data frame
predict	stats	Obtain predicted values from a model object
confint	stats	Calculate confidence interval
lm	stats	Fit a linear model to a data set (not using survey methods)
anova	stats	Compute an analysis of variance table from a model object
nobs	stats	Find out how many observations were used to fit a model
svydesign	survey	Specify the survey design
as.svrepdesign	survey	Create a replicate-weights survey design object from a design object that includes weighting, stratification, and clustering information
svyboxplot	survey	Draw boxplot of survey data, incorporating the weights
svyplot	survey	Draw scatterplot of survey data, incorporating the weights
svyglm	survey	Fit a generalized linear model to survey data
regTermTest	survey	Calculate Wald test statistic from model object created using <i>svyglm</i>

The main commands used for a typical analysis with the *svyglm* function are given below. Many other statements and options are available for the *svyglm* function, and these are described in the *survey* package documentation.

```
# categorical variable using function 'factor'
mydata$class_var1<-factor(mydata$class_var1)
mydata$class_var2<-factor(mydata$class_var2)
# enter design object
dobject <- svydesign(id = ~psu, strata = ~ strata, nest=TRUE,
                    weights=~wt, data = mydata)
# enter replicate methods for variance calculation if desired
# or use svrepdesign to use data already containing replicate weights
dobject2 <- as.svrepdesign(dobject, type=" ")
# enter domain info, if analysis is restricted within domain
dobjectsub<-subset(dobject, domain = )
```

```

# or using replicate methods for variance calculation
dobjectsub2<- subset(dobject2, domain = )
# perform regression analysis
# design = dobject for whole data analysis using linearization
# design = dobject2 for whole data analysis using replicate methods
# design = dobjectsub for domain analysis using linearization
# design = dobjectsub2 for domain analysis using replicate methods
fit<-svyglm(y ~x1+ x2+ class_var1 + class_var2, design= )
# logistic regression
fit<-svyglm(y ~x1+ x2+ class_var1 + class_var2,
            family=quasibinomial, design= )
# Display coefficient estimates etc
summary(fit)
# test linear hypotheses using Wald statistic
regTermTest(fit, ~variable1 + variable2)
# estimate R-squared
1 - fit$deviance/fit$null.deviance
# extract coefficients
fit$coefficients
# extract residuals, fitted values
resid <- fit$residuals
fitted.values <- fit$fitted.values

```

For replication variance estimation, specify which variance estimation method is used in the *type* argument of function *as.svrepdesign* or *svrepdesign*, as described in Chapter 9.

Tips and Warnings

- If your data set has item nonresponse, function *nobs* will tell you how many observations were used in the analysis. If many observations were excluded from the model because they were missing *y* or one of the *x* variables, you might want to consider an alternative model for the data. You may also want to investigate the pattern of missing data across psus.
- If separate regression models are desired for domains, use the *subset* function to define design objects for the domains.
- You can use the *svyby* (see Section 4.3) or *svyglm* functions to compute domain means.

In this chapter, we present some methods for analyzing simple two-phase samples and estimating the size of a population using capture-recapture methods. We also describe resources for fitting more complex models to estimate population size, and for fitting small area estimation models. The code is in file `ch12.R` on the book website.

12.1 Two-Phase Sampling

In a two-phase design, the final sample is taken in two steps. First, a sample is selected from a population using a probability sampling design. In the second step, information from the first-phase sample is used to set selection probabilities for a subsample to be drawn from the first-phase sample. The function `twophase` from the `survey` package (Lumley, 2020) will analyze data from two-phase samples.

Examples 12.1 and 12.4 of SDA. In this example, we use two-phase sampling to estimate the percentage of Vietnam-era veterans in U.S. Veterans Administration (VA) hospitals who actually served in Vietnam (Stockford and Page, 1984). In the first phase, the 1982 VA Annual Patient Census (APC) included a random sample of 20% of the patients in VA hospitals. Therefore, `phase1wt` is equal to 5. After the phase I data were collected, the answers to the question “Was military service in Vietnam?” (“Yes,” “No,” or “Not Available”) were obtained from medical records and used to determine strata for the second phase of sampling. The second phase obtained the true classification of Vietnam service from the military records of a stratified random subsample of the phase I sample. Variable `phase2wt` gives the stratified weights from the second phase of sampling.

The results for the question “Was service in Vietnam?” for this survey are given in Table 12.1.

TABLE 12.1
Results for the question, “Was service in Vietnam?”

APC Group	APC Classification	Phase II Sample Size	Vietnam Service
Yes	755	67	49
No	804	72	11
NotAvail	505	505	211
Total	2064	644	271

From Table 12.1, the percentage of veterans with Vietnam service differed for the three groups: Of the veterans with a “yes” response to the APC survey question, 73% actually served in Vietnam, compared with 15% for the “no” group and 42% for the veterans for whom the information was not available.

We can use the *twophase* function to estimate the population percentage of persons who served in Vietnam. The function has the form

```
twophase(id=list(~,~), strata = list(~, ~) , probs = list(~, ~),
         weights = list(~, ~), fpc = list(~, ~), subset,
         data, method=c("full","approx","simple"))
```

where the *id*, *strata*, *probs*, *weights*, and *fpc* arguments (usually, only some of these arguments are needed) are supplied with information for both design phases. The *subset* argument is used to specify which observations are selected in phase 2. Three methods are available for variance estimation. The "full" method, which requires the sampling probabilities for each stage, gives unbiased variance estimates for general multistage designs. The "simple" and "approx" methods are simpler and use less memory: these will calculate variances for designs in which an SRS is taken at phase I and a stratified random sample is taken at phase II.

Many two-phase surveys, such as the survey considered here and the majority of the surveys in the exercises for Chapter 12 of SDA, have an SRS at phase I and a stratified sample at phase II, so we show how to use the *twophase* function with the "simple" method for variance estimation.

The *id*, *weights*, and *strata* arguments to *twophase* are lists:

- We enter the weights at each phase as `weights=list(~phase1wt, ~phase2wt)`. For this application, each phase I weight, in variable *phase1wt*, equals N divided by the phase I sample size; the phase II weights in variable *phase2wt* are n_h/m_h , the observed phase I sample size in stratum h divided by the phase II sample size in stratum h .
- We use `id=list(~1,~1)` to show that there is no clustering at either phase of sampling.
- The phase I sample is an SRS, and the phase II sample is stratified by the information in variable *apc* that is gathered at phase I. This is entered as `strata=list(NULL,~apc)`, where the NULL for phase I indicates no stratification was used at that phase of sampling.

Only the records sampled in phase II have information for *vietnam*, the variable of interest. We thus analyze only the subset of records from the phase II sample; these are the records having *vietnam\$p2sample*=1.

```
data(vietnam)
print.data.frame(vietnam[1:6,])
##   apc p2sample vietnam phase1wt phase2wt finalwt p1apcsize p2apcsize
## 1 Yes      1      1      5 11.26866 56.34328      755      67
## 2 Yes      1      1      5 11.26866 56.34328      755      67
## 3 Yes      1      1      5 11.26866 56.34328      755      67
## 4 Yes      1      1      5 11.26866 56.34328      755      67
## 5 Yes      1      1      5 11.26866 56.34328      755      67
## 6 Yes      1      1      5 11.26866 56.34328      755      67
nrow(vietnam) #2064
## [1] 2064
# define logical variable to specify which obsns are selected in phase 2
vietnam$indexp2<- vietnam$p2sample==1
dphase2<-twophase(id=list(~1,~1), weights=list(~phase1wt, ~phase2wt),
                  strata=list(NULL,~apc), subset=~indexp2, data=vietnam, method="simple")
svymean(~vietnam, dphase2)
##           mean           SE
## vietnam 0.42926 0.0271
```

From the two-phase sample, the estimated percentage of Vietnam-era veterans in U.S. Veterans Administration hospitals who actually served in Vietnam is 0.42926 with a standard error of 0.0271.

12.2 Estimating the Size of a Population

12.2.1 Ratio Estimation of Population Size

As discussed in Section 13.1 of SDA, the simple two-sample capture-recapture estimate can be calculated using ratio estimation. Symmetric confidence intervals calculated using the t distribution can have poor coverage probability in small samples, so we also discuss calculating confidence intervals using inverted likelihood-ratio tests and bootstrap.

Example 13.1 of SDA. In this example, we estimate the total number of fish N in a lake together with the standard error based on a capture-recapture sample.

In the first step, catch and mark 200 fish in the lake, then release them. Allow the marked and released fish to mix with the other fish in the lake. Next, take a second, independent sample of 100 fish. Suppose that 20 of the fish in the second sample are marked. Assuming that the population of fish has not changed between the two samples and that each catch gives a simple random sample (SRS) of fish in the lake. Below is the data set illustrated in a 2×2 table.

TABLE 12.2
Data for Example 13.1 of SDA.

		In Sample 2		
		Yes	No	
In Sample 1	Yes	20	180	200
	No	80	?	?
		100	?	N

The function *svyratio* in the **survey** package can be used to estimate the population total with a symmetric confidence interval. We create a data set with records for the second sample, where every observation has a weight of 1.

```
# create data frame of records from sample 2 of size n2
n1 <- 200
n2 <- 100
m <- 20
fish<-data.frame(x=c(rep(1,m),rep(0,n2-m)),wt=rep(1,n2),n1=rep(n1,n2))
dfish<-svydesign(id=~1,weights=~wt,data=fish)
estpop<-svyratio(~n1,~x,dfish)
estpop
## Ratio estimator: svyratio.survey.design2(~n1, ~x, dfish)
## Ratios=
##      x
## n1 1000
## SEs=
##      x
## n1 201.0076
```

```
# calculate symmetric confidence interval
confint(estpop,df=n2-1)
##           2.5 %    97.5 %
## n1/x 601.1574 1398.843
```

For many applications, however, the distribution of the ratio is skewed so that a symmetric confidence interval may not have accurate coverage probability. The function *captureci* from the *SDAResources* package (Lu and Lohr, 2021) will compute a confidence interval using the method of Cormack (1992).

To apply function *captureci*, first construct a matrix *xmat*, where 1 = “in sample” and 0 = “not in sample.” For our example, *xmat* has two columns since there are two samples; the row (0,0) represents the category of not being in either sample. Next, we define *y* with the number of fish corresponding to *xmat*. Table 12.3 illustrates the definition of *xmat* and *y*.

TABLE 12.3

Matrix notation for Example 13.1 of SDA.

Description	Vector Notation, <i>xmat</i>	Number of Fish, <i>y</i>
In sample 1 and sample 2	(1,1)	20
In sample 1, not in sample 2	(1,0)	180
Not in sample 1, in sample 2	(0,1)	80

After defining *xmat* and *y*, simply type `captureci(xmat,y)` to derive the estimates.

```
# define xmat and y
xmat <- cbind(c(1,1,0),c(1,0,1))
y <- c(20,180,80)
cbind(xmat,y) # show xmat and y
##           y
## [1,] 1 1 20
## [2,] 1 0 180
## [3,] 0 1 80
captureci(xmat,y)
## $cell
## (Intercept)
##           720
##
## $N
## (Intercept)
##           1000
##
## $CI_cell
## [1] 436.199 1233.835
##
## $CI_N
## [1] 716.199 1513.835
##
## $deviance
## [1] 1.598721e-14
```

The function *captureci* reports the estimated *cell* value for the missing count for category (0,0), the estimated total $\hat{N}$, and the confidence intervals for the missing category count. The estimated total fish in the lake is 1000, with a 95% confidence interval of [716.199, 1513.835].

We can also derive a confidence interval for the population size using the bootstrap method. The first column of data *fish* contains a vector of length n_2 that indicates membership in sample 1. We can take R samples with replacement from this vector, calculate Chapman's (1951) estimate

$$\tilde{N} = \frac{(n_1 + 1)(n_2 + 1)}{m + 1} - 1$$

from each resample, and then use the R estimates from the bootstrap resamples to estimate the sampling distribution of $\tilde{N}$. We use Chapman's estimate here because it is guaranteed to be finite for every bootstrap resample, whereas $\tilde{N} = n_1 n_2 / m$ might be infinite if a resample contains no marked fish.

```
chapman<-function(y,n1) { (n1+1)*(length(y)+1)/(sum(y)+1) - 1}
Ntilde<-chapman(fish[,1],n1)
Ntilde
## [1] 965.7143
# generate 2000 bootstrap samples
nboot<-2000
set.seed(9231)
bootsamp<-matrix(sample(fish[,1], size = nrow(fish)*nboot, replace=TRUE), ncol=nboot)
dim(bootsamp) # nboot columns of resamples
## [1] 100 2000
# calculate Chapman's estimate for each column
Ntildeboot<-apply(bootsamp,2,chapman,n1)
# draw histogram of bootstrap distribution
par(las=1)
hist(Ntildeboot,xlab = "Estimated Population Size",
     main = "Histogram of Bootstrap Estimates",col="lightgray",
     breaks=20,border="white")
box(bty="l")
# calculate percentiles to get confidence interval
quantile(Ntildeboot,probs=c(0.025,0.975))
##      2.5%      97.5%
## 699.0345 1560.6154
```

The estimated confidence interval for total fish in the lake, using the bootstrap method with a seed of 9231, is [699, 1561]. The histogram in Figure 12.1 shows the distribution of the estimates from the bootstrap replicates. Note that the distribution is skewed, indicating that a symmetric confidence interval, produced under the assumption that $\tilde{N}$ follows a t distribution, is not a good choice for this data set.

12.2.2 Loglinear Models with Multiple Lists

Section 10.3 used the *svyglm* function to fit loglinear models to data from a complex survey. When loglinear models are used in multiple-recapture estimation, it is often assumed that the lists are simple random samples. This section uses the function *captureci* from package *SDAResources* to calculate confidence intervals for the missing cell and the population size. In *captureci*, the loglinear model is fit using Poisson regression with the *glm* function.

Example 13.3 of SDA. Domingo-Salvany et al. (1995) used capture–recapture to estimate the prevalence of opiate addiction in Barcelona, Spain. One of their data sets consisted of three samples from 1989: (1) a list of opiate addicts from emergency rooms (E list), (2) a list of persons who started treatment for opiate addiction during 1989, reported to the Catalonia Information System on Drug Abuse (T list), and (3) a list of heroin overdose

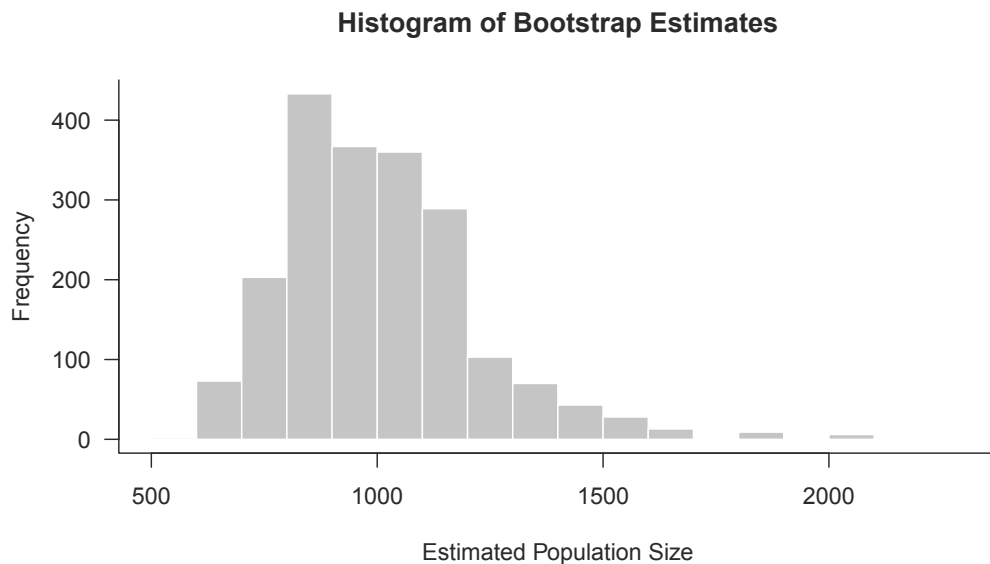


FIGURE 12.1: Histogram of population size estimates from bootstrap replicates.

deaths registered by the forensic institute in 1989 (D list). A total of 2864 distinct persons were on the three lists.

Persons on the three lists were matched, with the results listed in Table 12.4. The Membership column of the table gives the group membership in binary notation, with 1 denoting membership in the list. There are 712 observations on the T list but neither of the others, 69 observations in the D list but neither of the others, 314 observations on both the E list and the T list but not on the D list, and so on, with 6 observations on all three lists. The first line of the table shows a missing count for the units not on any of the lists.

TABLE 12.4

Notation for Example 13.3 of SDA.

List	Membership (E List, D List, T List)	Count
None	(0,0,0)	?
T	(0,0,1)	712
D	(0,1,0)	69
E	(1,0,0)	1728
DT	(0,1,1)	8
ET	(1,0,1)	314
ED	(1,1,0)	27
EDT	(1,1,1)	6

Using Table 12.4, we can define `xmat2` and `y2`, and apply function `captureci` to derive the estimates.

```
# define xmat2 and y2
xmat2<-cbind(c(1,1,1,0,1,0,0),c(1,1,0,1,0,1,0),c(1,0,1,1,0,0,1))
y2 <- c(6,27,314,8,1728,69,712)
# apply captureci
```

```
captureci(xmat2,y2)
## $cell
## (Intercept)
##      3966.743
##
## $N
## (Intercept)
##      6830.743
##
## $CI_cell
## [1] 3461.950 4547.747
##
## $CI_N
## [1] 6325.950 7411.747
##
## $deviance
## [1] 1.797782
```

From the output, the number of persons who have opiate addiction in Barcelona, Spain is estimated as 6830.743, with a CI of [6325.95, 7411.747].

Several contributed packages in R will fit these, and more complicated, capture-recapture models. The **Rcapture** package (Baillargeon and Rivest, 2007; Rivest and Baillargeon, 2019) estimates population sizes for open and closed populations using loglinear models, and will compute all of the estimates discussed in SDA (and more). The **SpadeR** (Species Prediction and Diversity Estimation) package (Chao et al., 2016) fits a variety of models for estimating population sizes and biodiversity indices.

12.3 Small Area Estimation

Many researchers have implemented algorithms for computing small area estimates (SAEs). Pratesi (2016) and Tzavidis et al. (2018) describe some of the macros and packages that have been developed to fit small area models with SAS and R software. The basic area-level and unit-level models described in SDA can be fit using the **nlme** (Pinheiro et al., 2021) or **lme4** (Bates et al., 2015, 2020) packages in R.

Numerous contributed packages are available for computing small area estimates in R; Rao and Molina (2015), Hidioglou et al. (2019), and Kreutzmann et al. (2019) describe some of the packages that have been developed. These packages calculate SAEs for a variety of situations—for example, accounting for spatial or temporal correlation among areas, or for measurement error among the covariates; using robust estimation of model parameters; or adopting a fully Bayesian approach.

The following list gives brief descriptions of three packages that have been used to produce small area estimates in various applications. All also compute the mean squared errors of the estimates.

- Package **sae**, “Small Area Estimation” (Molina and Marhuenda, 2015, 2020), calculates SAEs using a variety of models, including the Fay-Herriot model and the basic unit-level model. Models are also available that incorporate spatial or temporal information.

- Package `emdi`, “Estimating and Mapping Disaggregated Indicators” (Kreutzmann et al., 2019; Harmening et al., 2021), finds SAEs for small geographic areas, with an emphasis on poverty indicators; the package also includes a variety of diagnostic plots.
- Package `mme`, “Multinomial Mixed Effects Models” (Lopez-Vizcaino et al., 2019), fits Gaussian-multinomial models to calculate SAEs of proportions, accounting for temporal correlations.

The Asian Development Bank (2020) provides a step-by-step guide to calculating small area estimates in R, using the `sae` and `survey` packages, with code for computing and creating maps for small area poverty estimates.

12.4 Summary

Table 12.5 lists the major functions used in this chapter.

TABLE 12.5
Functions used for Chapter 12.

Function	Package	Usage
<code>sample</code>	<code>base</code>	Select with-replacement samples for use with bootstrap
<code>confint</code>	<code>stats</code>	Calculate confidence interval
<code>glm</code>	<code>stats</code>	Fit a generalized linear model (not using survey methods)
<code>svydesign</code>	<code>survey</code>	Specify the survey design
<code>svyratio</code>	<code>survey</code>	Calculate a ratio and its standard error from a survey
<code>twophase</code>	<code>survey</code>	Calculate estimates and standard errors from a two-phase survey
<code>captureci</code>	<code>SDAResources</code>	Calculate a confidence interval for population size using an inverted likelihood-ratio test

Continuing the journey. The capabilities of R continue to expand as new statistical methods are developed and implemented in packages. You can find recent contributions under “Packages” on the left panel of the website <https://cran.r-project.org/>.

Still have questions after reading this book? There are many resources available online that provide help for using R. The websites at

<https://support.rstudio.com/hc/en-us/articles/200552336-Getting-Help-with-R>
<https://www.r-bloggers.com>
<https://journal.r-project.org/>

provide links to articles and pages where members of the R user community post answers to questions. The chances are that you are not the first to have your question and that someone has posted an answer to it online.

Data Set Descriptions

The data sets referenced in SDA and described in this appendix are available from the book website (see the Preface for the website address) and in the contributed R package `SDAResources` (Lu and Lohr, 2021). These data sets are provided for instructional purposes only and without warranty. Anyone wishing to investigate the subject matter further should obtain the original data from the source. In some cases, the data sets referenced in SDA and this book are a subset of the original data; in others, the information has been modified to protect the confidentiality of the respondents.

All data sets ending in `.csv` use commas as a separator between fields.

These data sets have also been stored in SAS format (with the name ending in `.sas7bdat`) and R format with missing values recoded to the symbols used for missing data in the software package (`.` or blank in SAS and `NA` in R).

agpop.csv Data from the 1992 U.S. Census of Agriculture. Source: U.S. Bureau of the Census (1995). In columns 3–14, the value `–99` denotes missing data.

Column	Name	Value
1	county	county name (character variable)
2	state	state abbreviation (character variable)
3	acres92	number of acres devoted to farms, 1992
4	acres87	number of acres devoted to farms, 1987
5	acres82	number of acres devoted to farms, 1982
6	farms92	number of farms, 1992
7	farms87	number of farms, 1987
8	farms82	number of farms, 1982
9	largef92	number of farms with 1,000 acres or more, 1992
10	largef87	number of farms with 1,000 acres or more, 1987
11	largef82	number of farms with 1,000 acres or more, 1982
12	smallf92	number of farms with 9 acres or fewer, 1992
13	smallf87	number of farms with 9 acres or fewer, 1987
14	smallf82	number of farms with 9 acres or fewer, 1982
15	region	S = south; W = west; NC = north central; NE = northeast

agpps.csv Data from a without-replacement probability-proportional-to-size sample from file `agpop.csv`.

Column	Name	Value
1	county	county name
2	state	state abbreviation
3	acres92	number of acres devoted to farms, 1992
4	acres87	number of acres devoted to farms, 1987
5–15	...	same as variables 5–15 in <code>agpop.csv</code>
16	sizemeas	size measure used to select the pps sample
17	SelectionProb	inclusion probability for county i , π_i
18	SamplingWeight	sampling weight for county i , $w_i = 1/\pi_i$
19	Unit	unit number for indexing joint inclusion probabilities
20–34	JtProb_1– JtProb_15	columns of joint inclusion probabilities

agsrs.csv Data from an SRS of size 300 from the 1992 U.S. Census of Agriculture. Variables are the same as in `agpop.csv`. In columns 3–14, the value -99 denotes missing data.

agstrat.csv Data from a stratified random sample of size 300 from the 1992 U.S. Census of Agriculture data in `agpop.csv`. In columns 3–14, the value -99 denotes missing data.

Column	Name	Value
1	county	county name
2	state	state abbreviation
3	acres92	number of acres devoted to farms, 1992
4	acres87	number of acres devoted to farms, 1987
5	acres82	number of acres devoted to farms, 1982
6	farms92	number of farms, 1992
7	farms87	number of farms, 1987
8	farms82	number of farms, 1982
9	largef92	number of farms with 1,000 acres or more, 1992
10	largef87	number of farms with 1,000 acres or more, 1987
11	largef82	number of farms with 1,000 acres or more, 1982
12	smallf92	number of farms with 9 acres or fewer, 1992
13	smallf87	number of farms with 9 acres or fewer, 1987
14	smallf82	number of farms with 9 acres or fewer, 1982
15	region	S = south; W = west; NC = north central; NE = northeast
16	rn	random numbers used to select sample in each stratum
17	strwt	sampling weight for each county in sample

algebra.csv Hypothetical data for an SRS of 12 algebra classes in a city, from a population of 187 classes.

Column	Name	Value
1	class	Class number
2	Mi	Number of students (M_i) in class
3	score	Score of student on test

anthrop.csv Finger length and height for 3,000 criminals. Source: Macdonell (1901). This data set contains information for the entire population.

Column	Name	Value
1	finger	length of left middle finger (cm)
2	height	height (inches)

anthrs.csv Finger length and height for an SRS of size 200 from anthrop.csv.

Column	Name	Value
1	finger	length of left middle finger (cm)
2	height	height (inches)
3	wt	sampling weight

anthuneq.csv Finger length and height for a with-replacement unequal-probability sample of size 200 from **anthrop.csv**. The probability of selection, ψ_i , was proportional to 24 for $y < 65$, 12 for $y = 65$, 2 for $y = 66$ or 67, and 1 for $y > 67$.

Column	Name	Value
1	finger	length of left middle finger (cm)
2	height	height (inches)
3	wt	sampling weight

artfratio.csv Values from all possible SRSs for artificial population in Chapter 4 of SDA.

Column	Name	Value
1	sample	sample number
2	i1	first unit in sample
3	i2	second unit in sample
4	i3	third unit in sample
5	i4	fourth unit in sample
6	xbars	$\bar{x}_S$
7	ybars	$\bar{y}_S$
8	bhat	$\hat{B}$
9	tSRS	$\hat{t}_{y,SRS} = N\bar{y}_S$
10	thatr	$\hat{t}_{yr}$

asafellow.csv Information from a stratified random sample of Fellows of the American Statistical Association elected between 2000 and 2018. The list of Fellows serving as the population was downloaded from <https://www.amstat.org/ASA/Your-Career/Awards/ASA-Fellows-list.aspx> on March 18, 2019. All other information was obtained from public sources.

Column	Name	Value
1	awardyr	Year of award
2	gender	Gender of Fellow (character variable, M = male, F = female)

asafellow.csv (continued)

Column	Name	Value
3	popsize	Population size in stratum ($= N_h$)
4	sampsize	Sample size in stratum ($= n_h$)
5	field	Field of employment (character variable) acad = academia, ind = industry, govt = government
6	degreeyr	Year in which Fellow received terminal degree (year of Ph.D. if applicable, otherwise year of Master's or Bachelor's degree)
7	math	= 1 if majored in mathematics as undergraduate, 0 if did not major in math, -99 if missing

auditresult.csv Audit data used in Chapter 6 of SDA.

Column	Name	Value
1	account	audit unit
2	bookvalue	book value of account
3	psi	probability of selection
4	auditvalue	audit value of account

auditselect.csv Selection of accounts for audit data used in Chapter 6 of SDA.

Column	Name	Value
1	account	audit unit
2	bookval	book value of account
3	cumbv	cumulative book value
4	rn1	random number 1 selecting account
5	rn2	random number 2 selecting account
6	rn3	random number 3 selecting account

azcounties.csv Population and housing unit estimates for Arizona counties (excluding Maricopa and Pima counties), from the American Community Survey 2018 5-year estimates. Source: <https://data.census.gov/>, accessed November 27, 2020.

Column	Name	Value
1	number	County number
2	name	County name (character variable, length 15)
3	population	Population estimate for county
4	housing	Housing unit estimate for county
5	ownerocc	Number of owner-occupied housing units for county

baseball.csv Statistics on 797 baseball players, compiled by Jenifer Boshes from the rosters of all major league teams in November 2004. Source: Forman (2004). Missing values (for variables *pball*, *intwalk*, *hbp*, and *sacrflly*; all other variables have complete data) are coded as -9.

Column	Name	Value
1	team	team played for at beginning of the season
2	leagueid	AL or NL
3	player	a unique identifier for each baseball player
4	salary	player salary in 2004
5	pos	primary position coded as P, C, 1B, 2B, 3B, SS, RF, LF, or CF
6	gplay	games played
7	gstart	games started
8	inning	number of innings
9	putout	number of putouts
10	assist	number of assists
11	error	Errors
12	dplay	number of double plays
13	pball	number of passed balls (only applies to catchers)
14	gbat	number of games that player appeared at bat
15	atbat	number of at bats
16	run	number of runs scored
17	hit	number of hits
18	secbase	number of doubles
19	thirdbase	number of triples
20	homerun	number of home runs
21	rbi	number of runs batted in
22	stolenb	number of stolen bases
23	csteal	number of times caught stealing
24	walk	number of times walked
25	strikeout	number of strikeouts
26	intwalk	number of times intentionally walked
27	hbp	number of times hit by pitch
28	sacrhit	number of sacrifice hits
29	sacrfly	number of sacrifice flies
30	gidplay	grounded into double play

books.csv Data from homeowner's survey to estimate total number of books, used in Chapter 5.

Column	Name	Value
1	shelf	shelf number
2	Mi	number of books on shelf
3	booknumber	number of the book selected
4	purchase	purchase cost of book
5	replace	replacement cost of book

census1920.csv Population sizes for each state, from the 1920 U.S. census. The data set contains only the 48 states and excludes Washington D.C., Puerto Rico, and U.S. territories (these areas were not allowed to have voting representatives in Congress). Source: U.S. Bureau of the Census (1921).

Column	Name	Value
1	state	state name
2	population	state population in 1920 census

census2010.csv Population sizes for each state, from the 2010 U.S. census. Source: U.S. Census Bureau (2019). The data set contains only the 50 states and excludes the areas that, as of 2010, were not allowed to have voting representatives in Congress: Washington D.C., Puerto Rico, and U.S. territories.

Column	Name	Value
1	state	state name
2	population	state population in 2010 census

cherry.csv Data for a sample of 31 cherry trees. Source: Hand et al. (1994).

Column	Name	Value
1	diameter	Diameter of tree (inches)
2	height	Height of tree (feet)
3	volume	Timber volume of tree (cubic feet)

classes.csv Population sizes for 15 classes, used in Chapter 6 of SDA to illustrate unequal-probability sampling.

Column	Name	Value
1	class	Class ID number
2	class_size	Number of students in class

classpps.csv Two-stage unequal-probability sample without replacement from the population of classes in `classes.csv`.

Column	Name	Value
1	class	Class ID number
2	class_size	Number of students in class
3	finalweight	Sampling weight for student
4	hours	Number of hours spent studying statistics

classppsjp.csv Joint inclusion probabilities for unequal-probability sample without replacement from the population of classes in `classes.csv`.

Column	Name	Value
1	class	Class ID number
2	class_size	Number of students in class
3	SelectionProb	Probability of being included in sample, π_i
4	SamplingWeight	Sampling weight $w_i = 1/\pi_i$
5–9	JtProb_1–JtProb_5	Columns of joint inclusion probabilities, π_{ik}

college.csv Selected variables from the U.S. Department of Education College Scorecard Data (version updated on June 1, 2020). Source: U.S. Department of Education (2020), downloaded on August 25, 2020. Some of the variables in `college.csv` have been calculated from other variables in the original source; these have been given new variable names that are not found in the data dictionary at <https://collegescorecard.ed.gov/data/documentation/>.

This data set is made available for pedagogical purposes only. Anyone wishing to draw conclusions from College Scorecard data should obtain the full data set from the Department of Education. The original data set has 1,925 variables and includes institutions (such as those that do not grant undergraduate degrees) that are not in the file `college.csv`.

The data set `college.csv` includes institutions in the original data set that: (1) are located in the 50 states plus the District of Columbia, (2) contain information on average net price (NPT4), (3) are predominantly Bachelor’s degree-granting, (4) were currently operating as of June 2020, (5) are not private for-profit institutions or “global” campuses, (6) have Carnegie size classification (variable *ccsizset*) between 6 and 17 and Carnegie basic classification (variable *ccbasic*) between 14 and 22 (these offer Bachelor’s degrees), (7) enroll first-time students, and (8) are not U.S. Service Academies.

For all variables, missing data are coded as `–99`.

Column	Name	Value
1	unitid	Unit identification number
2	instnm	Institution name (character, length 81)
3	city	City (character, length 24)
4	stabbr	State abbreviation (character, length 2)
5	highdeg	Highest degree awarded 3 = Bachelor’s degree, 4 = Graduate degree
6	control	Control (ownership) of institution 1 = Public, 2 = Private nonprofit
7	region	Region where institution is located 1 New England (CT, ME, MA, NH, RI, VT) 2 Mid East (DE, DC, MD, NJ, NY, PA) 3 Great Lakes (IL, IN, MI, OH, WI) 4 Plains (IA, KS, MN, MO, NE, ND, SD) 5 Southeast (AL, AR, FL, GA, KY, LA, MS, NC, SC, TN, VA, WV) 6 Southwest (AZ, NM, OK, TX) 7 Rocky Mountains (CO, ID, MT, UT, WY) 8 Far West (AK, CA, HI, NV, OR, WA)
8	locale	Locale of institution 11 City: Large (population of 250,000 or more) 12 City: Midsize (population of at least 100,000 but less than 250,000) 13 City: Small (population less than 100,000) 21 Suburb: Large (outside principal city, in urbanized area with population of 250,000 or more) 22 Suburb: Midsize (outside principal city, in urbanized area with population of at least 100,000 but less than 250,000)

college.csv (continued)		
Column	Name	Value
		23 Suburb: Small (outside principal city, in urbanized area with population less than 100,000)
		31 Town: Fringe (in urban cluster up to 10 miles from an urbanized area)
		32 Town: Distant (in urban cluster more than 10 miles and up to 35 miles from an urbanized area)
		33 Town: Remote (in urban cluster more than 35 miles from an urbanized area)
		41 Rural: Fringe (rural territory up to 5 miles from an urbanized area or up to 2.5 miles from an urban cluster)
		42 Rural: Distant (rural territory more than 5 miles but up to 25 miles from an urbanized area or more than 2.5 and up to 10 miles from an urban cluster)
		43 Rural: Remote (rural territory more than 25 miles from an urbanized area and more than 10 miles from an urban cluster)
9	ccbasic	Carnegie basic classification
		15 Doctoral Universities: Very High Research Activity
		16 Doctoral Universities: High Research Activity
		17 Doctoral/Professional Universities
		18 Master's Colleges & Universities: Larger Programs
		19 Master's Colleges & Universities: Medium Programs
		20 Master's Colleges & Universities: Small Programs
		21 Baccalaureate Colleges: Arts & Sciences Focus
		22 Baccalaureate Colleges: Diverse Fields
10	ccsizset	Carnegie classification, size and setting
		6 Four-year, very small, primarily nonresidential
		7 Four-year, very small, primarily residential
		8 Four-year, very small, highly residential
		9 Four-year, small, primarily nonresidential
		10 Four-year, small, primarily residential
		11 Four-year, small, highly residential
		12 Four-year, medium, primarily nonresidential
		13 Four-year, medium, primarily residential
		14 Four-year, medium, highly residential
		15 Four-year, large, primarily nonresidential
		16 Four-year, large, primarily residential
		17 Four-year, large, highly residential
11	hbcu	Historically black college or university, 1 = yes, 0 = no
12	openadmp	Does the college have an open admissions policy, that is, does it accept any students that apply or have minimal requirements for admission? 1 = yes, 0 = no
13	adm_rate	Fall admissions rate, defined as the number of admitted undergraduates divided by the number of undergraduates who applied
14	sat_avg	Average SAT score (or equivalent) for admitted students
15	ugds	Number of number of degree-seeking undergraduate students enrolled in the fall term

college.csv (continued)		
Column	Name	Value
16	ugds_men	Proportion of <i>ugds</i> who are men
17	ugds_women	Proportion of <i>ugds</i> who are women
18	ugds_white	Proportion of <i>ugds</i> who are white (based on self-reports)
19	ugds_black	Proportion of <i>ugds</i> who are black/African American (based on self-reports)
20	ugds_hisp	Proportion of <i>ugds</i> who are Hispanic (based on self-reports)
21	ugds_asian	Proportion of <i>ugds</i> who are Asian (based on self-reports)
22	ugds_other	Proportion of <i>ugds</i> who have other race/ethnicity (created from other categories on original data file; race/ethnicity proportions sum to 1)
23	npt4	Average net price of attendance, derived from the full cost of attendance (including tuition and fees, books and supplies, and living expenses) minus federal, state, and institutional grant/scholarship aid, for full-time, first-time undergraduate Title IV-receiving students. NPT4 created from scorecard data variables NPT4_PUB if public institution and NPT4_PRIV if private
24	tuitionfee_in	In-state tuition and fees
25	tuitionfee_out	Out-of-state tuition and fees
26	avgfacsal	Average faculty salary per month
27	pftfac	Proportion of faculty that is full-time
28	c150_4	Proportion of first-year, full-time students who complete their degree within 150% of the expected time to complete; for most institutions, this is the proportion of students who receive a degree within 6 years
29	grads	Number of graduate students

collegerg.csv Five replicate SRSs from the set of public colleges and universities (having *control* = 1) in **college.csv**. Columns 1–29 are as in **college.csv**, with additional columns 30–32 listed below. Note that the selection probabilities and sampling weights are for the separate replicate samples, so that the weights for each replicate sample sum to the population size 500.

Column	Name	Value
30	selectionprob	Selection probability for each replicate sample
31	samplingweight	Sampling weight for each replicate sample
32	repgroup	Replicate group number

collshr.csv Probability-proportional-to-size sample of size 10 from the stratum of small, highly residential colleges (having *ccsiseset* = 11) in **college.csv**. Columns 1–29 are as in **college.csv**, with additional columns 30–34 listed below.

Column	Name	Value
30	mathfac	Number of mathematics faculty
31	psychfac	Number of psychology faculty
32	biolfac	Number of biology faculty
33	psii	Selection probability, = $ugds / (\text{sum of } ugds \text{ for stratum})$
34	wt	Sampling weight = $1 / (10\psi_i)$

coots.csv Selected information on egg size, from a larger study by Arnold (1991). Data provided courtesy of Todd Arnold. Not all observations are used for this data set, so results may not agree with those in Arnold (1991).

Column	Name	Value
1	clutch	clutch number from which eggs were subsampled.
2	csize	number of eggs in clutch (M_i)
3	length	length of egg (mm)
4	breadth	maximum breadth of egg (mm)
5	volume	calculated as $0.000507 * \text{length} * \text{breadth}^2$ (mm ³)
6	tmt	= 1 if received supplemental feeding, 0 otherwise

counties.csv Data (from 1990) from an SRS of 100 of the 3141 counties in the United States. Missing values are coded as −99. Source: U.S. Census Bureau (1994).

Column	Name	Value
1	RN	random number used to select the county
2	state	state abbreviation
3	county	county name
4	landarea	land area, 1990 (square miles)
5	totpop	total number of persons, 1992
6	physician	active non-Federal physicians on Jan. 1, 1990
7	enroll	school enrollment in elementary or high school, 1990
8	percpub	percent of school enrollment in public schools
9	civlabor	civilian labor force, 1991
10	unemp	number unemployed, 1991
11	farmpop	farm population, 1990
12	numfarm	number of farms, 1987
13	farmacre	acreage in farms, 1987
14	fedgrant	total expenditures in federal funds and grants, 1992 (millions of dollars)
15	fedciv	civilians employed by federal government, 1990
16	milit	military personnel, 1990
17	veterans	number of veterans, 1990
18	percviet	percent of veterans from Vietnam era, 1990

crimes.csv Data from selected variables in a simple random sample of 5,000 records from the 7,048,107 records with dates between 2001 and 2019 in the City of Chicago database “Crimes—2001 to Present.” This file was downloaded on August 11, 2020 from <https://data.cityofchicago.org/>. These data are provided for pedagogical purposes only. Anyone

wishing to publish analyses of Chicago crime data should obtain the most recent data from <https://data.cityofchicago.org/>. For a list and map of Community Areas, see https://www.chicago.gov/city/en/depts/dgs/supp_info/citywide_maps.html.

Column	Name	Value
1	year	Year in which crime occurred (between 2001 and 2019)
2	crimetype	Type of crime, determined from detailed crime description in database homicide = homicide, sexualasslt = sexual assault, robbery = robbery, aggasslt = aggravated assault, burglary = burglary, mvtheft = motor vehicle theft, idtheft = identity theft, theft = other type of theft, arson = arson, simpleasslt = simple assault (assaults that are not aggravated), threat = threat or harassment, fraud = fraud, weapon = weapons violation, trespass = trespassing, narcotics = narcotics or liquor law violation, vandalism = vandalism, other = other
3	violent	= 1 if violent crime, 0 otherwise
4	arrest	= 1 if an arrest was made, 0 otherwise
5	domestic	= 1 if crime was domestic-related as defined by the Illinois Domestic Violence Act, 0 otherwise
6	commarea	Number of the Community Area in Chicago where the crime occurred
7	location	Type of location where crime occurred (e.g., street, apartment)

deadtrees.csv Number of dead trees recorded by photograph and field count for a (fictional) SRS of 25 plots taken from a population of 100 plots.

Column	Name	Value
1	photo	Number of dead trees in plot from photograph
2	field	Number of dead trees in plot from field observation

divorce.csv Data from a sample of divorce records for states in the Divorce Registration Area. Source: National Center for Health Statistics (1987).

Column	Name	Value
1	state	state name (character variable)
2	abbrev	state abbreviation (character variable)
3	samprate	sampling rate for state
4	numrecs	number of records sampled in state
5	hsblt20	number of records in sample with husband's age < 20
6	hsb20to24	number of records with $20 \leq$ husband's age ≤ 24
7	hsb25to29	number of records with $25 \leq$ husband's age ≤ 29
8	hsb30to34	number of records with $30 \leq$ husband's age ≤ 34
9	hsb35to39	number of records with $35 \leq$ husband's age ≤ 39
10	hsb40to44	number of records with $40 \leq$ husband's age ≤ 44

divorce.csv (continued)		
Column	Name	Value
11	hsb45to49	number of records with $45 \leq \text{husband's age} \leq 49$
12	hsbge50	number of records with husband's age ≥ 50
13	wflt20	number of records with wife's age < 20
14	wf20to24	number of records with $20 \leq \text{wife's age} \leq 24$
15	wf25to29	number of records with $25 \leq \text{wife's age} \leq 29$
16	wf30to34	number of records with $30 \leq \text{wife's age} \leq 34$
17	wf35to39	number of records with $35 \leq \text{wife's age} \leq 39$
18	wf40to44	number of records with $40 \leq \text{wife's age} \leq 44$
19	wf45to49	number of records with $45 \leq \text{wife's age} \leq 49$
20	wfge50	number of records with wife's age ≥ 50

gini.csv Data from the population of districts for the 1921 Italian general census. Source: Gini and Galvani (1929, pp. 73–78).

Column	Name	Value
1	id	ID number
2	district	District name
3	birth_rate	Births per 1,000 population
4	death_rate	Deaths per 1,000 population
5	marriage_rate	Marriages per 1,000 population
6	agricultural_pop	Percentage of males over 10 years old who work in agriculture
7	urban_population	Percentage of population in urban areas
8	income	Average income
9	altitude	Average altitude above sea level (meters)
10	pop_density	Number of inhabitants per square kilometer
11	natural_growth	Rate of average increase of the population
12	population	Population of area
13	area	Land area (square kilometers)
14	in_GG_sample	= 1 if in the purposive sample selected by Gini and Galvani; 0 otherwise

golfsrs.csv A simple random sample of 120 golf courses, taken from the population on the website ww2.golfcourse.com on August 5, 1998. Missing data in the .csv file are denoted by blanks.

Column	Name	Value
1	RN	random number used to select golf course for sample
2	state	state name
3	holes	number of holes
4	type	type of course: priv = private, semi = semi-private, pub = public, mili = military, resort
5	yearblt	year course was built
6	wkday18	greens fee for 18 holes during week
7	wkday9	greens fee for 9 holes during week
8	wkend18	greens fee for 18 holes on weekend

golfsrs.csv (continued)		
Column	Name	Value
9	wkend9	greens fee for 9 holes on weekend
10	backtee	back tee yardage
11	rating	course rating
12	par	par for course
13	cart18	golf cart rental fee for 18 holes
14	cart9	golf cart rental fee for 9 holes
15	caddy	Are caddies available? (y or n)
16	pro	Is a golf pro available? (y or n)

gpa.csv GPA data from Chapter 5 of SDA.

Column	Name	Value
1	suite	Suite (psu) identifier
2	gpa	Grade point average of person in suite
3	wt	Sampling weight, = 20 for every observation

healthjournals.csv Randomization and statistical inference practices in a stratified random sample of 196 public health articles. The data, provided courtesy of Dr. Matt Hayat, are discussed in Hayat and Knapp (2017). The variables provided in **healthjournals.csv** are a subset of the variables collected by the authors.

Column	Name	Value
1	journal	Journal that published the article AJPH = <i>American Journal of Public Health</i> AJPM = <i>American Journal of Preventive Medicine</i> PM = <i>Preventive Medicine</i>
2	NumAuthors	Number of authors
3	RandomSel	= “Yes” if data in the article were from a randomly selected (probability) sample; “No” otherwise
4	RandomAssn	= “Yes” if study subjects for the article were randomly assigned to treatment groups; “No” otherwise
5	ConfInt	= “Yes” if a confidence interval appeared in the article’s main text, tables, or figures; “No” otherwise
6	HypTest	= “Yes” if a p-value or significance test appeared in the article’s main text, tables, or figures; “No” otherwise
7	Asterisks	= “Yes” if asterisks were used to represent p-value ranges; “No” otherwise

htcdf.csv Empirical distribution function and empirical probability mass function of data in **htpop.csv**.

Column	Name	Value
1	height	height value, cm
2	frequency	number of times height value in column 1 occurs in population
3	epmf	empirical probability mass function
4	ecdf	empirical distribution function

htpop.csv Height and gender of 2,000 persons in an artificial population.

Column	Name	Value
1	height	height of person, cm
2	gender	M=male, F=female

htsrs.csv Height and gender for a SRS of 200 persons, taken from **htpop.csv**.

Column	Name	Value
1	rn	random number used to select unit
2	height	height of person, cm
3	gender	M=male, F=female

htstrat.csv Height and gender for a stratified random sample of 160 women and 40 men, taken from **htpop.csv**. The columns and names are as in **htsrs.csv**.

hunting.csv Population and sample sizes for the poststrata used for the Sunday hunting survey. Source: Virginia Polytechnic and State University/Responsive Management (2006).

Column	Name	Value
1	region	Region of state (East, Central, West)
2	gender	Gender (female, male)
3	age	Age group (16-24, 25-34, 35-44, 45-54, 55-64, 65+)
4	popsize	Population size in poststratum from the 2000 U.S. census
5	sampsize	Sample size in poststratum

impute.csv Small artificial data set used to illustrate imputation methods. Missing values are denoted by -99.

Column	Name	Value
1	person	identification number for person
2	age	age in years
3	gender	M=male, F=female
4	education	number of years of education
5	crime	= 1 if victim of any crime, 0 otherwise
6	violcrime	= 1 if victim of violent crime, 0 otherwise

integerwt.csv Artificial population of 2,000 observations.

Column	Name	Value
1	stratum	Stratum number
2	y	y value of observation

intellonline.csv Data from the online (Mechanical Turk) survey. Source: Heck et al. (2018). The data were downloaded from <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0200103> on February 8, 2020; the variables extracted from the full data set are provided here for educational purposes only.

Column	Name	Value
1	int	Response to question about agreement with the statement “I am more intelligent than the average person.” 1 = Strongly Agree; 2 = Mostly Agree; 3 = Mostly Disagree; 4 = Strongly Disagree; 5 = Don’t Know or Not Sure
2	region	Census region of respondent (character variable, length 10): Northeast, South, Midwest, West
3	sex	Sex (character variable, length 8): Male, Female
4	race	Race (character variable, length 18): White, African American, Asian American, Hispanic American, Another origin
5	age	Age, years
6	income	Household income level (character variable, length 8): <\$40k, \$40–80k, or >\$80k
7	education	Highest education level attained (character variable, length 12): No College, Some College, College Grad, Grad School
8	postwt	Relative weight, obtained by poststratifying to demographic proportions in the 2010 U.S. Census. The weights are normed so that they sum to 750.

intelltel.csv Data from the telephone survey studied by Heck et al. (2018). The data were downloaded from <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0200103> and are provided here for educational purposes only. The variables are the same as in **intellonline.csv**.

intellwts.csv Relative weights for demographic groups in **intellonline.csv** and **intelltel.csv** (Heck et al., 2018). Each sample was weighted using the 2010 U.S. Census demographics for sex (male, female), age (< 44, ≥ 44), and race/ethnicity (white, nonwhite). The table entries give the weights for each of these eight demographic groups.

Column	Name	Value
1	sex	Sex
2	agegroup	Age group: Young = (age less than 44), Old = (age greater than or equal to 44)
3	race	Race: White or Nonwhite
4	tel_n	Number of telephone survey respondents in the sex/age-group/race class

intellwts.csv (continued)		
Column	Name	Value
5	online_n	Number of online survey respondents in the sex/agegroup/race class
6	tel_wgt	Relative weight for each respondent to the telephone survey in this sex/agegroup/race class
7	online_wgt	Relative weight for each respondent to the telephone survey in this sex/agegroup/race class

ipums.csv Data extracted from the 1980 Census Integrated Public Use Microdata Series, using the “Small Sample Density” option in the data extract tool, on September 17, 2008. The stratum and psu variables were constructed for use in the book exercises. Data analyses on this file do NOT give valid results for inference to the 1980 U.S. population. Source: Ruggles et al. (2004).

Column	Name	Value
1	stratum	stratum number (1–9)
2	psu	psu number (1–90)
3	inctot	total personal income (dollars), topcoded at \$75,000
4	age	age, with range 15–90
5	sex	1 = Male, 2 = Female
6	race	1 = White, 2 = Black, 3 = American Indian or Alaska Native, 4 = Asian or Pacific Islander, 5 = Other Race
7	hispanic	0 = Not Hispanic, 1 = Hispanic
8	marstat	Marital Status: 1 = Married, 2 = Separated, 3 = Divorced, 4 = Widowed, 5 = Never married/single
9	ownershg	Ownership of housing unit: 0 = Not Applicable (N/A), 1 = Owned or being bought, 2 = Rents
10	yrsusa	Number of years a foreign-born person has lived in the U.S.: 0= N/A, 1= 0–5 years, 2= 6–10 years, 3= 11–15 years, 4= 16–20 years, 5= 21+ years
11	school	Is person in school? 0 = N/A, 1 = No, not in school, 2 = Yes, in school
12	educrec	Educational Attainment: 1= None or preschool, 2= Grade 1, 2, 3, or 4, 3= Grade 5, 6, 7, or 8, 4= Grade 9, 5= Grade 10, 6= Grade 11, 7= Grade 12, 8= 1 to 3 years of college, 9= 4+ years of college
13	labforce	In labor force? 0 = Not Applicable, 1 = No, 2 = Yes
14	classwk	class of worker: 0=Not applicable, 13= Self-employed, not incorporated, 14= Self-employed, incorporated, 22= Wage/salary, private, 25= Federal government employee, 27= State government employee, 28= Local government employee, 29= Unpaid family worker
15	vetstat	Veteran Status 0 = Not Applicable, 1 = No Service, 2 = Yes

journal.csv Types of sampling used for articles in a sample of journals. Source: Jacoby and Handlin (1991).

Note that columns 2 and 3 do not always sum to column 1; for some articles, the investigators could not determine which type of sampling was used. When working with these data, you may wish to create a fourth column, “indeterminate,” which equals $\text{column1} - (\text{column2} + \text{column3})$.

Column	Name	Value
1	numemp	number of articles in 1988 that used sampling
2	prob	number of articles that used probability sampling
3	nonprob	number of articles that used non-probability sampling

measles.csv Roberts et al. (1995) reported on the results of a survey of parents whose children had not been immunized against measles during a recent campaign to immunize all children in the first five years of secondary school. The original data were unavailable; univariate and multivariate summary statistics from these artificial data, however, are consistent with those in the paper. All variables are coded as 1 for yes, 0 for no, and 9 for no answer. A parent who refused consent (variable 4) was asked why, with responses in variables 5 through 10. If a response in variables 5 through 10 was checked, it was assigned value 1; otherwise, it was assigned value 0. A parent could give more than one reason for not having the child immunized.

Column	Name	Value
1	school	school attended by child
2	form	Parent received consent form
3	returnf	Parent returned consent form
4	consent	Parent gave consent for measles immunization
5	hadmeas	Child had already had measles
6	previmm	Child had been immunized against measles
7	sideeff	Parent concerned about side effects
8	gp	Parent wanted general practitioner (GP) to give vaccine
9	noshot	Child did not want injection
10	notser	Parent thought measles was not a serious illness
11	gpadv	GP advised that vaccine was not needed
12	Mitotal	Population size in school
13	mi	Sample size in school

mysteries.csv Data from a stratified random sample of books nominated for the Edgar[®] awards for Best Novel and Best First Novel. The sample was drawn from the population listing of 655 books at <http://theedgars.com/awards/> on August 14, 2020.

Column	Name	Value
1	stratum	Stratum number, from 1 to 12, computed from the stratification variables in columns 2–4
2	time	Time period in which award was given: 1 = 1946–1980, 2 = 1981–2000, 3 = 2001–2020
3	category	Award category (character variable, length 16): Best Novel, or Best First Novel

mysteries.csv (continued)		
Column	Name	Value
4	winner	= 1 if book won the award that year, = 0 if book was nominated but did not win award
5	popsize	Number of population books in stratum ($= N_h$)
6	sampsize	Number of sampled books in stratum ($= n_h$)
7	obtained	= 1 if book was obtained (responded) in original sample, = 2 if book was obtained in phase II subsample of nonrespondents, = 0 if not obtained
8	p1weight	Weight for phase I sample, calculated as N_h/n_h ; use for exercises in Chapters 1–11 of SDA
9	p2weight	Final weight for phase II sample; use for exercises in Chapter 12 of SDA and analyses involving variables <i>victims</i> and <i>firearm</i>
10	genre	Genre of book (character variable, length 11). Values “private eye” (protagonist is a private detective), “procedural” (a detailed, step-by-step analysis of how the crime is solved, using the skills of the detective), or “suspense” (the protagonist is at the center of action or is involved in espionage, but is not a professional detective)
11	historical	= 1 if the main action in the book takes place at least 20 years before the book’s publication date, = 0 if book action is within 20 years of the publication date
12	urban	= 1 if the main action in the book takes place primarily in urban areas, = 0 otherwise
13	authorgender	Gender of author (character variable, length 1) = “F” if author is female, “M” if author is male
14	fdetect	Number of female detectives (or protagonists, if book has no detective) in book
15	mdetect	Number of male detectives (or protagonists, if book has no detective) in book
16	victims	Number of murder victims in book (missing value set to -9 if <i>obtained</i> = 0)
17	firearm	Number of murders committed with firearms in book (missing value set to -9 if <i>obtained</i> = 0)

nhanes.csv Selected variables from the 2015–2016 National Health and Nutrition Examination Survey (NHANES). Source: Centers for Disease Control and Prevention (2017). This data set is provided for educational purposes only. Anyone wishing to publish or use results from analyses of NHANES data should obtain the data files directly from the source.

The data files merged to create **nhanes.csv** can be read directly from the SAS transport files DEMO_I.XPT, BMX_I.XPT, TCHOL_I.XPT, and BPX_I.XPT from the NHANES website. Variables 1–23 have the same names as in the SAS transport files.

The blood pressure variables *sbp* and *dbp* were created as follows. In the medical examination, three consecutive blood pressure readings were obtained after participants sat quietly for 5 minutes, and the maximum inflation level was determined. A fourth measurement was conducted for some persons who had an incomplete or interrupted blood pressure reading.

The variables *sbp* and *dbp* were calculated by discarding the first blood pressure reading and calculating the average of the remaining valid readings. Note that some of the diastolic blood pressure readings are 0.

In the comma-delimited file `nhanes.csv`, missing values are denoted by `-9`. In the SAS data file, missing values are denoted by a period. In the R data file, missing values are denoted by `NA`. Note that some of the codes for variables in the table below also denote missing values; for example, the value 7 for *dmdeduc2* indicates “Refused,” and these codes for special types of missing values remain in the SAS and R data files.

Column	Name	Value
1	<i>sdmvstra</i>	Pseudo-stratum. These are groups of secondary sampling units used for variance estimation on the publicly available data. Pseudo-strata and pseudo-psus are released instead of the actual strata and psus to protect the confidentiality of respondents’ information. Use <i>sdmvstra</i> as the variable defining the strata.
2	<i>sdmvpsu</i>	Pseudo-psu. Use <i>sdmvpsu</i> as the primary sampling unit (psu). There are two pseudo-psus per pseudo-stratum, numbered 1 and 2.
3	<i>wtint2yr</i>	Interview weight (use as weight for variables 5–12)
4	<i>wtmec2yr</i>	Mobile Examination Center weight (use as weight for any analysis involving variables 13–25)
5	<i>ridstatr</i>	Interview/examination status, = 1 if interviewed only, = 2 if interviewed and had medical examination
6	<i>ridageyr</i>	Age in years at screening, from 0 to 80. Anyone with age > 80 years is recorded (topcoded) as 80. No values are missing for this variable.
7	<i>ridagemn</i>	Age in months at screening (reported only for persons with age 24 months or younger at the time of exam, otherwise missing)
8	<i>riagendr</i>	= 1 if male, 2 if female (no missing values)
9	<i>ridreth3</i>	Race/ethnicity code (no missing values) 1 = Mexican American 2 = Other Hispanic 3 = Non-Hispanic White 4 = Non-Hispanic Black 6 = Non-Hispanic Asian 7 = Other Race, Including Multi-Racial
10	<i>dmdeduc2</i>	Education level of person interviewed (given for adults age 20+ only) 1 = Less than 9th grade 2 = 9th to 11th grade (including 12th grade with no diploma) 3 = High school graduate (including GED) 4 = Some college or associate’s degree 5 = College graduate or above 7 = Refused 9 = Don’t know
11	<i>dmdfmsiz</i>	Total number of people in the family. Values 1–6 indicate the number of people is that number; value 7 indicates 7 or more people in family. No missing values.

nhanes.csv (continued)		
Column	Name	Value
12	indfmpir	Ratio of family income to poverty guideline. A value less than 1 indicates the family is below the poverty threshold. Variable <i>indfmpir</i> is a continuous variable where values between 0 and 4.99 indicate the actual poverty ratio. A value of 5 indicates that the ratio of family income to the poverty guideline for that family is 5 or more.
13	bmxtwt	Weight (kg)
14	bmxtht	Standing height (cm)
15	bmxbmi	Body mass index (kg/m ²), calculated as $\text{bmxtwt}/(\text{bmxtht}/100)^2$
16	bmxtwaist	Waist circumference (cm)
17	bmxtleg	Upper leg length (cm)
18	bmxtarm1	Upper arm length (cm)
19	bmxtarmc	Upper arm circumference (cm)
20	bmdavasad	Average sagittal abdominal diameter (SAD, the distance from the small of the back to the upper abdomen), in cm. Calculated by averaging the SAD readings on the person (up to four).
21	lbxtc	Serum total cholesterol (mg/dL)
22	bpxpls	60-second pulse
23	sbp	Average systolic blood pressure (mm Hg)
24	dbp	Average diastolic blood pressure (mm Hg)
25	bpread	Number of blood pressure readings

nybight.csv Data collected in the New York Bight for June 1974 and June 1975. Two of the original strata were combined because of insufficient sample sizes. For variable *catchwt*, weights less than 0.5 were recorded as 0.5 kg. Source: Wilk et al. (1977).

Column	Name	Value
1	year	year of data collection, 1974 or 1975
2	stratum	stratum membership, based on depth
3	catchnum	number of fish caught during trawl
4	catchwt	total weight (kg) of fish caught during trawl
5	numsp	number of species of fish caught during trawl
6	depth	depth of station (m)
7	temp	surface temperature (degrees C), missing = -99

otters.csv Data on number of holts (dens) in Shetland, U.K., used in Kruuk et al. (1989). Data courtesy of Hans Kruuk.

Column	Name	Value
1	section	section of coastline
2	habitat	type of habitat (stratum)
3	holts	number of holts (dens)

ozone.csv Hourly ozone readings (parts per billion, ppb) from a site in Monterey County, California, for 2018 and 2019. Source: https://aqs.epa.gov/aqsweb/airdata/download_files.html#Raw, accessed November 19, 2020. Missing values are denoted by -9.

Column	Name	Value
1	year	year of reading (2018 or 2019)
2	month	month of reading (1–12)
3	day	day of reading (1–31)
4	hr0	ozone reading (ppb) at 0:00 local time
5	hr1	ozone reading (ppb) at 1:00 local time
⋮	⋮	⋮
27	hr23	ozone reading (ppb) at 23:00 local time

pitcount.csv Fictional data from a fictional point-in-time (PIT) survey taken to estimate the number of persons experiencing homelessness.

Column	Name	Value
1	strat	Stratum number (from 1 to 8)
2	division	Geographic division, used to form strata
3	density	Expected density of persons experiencing homelessness (character variable, with values High or Low)
4	popsize	$= N_h$, the number of areas in the population for stratum h
5	sampsize	$= n_h$, the number of areas in the sample for stratum h
6	areawt	$= N_h/n_h$, the sampling weight for the area
7	y	Number of persons experiencing unsheltered homelessness found in the area during the PIT count

profresp.csv The data described in Zhang et al. (2020) were downloaded from <http://doi.org/10.3886/E109021V1> on January 22, 2020, from file **survey4.rds**. The data set **profresp.csv** contains selected variables from the set of 2,407 respondents who completed the survey and provided information on the demographic variables and the information needed to calculate “professional respondent” status. The full data set **survey4.rds** contains numerous additional questions about behavior that are not included here, as well as the data from the partially completed surveys. The website also contains data for three other online panel surveys. Because **profresp.csv** is a subset of the full data, statistics calculated from it may differ from those in Zhang et al. (2020).

Missing values are denoted by –9.

Column	Name	Value
1	prof_cat	Level of professionalism 1 = novice, 2 = average, 3 = professional
2	panelnum	Number of panels respondent has belonged to. A response between 1 and 6 means that the person has belonged to that number of panels; 7 means 7 or more.
3	survnum_cat	How many Internet surveys have you completed before this one? 1 = This is my first one, 2 = 1–5, 3 = 6–10, 4 = 11–15, 5 = 16–20, 6 = 21–30, 7 = More than 30
4	panelq1	Are you a member of any online survey panels besides this one? 1 = yes, 2 = no
5	panelq2	To how many other online panels do you belong?

profresp.csv (continued)		
Column	Name	Value
		1 = None, 2 = 1 other panel, 3 = 2 others, 4 = 3 others, 5 = 4 others, 6 = 5 others, 7 = 6 others or more. This question has a missing value if <i>panelq1</i> = 2. If you want to estimate how many panels a respondent belongs to, create a new variable <i>numpanel</i> that equals <i>panelq2</i> if <i>panelq2</i> is not missing and equals 1 if <i>panelq1</i> = 2.
6	age4cat	Age category. 1 = 18 to 34, 2 = 35 to 49, 3 = 50 to 64, 4 = 65 and over
7	edu3cat	Education category. 1 = high school or less, 2 = some college or associates' degree, 3 = college graduate or higher
8	gender	Gender: 1 = male, 2 = female
9	non_white	1 = race is non-white, 0 = race is white
10	motive	Which best describes your main reason for joining on-line survey panels? 1 = I want my voice to be heard, 2 = Completing surveys is fun, 3 = To earn money, 4 = Other (Please specify)
11	freq_q1	During the PAST 12 MONTHS, how many times have you seen a doctor or other health care professional about your own health? Response is number between 0 and 999.
12	freq_q2	During the PAST MONTH, how many days have you felt you did not get enough rest or sleep?
13	freq_q3	During the PAST MONTH, how many times have you eaten in restaurants? Please include both full-service and fast food restaurants.
14	freq_q4	During the PAST MONTH, how many times have you shopped in a grocery store? If you shopped at more than one grocery store on a single trip, please count them separately.
15	freq_q5	During the PAST 2 YEARS, how many overnight trips have you taken?

profrespacs.csv Population estimates from the 2011 American Community Survey (ACS) for age/gender/education categories measured in **profresp.csv** (Zhang et al., 2020). Note that *age3cat* has 3 categories, while the age variable in **profresp.csv** has 4 categories.

Column	Name	Value
1	gender	Gender: 1 = male, 2 = female
2	age3cat	Age category. 1 = 18 to 34, 2 = 35 to 64, 3 = 65 and over
3	edu3cat	Education category. 1 = high school or less, 2 = some college or associates' degree, 3 = college graduate or higher
4	count	Population size from ACS for the gender/age/education level combination

radon.csv Radon readings for a stratified sample of 1003 homes in Minnesota. Source: Nolan and Speed (2000). The data were downloaded in April 2008 from an earlier version of the website now located at www.stat.berkeley.edu/users/statlabs/labs.html.

Column	Name	Value
1	countyname	County Name
2	countynum	County Number
3	sampsize	Sample size in county
4	popsize	Population size in county
5	radon	Radon concentration (picocuries per liter)

rectlength.csv Lengths of rectangles.

Column	Name	Value
1	rectangle	Rectangle number
2	length	Rectangle length

rnt.csv Page from a random number table. Open the .csv file in a text editor instead of a spreadsheet, because a spreadsheet strips off the leading zeroes. The columns have format z5.0 in the SAS file, and are character variables in the R file, so that leading zeroes are displayed in those formats.

Column	Name	Value
1	col1	Column of 5-digit random numbers
2	col2	Column of 5-digit random numbers
3	col3	Column of 5-digit random numbers
4	col4	Column of 5-digit random numbers
5	col5	Column of 5-digit random numbers
6	col6	Column of 5-digit random numbers

sample70.csv All possible simple random samples that can be generated from the population in Example 2.2 of SDA.

Column	Name	Value
1	sampnum	Sample number
2–5	u1–u4	Sampled units in $\mathcal{S}$
6–9	y1–y4	Values of y_i in sample $\mathcal{S}$
10	total	Estimated population total

santacruz.csv The number of seedlings in the sampled psus on Santa Cruz Island, California, in 1992 and 1994. Source: Peart (1994).

Column	Name	Value
1	tree	Tree number
2	seed92	Number of seedlings in 1992
3	seed94	Number of seedlings in 1994

schools.csv Math and reading test results from a two-stage cluster sample of tenth-grade students. An SRS of 10 schools was selected from the 75 schools in the population, and then 20 students were sampled from each school. These data are fictional but the summary statistics are consistent with those seen in educational studies.

Column	Name	Value
1	schoolid	School number (use as cluster variable)
2	gender	Gender of student (character variable, F = female, M = male)
3	math	Score on math test
4	reading	Score on reading test
5	mathlevel	Category level for math test score: 1 if $1 \leq \text{math} \leq 40$ 2 if $41 \leq \text{math}$
6	readlevel	Category level for reading test score: 1 if $1 \leq \text{read} \leq 32$ 2 if $33 \leq \text{read} \leq 50$
7	Mi	Number of students in school, M_i
8	finalwt	Weight for student in sample

seals.csv Data on number of breathing holes found in sampled areas of Svalbard fjords, reconstructed from summary statistics given in Lydersen and Ryg (1991).

Column	Name	Value
1	zone	zone number for sampled area
2	holes	number of breathing holes Imjak found in area

shapespop.csv Population of black and gray squares and circles.

Column	Name	Value
1	ID	identification number for object
2	shape	shape of object (square or circle)
3	color	color of object (gray or black)
4	area	area of object (cm^2)
5	conv	= 1 if object can be reached through convenience sample, 0 otherwise

shorebirds.csv Two-phase sample of shorebird nests. These are artificial data constructed from summary statistics given in Bart and Earnst (2002).

Column	Name	Value
1	plot	Plot number
2	rapid	Rapid-method count of number of birds in plot
3	intense	Intensive-method count of number of nests in plot = -9 if the plot is not in the phase II sample

sp500.csv Companies in the S&P 500[®] Stock Market Index as of September 15, 2020. Source: Downloaded from <https://fknol.com/list/eps-sp-500-index-companies.php> on September 19, 2020.

Column	Name	Value
1	Company	Company name (character variable, length 37)
2	Symbol	Stock symbol (character variable, length 5)
3	MarketCap	Market capitalization, in billions of U.S. dollars
4	StockPrice	Price per share of stock
5	PE_Ratio	Price-to-earnings ratio
6	EPS	Earnings per share

spanish.csv Fictional cluster sample of introductory Spanish students.

Column	Name	Value
1	class	Class number
2	score	Score on vocabulary test (out of 100)
3	trip	= 1 if plan a trip to a Spanish-speaking country, 0 otherwise

srs30.csv An SRS of size 30 taken from an artificial population of size 100.

Column	Name	Value
1	y	Value of observation

ssc.csv SRS of 150 members of the Statistical Society of Canada, downloaded from `ssc.ca` in August, 2006.

Column	Name	Value
1	gender	m = male, f=female
2	occupation	a = academic, g = government, i = industry, n = not determined
3	ASA	= 1 if person is member of American Statistical Association, 0 otherwise

statepop.csv Data from an unequal-probability sample of 100 counties from the 1994 *County and City Data Book* (U.S. Census Bureau, 1994). The sample was selected with probability proportional to population.

Column	Name	Value
1	county	county name (character variable, length 14)
2	state	state name (character variable)
3	landarea	land area of county, 1990 (square miles)
4	popn	population of county, 1992
5	phys	number of physicians, 1990
6	farmpop	farm population, 1990
7	numfarm	number of farms, 1987
8	farmacre	number of acres devoted to farming, 1987
9	veterans	number of veterans, 1990
10	percviet	percent of veterans from Vietnam era, 1990
11	psii	ψ_i , probability of selection
12	wt	sampling weight, $= 1/(100\psi_i)$

statepps.csv Number of counties (or county equivalents; Alaska has boroughs, Louisiana has parishes, and some states have independent cities), population estimates for 2019, land area, and water area for the 50 states plus the District of Columbia. Total area for a state can be calculated by summing land area and water area.

Source: Population estimates are from U.S. Census Bureau (2019). Land and water areas are from U.S. Census Bureau (2012).

Column	Name	Value
1	state	state name (character variable, length 20)
2	counties	number of counties or county equivalents
3	pop2019	population of state, 2019
4	landarea	land area of state (square kilometers)
5	waterarea	water area of state (square kilometers)

swedishlcs.csv Data on call attempts from the Swedish Survey of Living Conditions. Source: Lundquist and Särndal (2013).

Column	Name	Value
1	attempt	call attempt number
2	resprate	response rate at call attempt (percent)
3	benefits	relative bias for variable <i>benefits</i>
4	income	relative bias for variable <i>income</i>
5	employed	relative bias for variable <i>employed</i>
6	note	Character variable, length 25: notes about data collection

The variable *attempt* takes on values 1–25 for the initial fieldwork period. Values 31–40 denote the follow-up period, and value 45 gives the final estimates. The gaps in the attempt variable allow one to see the separation of the periods on the graph.

syc.csv Selected variables from the Survey of Youth in Custody (Beck et al., 1988). Source: U.S. Department of Justice (1989). Strata 6–16 each contain one facility; the psus in those strata are residents. In strata 1–5, the psus are facilities. The number of facilities in the population (N_h) for those five strata are: $N_1 = 99$, $N_2 = 39$, $N_3 = 30$, $N_4 = 13$, $N_5 = 14$. Eleven facilities are sampled from stratum 1, and seven facilities are sampled from each of strata 2–5.

The table gives missing value codes for individual variables in the .csv file (these codes are the same as in the original data source, but have been changed to the appropriate missing value codes for the respective software packages in the SAS and R data files).

Column	Name	Value
1	stratum	stratum number
2	psu	psu number, = facility number for residents in strata 1–5 and person number for residents in strata 6–16
3	facility	facility number
4	facsize	number of eligible residents in psu
5	finalwt	final weight
6	randgrp	random group number
7	age	age of resident (99=missing)

syc.csv (continued)		
Column	Name	Value
8	race	race of resident 1 = white; 2 = Black; 3 = Asian/Pacific Islander; 4 = American Indian, Alaska Native; 5 = Other; 9 = Missing
9	ethnicity	1 = Hispanic, 2 = not Hispanic, 9=missing
10	educ	highest grade attended before sent to correctional institution 0 = Never attended school; 1–12 = highest grade attended; 13 = GED; 14 = Other
11	gender	1 = male, 2 = female
12	livewith	Who did you live with most of the time you were growing up? 1 = Mother only, 2 = Father only 3 = Both mother and father, 4 = Grandparents, 5 = Other relatives, 6 = Friends, 7 = Foster home, 8 = Agency or institution, 9 = Someone else, 99 = Blank
13	famtime	Has anyone in your family, such as your mother, father, brother, sister, ever served time in jail or prison? 1 = Yes, 2 = No, 7 = Don't know, 9 = Blank
14	crimtype	most serious crime in current offense 1 = violent (e.g., murder, rape, robbery, assault) 2 = property (e.g., burglary, larceny, arson, fraud, motor vehicle theft) 3 = drug (drug possession or trafficking) 4 = public order (weapons violation, perjury, failure to appear in court) 5 = juvenile status offense (truancy, running away, incorrigible behavior) 9 = missing
15	everviol	ever put on probation or sent to correctional inst for violent offense: 1 = yes, 0 = no
16	numarr	number of times arrested (99=missing)
17	probtn	number of times on probation (99=missing)
18	corrinst	number of times previously committed to correctional institution (99=missing)
19	vertime	Prior to being sent here did you ever serve time in a correctional institution? 1 = yes, 2 = no, 9 = missing
20	prviol	=1 if previously arrested for violent offense, 0 otherwise
21	prprop	=1 if previously arrested for property offense, 0 otherwise
22	prdrug	=1 if previously arrested for drug offense, 0 otherwise
23	prpub	=1 if previously arrested for public order offense, 0 otherwise
24	prjuv	=1 if previously arrested for juvenile status offense, 0 otherwise
25	agefirst	age first arrested (99=missing)
26	usewepn	Did you use a weapon ... for this incident? 1 = Yes, 2 = No, 9 = Blank
27	alcuse	Did you drink alcohol at all during the year before being sent here this time? 1 = Yes; 2 = No, didn't drink during year before; 3 = No, don't drink at all, 9 = missing
28	everdrug	Ever used illegal drugs; 0=no, 1=yes, 9=missing

teachers.csv Selected variables from a study on elementary school teacher workload in Maricopa County, Arizona. Data courtesy of Rita Gnap (Gnap, 1995). The psu sizes are given in file **teachmi.csv**. The large stratum had 245 schools; the small/medium stratum had 66 schools. Missing values are coded as -9 .

Column	Name	Value
1	dist	school district size. Character variable: large or med/small
2	school	school identifier
3	hrwork	number of hours required to work at school per week
4	size	class size
5	preprmin	minutes spent per week in school on preparation
6	assist	minutes per week that a teacher's aide works with the teacher in the classroom

teachmi.csv Cluster sizes for data in **teachers.csv**.

Column	Name	Value
1	dist	School district size: large or med/small
2	school	school identifier
3	popteach	number of teachers in that school
4	ssteach	number of surveys returned from that school

teachnr.csv Data from a follow-up study of nonrespondents from Gnap (1995).

Column	Name	Value
1	hrwork	number of hours required to work at school per week
2	size	class size
3	preprmin	minutes spent per week in school on preparation
4	assist	minutes per week that a teacher's aide works with the teacher in the classroom

uneqvar.csv Artificial data used in exercises of Chapter 11.

Column	Name	Value
1	x	x
2	y	y

vietnam.csv Vietnam-service data from Stockford and Page (1984).

Column	Name	Value
1	apc	APC stratum. Character variable with options "Yes," "No," "NotAvail"
2	p2sample	Indicator variable for phase II sample, = 1 if in phase II sample, 0 otherwise
3	vietnam	= 1 if service in Vietnam, = 0 if service not in Vietnam, = -9 if not in phase II sample

vietnam.csv (continued)		
Column	Name	Value
4	phase1wt	weight for phase I sample
5	phase2wt	conditional weight for phase II sample, calculated as (phase I sample size in stratum) / (phase II sample size in stratum). phase2wt = −9 for observations not in phase 2 sample.
6	finalwt	final weight for phase II sample, calculated as phase1wt*phase2wt (= −9 for observations not in phase II sample)
7	p1apcsize	number of observations in the observation's APC stratum that are in the phase I sample (n_h)
8	p2apcsize	number of observations in the observation's APC stratum that are in the phase II sample (m_h)

vius.csv Selected variables from the 2002 U.S. Vehicle Inventory and Use Survey (VIUS). Source: U.S. Census Bureau (2006). The data were downloaded from www.census.gov/svsd/www/vius in May, 2006. The website from which the data were downloaded no longer exists, and online information about VIUS may now be found at <https://www.bts.gov/vius>, which provides a link to the archived 2002 data. The missing value of *state* for records with *adm_state* = 42 was recoded to “PA,” the state that has code 42. This data set has 98,682 records, which may be too large for some software packages to handle; the file **viusca.csv** is a smaller data set, with the same columns described below, containing only vehicles from California. The variable descriptions below are taken from the VIUS Data Dictionary.

Missing values are coded as −99. For some variables, the value is missing because the question is not applicable or the vehicle is not in use; see the individual variable descriptions.

Note that a new VIUS is planned for 2022, with data to be released in 2023; see <https://www.bts.gov/vius>.

Column	Name	Value
1	stratum	stratum number (contains all 255 strata)
2	adm_state	state number
3	state	state name
4	trucktype	type of truck, used in stratification 1. pickups 2. minivans, other light vans, and sport utility vehicles 3. light single-unit trucks with gross vehicle weight less than 26,000 pounds 4. heavy single-unit trucks with gross vehicle weight greater than or equal to 26,000 pounds 5. truck-tractors
5	tabtrucks	column of sampling weights
6	bodytype	body type of vehicle 01. Pickup 02. Minivan 03. Light van other than minivan 04. Sport utility

vius.csv (continued)		
Column	Name	Value
		05. Armored
		06. Beverage
		07. Concrete mixer
		08. Concrete pumper
		09. Crane
		10. Curtainside
		11. Dump
		12. Flatbed, stake, platform, etc.
		13. Low boy
		14. Pole, logging, pulpwood, or pipe
		15. Service, utility
		16. Service, other
		17. Street sweeper
		18. Tank, dry bulk
		19. Tank, liquids or gases
		20. Tow/Wrecker
		21. Trash, garbage, or recycling
		22. Vacuum
		23. Van, basic enclosed
		24. Van, insulated non-refrigerated
		25. Van, insulated refrigerated
		26. Van, open top
		27. Van, step, walk-in, or multistop
		28. Van, other
		99. Other not elsewhere classified
7	adm_modelyear	model year
		01. 2003, 2002
		02. 2001
		03. 2000
		04. 1999
		05. 1998
		06. 1997
		07. 1996
		08. 1995
		09. 1994
		10. 1993
		11. 1992
		12. 1991
		13. 1990
		14. 1989
		15. 1988
		16. 1987
		17. Pre-1987
8	vius_gvw	Gross vehicle weight based on average reported weight
		01. Less than 6,001 lbs.
		02. 6,001 to 8,500 lbs.
		03. 8,501 to 10,000 lbs.
		04. 10,001 to 14,000 lbs.

vius.csv (continued)		
Column	Name	Value
		05. 14,001 to 16,000 lbs.
		06. 16,001 to 19,500 lbs.
		07. 19,501 to 26,000 lbs.
		08. 26,001 to 33,000 lbs.
		09. 33,001 to 40,000 lbs.
		10. 40,001 to 50,000 lbs.
		11. 50,001 to 60,000 lbs.
		12. 60,001 to 80,000 lbs.
		13. 80,001 to 100,000 lbs.
		14. 100,001 to 130,000 lbs.
		15. 130,001 lbs. or more
9	miles_annl	Number of Miles Driven During 2002
10	miles_life	Number of Miles Driven Since Manufactured
11	mpg	Miles Per Gallon averaged during 2002. Range from 0.3 to 35. -99 denotes not reported or not applicable.
12	opclass	Operator Classification With Highest Percent
		1. Private
		2. Motor carrier
		3. Owner operator
		4. Rental
		5. Personal transportation
		6. Not applicable (Vehicle not in use)
13	opclass_mtr	Percent of Miles Driven as a Motor Carrier. -99 denotes vehicle not in use
14	opclass_own	Percent of Miles Driven as an Owner Operator. -99 denotes vehicle not in use
15	opclass_psl	Percent of Miles Driven for Personal Transportation. -99 denotes vehicle not in use
16	opclass_pvt	Percent of Miles Driven as Private (Carry Own Goods or Internal Company Business Only). -99 denotes vehicle not in use
17	opclass_rnt	Percent of Miles Driven as Rental. -99 denotes vehicle not in use
18	transmssn	Type of Transmission
		1. Automatic
		2. Manual
		3. Semi-Automated Manual
		4. Automated Manual
19	trip_primary	Primary Range of Operation
		1. Off-the-road
		2. Less than 50 miles
		3. 51 to 100 miles
		4. 101 to 200 miles
		5. 201 to 500 miles
		6. 501 miles or more
		7. Not reported
		8. Not applicable (Vehicle not in use)
20	trip0_50	Percent of Annual Miles Accounted for with Trips

vius.csv (continued)		
Column	Name	Value
21	trip051_100	50 Miles or Less from the Home Base Percent of Annual Miles Accounted for with Trips 51 to 100 Miles from the Home Base
22	trip101_200	Percent of Annual Miles Accounted for with Trips 101 to 200 Miles from the Home Base
23	trip201_500	Percent of Annual Miles Accounted for with Trips 201 to 500 Miles from the Home Base
24	trip500more	Percent of Annual Miles Accounted for with Trips 501 or More Miles from Home Base
25	adm_make	Make of vehicle 01. Chevrolet 02. Chrysler 03. Dodge 04. Ford 05. Freightliner 06. GMC 07. Honda 08. International 09. Isuzu 10. Jeep 11. Kenworth 12. Mack 13. Mazda 14. Mitsubishi 15. Nissan 16. Peterbilt 17. Plymouth 18. Toyota 19. Volvo 20. White 21. Western Star 22. White GMC 23. Other (domestic) 24. Other (foreign)
26	business	Business in which vehicle was most often used during 2002 01. For-hire transportation or warehousing 02. Vehicle leasing or rental 03. Agriculture, forestry, fishing, or hunting 04. Mining 05. Utilities 06. Construction 07. Manufacturing 08. Wholesale trade 09. Retail trade 10. Information services 11. Waste management, landscaping, or administra- tive/support services

vius.csv (continued)		
Column	Name	Value
		12. Arts, entertainment, or recreation services
		13. Accommodation or food services
		14. Other services
		–99. Not reported or not applicable

winter.csv Selected variables from the Arizona State University Winter Closure Survey, taken in January 1995 (provided courtesy of the ASU Office of University Evaluation). This survey was taken to investigate the attitudes and opinions of university employees towards the closing of the university (for budgetary reasons) between December 25 and January 1. For the yes/no questions, the responses are coded as 1 = No, 2 = Yes. The variables *treatsta* and *treatme* are coded as 1=strongly agree, 2=agree, 3=undecided, 4=disagree, 5=strongly disagree. The variables *process* and *satbreak* are coded as 1=very satisfied, 2=satisfied, 3=undecided, 4=dissatisfied, 5=very dissatisfied. Variables *ownsupp* through *offclose* are coded 1 if the person checked that the statement applied to him/her, and 2 if the statement was not checked.

Missing values are coded as 9.

Column	Name	Value
1	class	stratum number 1 = faculty ; 2 = classified staff; 3 = administrative staff; 4 = academic professional
2	yearasu	number of years worked at ASU 1 = 1–2 years; 2 = 3–4 years; 3 = 5–9 years; 4 = 10–14 years; 5 = 15 or more years
3	vacation	In the past, have you <i>usually</i> taken vacation days the entire period between December 25 and January 1?
4	work	Did you work on campus during Winter Break Closure?
5	havediff	Did the Winter Break Closure cause you any difficulty/concerns?
6	negaeffe	Did the Winter Break Closure <i>negatively</i> affect your work productivity?
7	ownsupp	I was unable to obtain staff support in my department/office
8	othersup	I was unable to obtain staff support in other departments/offices
9	utility	I was unable to access computers, copy machine, etc. in my department/office
10	environ	I was unable to endure environmental conditions, e.g., not properly climatized
11	uniserve	I was unable to access university services necessary to my work
12	workelse	I was unable to work on my assignments because I work in another department/office
13	offclose	I was unable to work on my assignments because my office was closed
14	treatsta	Compared to other departments/offices, I feel staff in my department/office were treated fairly

winter.csv (continued)		
Column	Name	Value
15	treatme	Compared to other people working in my department/office, I feel I was treated fairly
16	process	How satisfied are you with the process used to inform staff about Winter Break Closure?
17	satbreak	How satisfied are you with the fact that ASU had a Winter Break Closure this year?
18	breakaga	Would you want to have Winter Break Closure again?

wtshare.csv Hypothetical sample of size 100, with indirect sampling. The data set has multiple records for adults with more than one child; if adult 254 has 3 children, adult 254 is listed 3 times in the data set. Note that to obtain L_k , you need to take $numadult + 1$.

Column	Name	Value
1	id	Identification number of adult in sample
2	child	= 1 if record is for a child, 0 if adult has no children
3	preschool	= 1 if child is in preschool, 0 otherwise
4	numadult	number of other adults in population who link to that child

Bibliography

- Allison, P. D. (2012). *Logistic Regression using SAS[®]: Theory and Application*. Cary, NC: SAS Institute, Inc.
- Arnold, T. W. (1991). Intraclutch variation in egg size of American coots. *The Condor* 93, 19–27.
- Asian Development Bank (2020). *Introduction to Small Area Estimation Techniques: A Practical Guide for National Statistics Offices*. Manila: Asian Development Bank.
- Azur, M. J., E. A. Stuart, C. Frangakis, and P. J. Leaf (2011). Multiple imputation by chained equations: What is it and how does it work? *International Journal of Methods in Psychiatric Research* 20(1), 40–49.
- Baillargeon, S. and L.-P. Rivest (2007). Rcapture: Loglinear models for capture-recapture in R. *Journal of Statistical Software* 19(5), 1–31.
- Baillargeon, S. and L.-P. Rivest (2011). The construction of stratified designs in R with the package stratification. *Survey Methodology* 37(1), 53–65.
- Barcaroli, G. (2014). SamplingStrata: An R package for the optimization of stratified sampling. *Journal of Statistical Software* 61(4), 1–24.
- Barcaroli, G., M. Ballin, H. Odendaal, D. Pagliuca, E. Willighagen, and D. Zardetto (2020). *SamplingStrata: Optimal Stratification of Sampling Frames for Multipurpose Sampling Surveys*. R package version 1.5-1, <https://CRAN.R-project.org/package=SamplingStrata> (accessed March 11, 2021).
- Bart, J. and S. Earnst (2002). Double-sampling to estimate density and population trends in birds. *The Auk* 119, 36–45.
- Bates, D., M. Mächler, B. Bolker, and S. Walker (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software* 67(1), 1–48.
- Bates, D., M. Mächler, B. Bolker, S. Walker, R. H. B. Christensen, H. Singmann, B. Dai, F. Scheipl, G. Grothendieck, P. Green, J. Fox, A. Bauer, and P. N. Krivitsky (2020). *lme4: Linear Mixed-Effects Models using ‘Eigen’ and S4*. R package version 1.1-26, <https://CRAN.R-project.org/package=lme4> (accessed March 20, 2021).
- Beck, A. J., S. A. Kline, and L. A. Greenfeld (1988). Survey of Youth in Custody. Technical Report NCJ-113365, Bureau of Justice Statistics, Washington, DC.
- Bretz, F., T. Hothorn, and P. Westfall (2016). *Multiple Comparisons using R*. Boca Raton, FL: CRC Press.
- Brewer, K. R. W. (1963). Ratio estimation and finite populations: Some results deducible from the assumption of an underlying stochastic process. *The Australian Journal of Statistics* 5(3), 93–105.

- Brewer, K. R. W. (1975). A simple procedure for sampling π pswor. *The Australian Journal of Statistics* 17, 166–172.
- Brick, J. M., D. Morganstein, and R. Valliant (2000). *Analysis of Complex Sample Data Using Replication*. Rockville, MD: Westat.
- Bueno, E. (2020). *optimStrat: Choosing the Sample Strategy*. R package version 2.3. <https://CRAN.R-project.org/package=optimStrat> (accessed March 12, 2021).
- Canty, A. J. and A. C. Davison (1999). Resampling-based variance estimation for labour force surveys. *The Statistician* 48, 379–391.
- Centers for Disease Control and Prevention (2017). NHANES Questionnaires, Datasets, and Related Documentation. <https://wwwn.cdc.gov/nchs/nhanes/> (accessed August 15, 2020).
- Chao, A., K. H. Ma, T. C. Hsieh, and C.-H. Chiu (2016). *SpadeR: Species-Richness Prediction and Diversity Estimation with R*. R package version 0.1.1, <https://CRAN.R-project.org/package=SpadeR> (accessed March 15, 2021).
- Chapman, D. G. (1951). Some properties of the hypergeometric distribution with applications to zoological sample censuses. *University of California Publications in Statistics* 1, 131–160.
- Chauvet, G. and Y. Tillé (2006). A fast algorithm for balanced sampling. *Computational Statistics* 21(1), 53–62.
- Cho, I., J.-K. Kim, J. Im, and Y. Yang (2020). *FHDI: Fractional Hot Deck and Fully Efficient Fractional Imputation*. R package version 1.4.1, <https://CRAN.R-project.org/package=FHDI> (accessed April 15, 2021).
- Cormack, R. M. (1992). Interval estimation for mark-recapture studies of closed populations. *Biometrics* 48, 567–576.
- Dippo, C. S., R. E. Fay, and D. H. Morganstein (1984). Computing variances from complex samples with replicate weights. In *Proceedings of the Survey Research Methods Section*, pp. 489–494. Alexandria, VA: American Statistical Association.
- Domingo-Salvany, A., R. L. Hartnoll, A. Maquire, J. M. Suelves, and J. M. Anto (1995). Use of capture-recapture to estimate the prevalence of opiate addiction in Barcelona, Spain, 1989. *American Journal of Epidemiology* 141, 567–574.
- Fienberg, S. E. and A. Rinaldo (2007). Three centuries of categorical data analysis: Log-linear models and maximum likelihood estimation. *Journal of Statistical Planning and Inference* 137(11), 3430–3445.
- Forman, S. L. (2004). Baseball-reference.com—Major league statistics and information. www.baseball-reference.com (accessed November 2004).
- Gambino, J. G. (2021). *pps: PPS Sampling*. R package version 1.0. <https://CRAN.R-project.org/package=pps> (accessed March 12, 2021).
- Gini, C. and L. Galvani (1929). Di una applicazione del metodo rappresentativo all’ultimo censimento italiano della popolazione. *Annali di Statistica* 6(4), 1–105.
- Gnap, R. (1995). *Teacher Load in Arizona Elementary School Districts in Maricopa County*. Ph.D. dissertation. Tempe, AZ: Arizona State University.

- Goga, C. (2018). Brief overview of survey sampling techniques with R. *Romanian Statistical Review* 2018(1), 83–94.
- Grafström, A. and J. Lisic (2019). *BalancedSampling: Balanced and Spatially Balanced Sampling*. R package version 1.5.5, <https://CRAN.R-project.org/package=BalancedSampling> (accessed March 12, 2021).
- Hand, D. J., F. Daly, A. D. Lunn, K. J. McConway, and E. Ostrowski (1994). *A Handbook of Small Data Sets*. London: Chapman and Hall.
- Hanurav, T. V. (1967). Optimum utilization of auxiliary information: π ps sampling of two units from a stratum. *Journal of the Royal Statistical Society, Series B* 29, 374–391.
- Harmening, S., A.-K. Kreutzmann, S. Pannier, N. Rojas-Perilla, N. Salvati, T. Schmid, M. Templ, N. Tzavidis, and N. Würz (2021). *emdi: Estimating and Mapping Disaggregated Indicators*. R package version 2.0.2, <https://CRAN.R-project.org/package=emdi> (accessed April 30, 2021).
- Harrell, F. E. (2021). *Hmisc: Harrell miscellaneous*. R package version 4.5-0, <https://CRAN.R-project.org/package=Hmisc> (accessed April 7, 2021).
- Hartley, H. O. and J. N. K. Rao (1962). Sampling with unequal probabilities and without replacement. *The Annals of Mathematical Statistics* 33, 350–374.
- Hayat, M. and T. Knapp (2017). Randomness and inference in medical and public health research. *Journal of the Georgia Public Health Association* 7(1), 7–11.
- Haziza, D. (2009). Imputation and inference in the presence of missing data. In D. Pfeffermann and C. R. Rao (Eds.), *Sample Surveys: Design, Methods, and Applications. Handbook of Statistics, Volume 29A*, pp. 215–246. Amsterdam: North-Holland.
- Heck, P. R., D. J. Simons, and C. F. Chabris (2018). 65% of Americans believe they are above average in intelligence: Results of two nationally representative surveys. *PloS One* 13(7), 1–11.
- Hidiroglou, M. A., J.-F. Beaumont, and W. Yung (2019). Development of a small area estimation system at Statistics Canada. *Survey Methodology* 45(1), 101–126.
- Horton, N. J. and K. Kleinman (2015). *Using R and RStudio for Data Management, Statistical Analysis, and Graphics, 2nd ed.* Boca Raton, FL: CRC Press.
- Horton, N. J., R. Pruim, and D. T. Kaplan (2018). *A Student's Guide to R*. Amherst, MA: Project MOSAIC.
- Hyndman, R. J. and Y. Fan (1996). Sample quantiles in statistical packages. *The American Statistician* 50(4), 361–365.
- Im, J., I. H. Cho, and J.-K. Kim (2018). FHDI: An R package for fractional hot deck imputation. *R Journal* 10(1), 140–154.
- Ismay, C. and P. C. Kennedy (2019). *Getting Used to R, RStudio, and R Markdown*. <https://rbasics.netlify.app/> (accessed March 2, 2021).
- Jacoby, J. and A. H. Handlin (1991). Non-probability sampling designs for litigation surveys. *Trademark Reporter* 81, 169–179.

- Judkins, D. (1990). Fay's method for variance estimation. *Journal of Official Statistics* 6, 223–240.
- Kabacoff, R. I. (2021). *R in Action, 3rd ed.* Shelter Island, NY: Manning Publications.
- Kim, J. K. (2011). Parametric fractional imputation for missing data analysis. *Biometrika* 98(1), 119–132.
- Kim, J. K. and W. Fuller (2004). Fractional hot deck imputation. *Biometrika* 91(3), 559–578.
- Koch, G. G., D. H. Freeman, and J. L. Freeman (1975). Strategies in the multivariate analysis of data from complex surveys. *International Statistical Review* 43, 59–78.
- Koenker, R. (2005). *Quantile Regression*. Cambridge: Cambridge University Press.
- Koenker, R., S. Portnoy, P. T. Ng, B. Melly, A. Zeileis, P. Grosjean, C. Moler, Y. Saad, V. Chernozhukov, I. Fernandez-Val, and B. D. Ripley (2021). *quantreg: Quantile Regression*. R package version 2.23-18, <https://CRAN.R-project.org/package=quantreg> (accessed April 7, 2021).
- Korn, E. L. and B. I. Graubard (1998). Confidence intervals for proportions with small expected number of positive counts estimated from survey data. *Survey Methodology* 24, 193–201.
- Kott, P. S. (2012). Why one should incorporate the design weights when adjusting for unit nonresponse using response homogeneity groups. *Survey Methodology* 38(1), 95–99.
- Kowarik, A. and M. Templ (2016). Imputation with the R package VIM. *Journal of Statistical Software* 74(7), 1–16.
- Kreutzmann, A.-K., S. Pannier, N. Rojas-Perilla, T. Schmid, M. Templ, and N. Tzavidis (2019). The R package emdi for estimating and mapping regionally disaggregated indicators. *Journal of Statistical Software* 91, 1–33.
- Kruuk, H., A. Moorhouse, J. W. H. Conroy, L. Durbin, and S. Frears (1989). An estimate of numbers and habitat preferences of otters *Lutra lutra* in Shetland, UK. *Biological Conservation* 49, 241–254.
- Little, R. J. and S. Vartivarian (2003). On weighting the rates in non-response weights. *Statistics in Medicine* 22(9), 1589–1599.
- Lohr, S. L. (2022). *SAS® Software Companion for Sampling: Design and Analysis*, 3rd ed. Boca Raton, FL: CRC Press.
- Lopez-Vizcaino, E., M. Lombardia, and D. Morales (2019). *mme: Multinomial Mixed Effects Models*. R package version 0.1-6. <https://CRAN.R-project.org/package=mme> (accessed January 16, 2021).
- Lu, Y. and S. L. Lohr (2021). *SDAResources: Datasets and Functions for “Sampling: Design and Analysis.”* R package version 0.1.0. <https://CRAN.R-project.org/package=SDAResources> (accessed May 17, 2021).
- Lumley, T. (2004). Analysis of complex survey samples. *Journal of Statistical Software* 9(1), 1–19.
- Lumley, T. (2010). *Complex Surveys: A Guide to Analysis using R*. Hoboken, NJ: Wiley.

- Lumley, T. (2020). *survey: Analysis of Complex Survey Samples*. R package version 4.0. <https://CRAN.R-project.org/package=survey> (accessed September 20, 2020).
- Lundquist, P. and C.-E. Särndal (2013). Aspects of responsive design with applications to the Swedish Living Conditions Survey. *Journal of Official Statistics* 29(4), 557–582.
- Luraschi, J. (2021). Importing data with RStudio. <https://support.rstudio.com/hc/en-us/articles/218611977-Importing-Data-with-RStudio> (accessed March 12, 2021).
- Lydersen, C. and M. Ryg (1991). Evaluating breeding habitat and populations of ringed seals *Phoca hispida* in Svalbard fjords. *Polar Record* 27, 223–228.
- Macdonell, W. R. (1901). On criminal anthropometry and the identification of criminals. *Biometrika* 1, 177–227.
- Molina, I. and Y. Marhuenda (2015). sae: An R package for small area estimation. *The R Journal* 7(1), 1–98.
- Molina, I. and Y. Marhuenda (2020). sae: *Small Area Estimation*. R package version 1.3. <https://CRAN.R-project.org/package=sae> (accessed December 20, 2020).
- National Center for Health Statistics (1987). *Vital Statistics of the United States, Volume 3: Marriage and Divorce*. Washington, DC: U.S. Government Printing Office.
- Nolan, D. and T. Speed (2000). *Stat Labs: Mathematical Statistics Through Applications*. New York: Springer.
- Oetiker, T., H. Partl, I. Hyna, and E. Schlegl (2021). *The Not So Short Introduction to L^AT_EX 2_ε, Version 6.4*. Olten, Switzerland: Tobias Oetiker. <https://tobi.oetiker.ch/lshort/lshort.pdf> (accessed March 18, 2021).
- Peart, D. (1994). *Impacts of Feral Pig Activity on Vegetation Patterns Associated with Quercus agrifolia on Santa Cruz Island, California*. Ph.D. dissertation. Tempe, AZ: Arizona State University.
- Pinheiro, J., D. Bates, S. DebRoy, D. Sarkar, and R Core Team (2021). nlme: *Linear and Nonlinear Mixed Effects Models*. R package version 3.1-152, <https://CRAN.R-project.org/package=nlme> (accessed March 20, 2021).
- Pratesi, M. (Ed.) (2016). *Analysis of Poverty Data by Small Area Estimation*. Hoboken, NJ: Wiley.
- Preston, J. (2009). Rescaled bootstrap for stratified multistage sampling. *Survey Methodology* 35(2), 227–234.
- R Core Team (2021). *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. Version 4.0.4, <https://www.R-project.org>.
- Rao, J. N. K. and I. Molina (2015). *Small Area Estimation, 2nd ed.* Hoboken, NJ: Wiley.
- Rao, J. N. K. and A. J. Scott (1981). The analysis of categorical data from complex sample surveys: Chi-squared tests for goodness of fit and independence in two-way tables. *Journal of the American Statistical Association* 76, 221–230.
- Rao, J. N. K. and A. J. Scott (1984). On chi-squared tests for multiway contingency tables with cell proportions estimated from survey data. *The Annals of Statistics* 12, 46–60.

- Rao, J. N. K., C. F. J. Wu, and K. Yue (1992). Some recent work on resampling methods for complex surveys. *Survey Methodology* 18, 209–217.
- Reiter, J. P., T. E. Raghunathan, and S. K. Kinney (2006). The importance of modeling the sampling design in multiple imputation for missing data. *Survey Methodology* 32(2), 143–149.
- Rivest, L.-P. and S. Baillargeon (2017). *stratification: Univariate Stratification of Survey Populations*. R package version 2.2-6, <https://CRAN.R-project.org/package=stratification> (accessed March 10, 2021).
- Rivest, L.-P. and S. Baillargeon (2019). *Rcapture: Loglinear Models for Capture-Recapture Experiments*. R package version 1.4-3, <https://CRAN.R-project.org/package=Rcapture> (accessed March 15, 2021).
- Roberts, R. J., Q. D. Sandifer, M. R. Evans, M. Z. Nolan-Ferrell, and P. M. Davis (1995). Reasons for non-uptake of measles, mumps, and rubella catch up immunisation in a measles epidemic and side effects of the vaccine. *British Medical Journal* 310, 1629–1632.
- Ruggles, S., M. Sobek, T. Alexander, C. A. Fitch, R. Goeken, P. K. Hall, M. King, and C. Ronnander (2004). Integrated Public Use Microdata Series: Version 3.0 [machine-readable database]. www.ipums.org/usa (accessed September 17, 2008).
- Sampford, M. R. (1967). On sampling without replacement with unequal probabilities of selection. *Biometrika* 54, 499–513.
- SAS Institute Inc. (2021). *SAS/STAT® User's Guide*. Cary, NC: SAS Institute Inc. <https://documentation.sas.com/> (accessed April 27, 2021).
- Stockford, D. D. and W. F. Page (1984). Double sampling and the misclassification of Vietnam service. In *Proceedings of the Social Statistics Section*, pp. 261–264. Alexandria, VA: American Statistical Association.
- Talbot, N. L. C. (2012). *LaTeX for Complete Novices*. Saxlingham Nethergate, UK: Dickimaw Books. <https://www.dickimaw-books.com/latex/novices/> (accessed March 10, 2021).
- Templ, M., A. Kowarik, A. Alfons, G. de Cillia, B. Prantner, and W. Rannetbauer (2021). *VIM: Visualization and Imputation of Missing Values*. R package version 6.1.0, <https://CRAN.R-project.org/package=VIM> (accessed April 21, 2021).
- Thomas, D. R. and J. N. K. Rao (1987). Small-sample comparisons of level and power for simple goodness-of-fit statistics under cluster sampling. *Journal of the American Statistical Association* 82, 630–636.
- Tillé, Y. (2006). *Sampling Algorithms*. New York: Springer.
- Tillé, Y. and A. Matei (2010). Teaching survey sampling with the ‘sampling’ R package. In *Proceedings of the 8th International Conference on Teaching Statistics (ICOTS)*, pp. 1–6. Auckland, NZ: International Association of Statistical Education.
- Tillé, Y. and A. Matei (2021). *sampling: Survey Sampling*. R package version 2.9, <https://CRAN.R-project.org/package=sampling> (accessed March 12, 2021).
- Tillé, Y. and M. Wilhelm (2017). Probability sampling designs: Principles for choice of design and balancing. *Statistical Science* 32(2), 176–189.

- Tzavidis, N., L.-C. Zhang, A. Luna, T. Schmid, and N. Rojas-Perilla (2018). From start to finish: A framework for the production of small area official statistics. *Journal of the Royal Statistical Society: Series A* 181(4), 927–979.
- U.S. Bureau of the Census (1921). *Fourteenth Census of the United States Taken in the Year 1920*. Washington, DC: U.S. Government Printing Office. <https://www.census.gov/library/publications/1921/dec/vol-01-population.html> (accessed August 4, 2020).
- U.S. Bureau of the Census (1995). *1992 Census of Agriculture, Volume 1: Geographic Area Series*. Washington, DC: U.S. Bureau of the Census.
- U.S. Census Bureau (1994). *County and City Data Book: 1994*. Washington, DC: U.S. Census Bureau.
- U.S. Census Bureau (2006). *Vehicle Inventory and Use Survey—Methods*. Washington, DC: U.S. Census Bureau.
- U.S. Census Bureau (2012). *United States Summary, 2010*. Washington, DC: U.S. Census Bureau. <https://www.census.gov/prod/cen2010/cph-2-1.pdf> (accessed October 3, 2020).
- U.S. Census Bureau (2019). State population totals: 2010-2019. Table 1. Annual estimates of the resident population for the United States, regions, states, and Puerto Rico: April 1, 2010 to July 1, 2019 (NST-EST2019-01). <https://www.census.gov/data/datasets/time-series/demo/popest/2010s-state-total.html> (accessed August 3, 2020).
- U.S. Department of Education (2020). College scorecard data. <https://collegescorecard.ed.gov/data/> (accessed August 25, 2020).
- U.S. Department of Justice (1989). *Survey of Youth in Custody, 1987, United States computer file, Conducted by Department of Commerce, Bureau of the Census, 2nd ICPSR ed.* Ann Arbor, MI: Inter-University Consortium for Political and Social Research.
- Valliant, R., J. A. Dever, and F. Kreuter (2018). *Practical Tools for Designing and Weighting Survey Samples*. New York: Springer.
- Valliant, R., J. A. Dever, and F. Kreuter (2020). *PracTools: Tools for Designing and Weighting Survey Samples*. R package version 1.2.2. <https://CRAN.R-project.org/package=PracTools> (accessed March 12, 2021).
- Valliant, R. and K. F. Rust (2010). Degrees of freedom approximations and rules-of-thumb. *Journal of Official Statistics* 26(4), 585–602.
- van Buuren, S. (2018). *Flexible Imputation of Missing Data*. Boca Raton, FL: CRC Press.
- van Buuren, S., K. Groothuis-Oudshoorn, G. Vink, R. Schouten, A. Robitzsch, P. Rocken-schaub, L. Doove, S. Jolani, M. Moreno-Betancur, I. White, P. Gaffert, F. Meinfelder, B. Gray, and V. Arel-Bundock (2021). *mice: Multivariate Imputation by Chained Equations*. R package version 3.13.0, <https://CRAN.R-project.org/package=mice> (accessed April 15, 2021).
- Vijayan, K. (1968). An exact π ps sampling scheme: Generalization of a method of Hanurav. *Journal of the Royal Statistical Society, Series B* 30, 556–566.

- Virginia Polytechnic and State University/Responsive Management (2006). *An Assessment of Public and Hunter Opinions and the Costs and Benefits to North Carolina of Sunday Hunting*. Blacksburg, VA: Virginia Polytechnic and State University.
- Wand, M., C. Moler, and B. Ripley (2020). *KernSmooth: Functions for Kernel Smoothing Supporting Wand & Jones (1995)*. R package version 2.23-18, <https://CRAN.R-project.org/package=KernSmooth> (accessed April 7, 2021).
- Wand, M. P. and M. C. Jones (1995). *Kernel Smoothing*. London: Chapman & Hall.
- Wang, J. (2021). The pseudo maximum likelihood estimator for quantiles of survey variables. *Journal of Survey Statistics and Methodology* 9(1), 185–201.
- Wickham, H. (2015). *R Packages: Organize, Test, Document, and Share Your Code*. Sebastopol, CA: O'Reilly Media.
- Wickham, H. (2019). *Advanced R, 2nd ed.* Boca Raton, FL: CRC Press.
- Wickham, H., W. Chang, L. Henry, T. L. Pedersen, K. Takahashi, C. Wilke, K. Woo, H. Yutani, and D. Dunnington (2020). *ggplot2: Create Elegant Data Visualisations Using the Grammar of Graphics*. R package version 3.3.3, <https://CRAN.R-project.org/package=ggplot2> (accessed March 30, 2021).
- Wikibooks contributors (2021). *LaTeX*. Wikibooks, The Free Textbook Project. <https://en.wikibooks.org/wiki/LaTeX> (accessed March 18, 2021).
- Wilk, S. J., W. W. Morse, D. E. Ralph, and T. R. Azarovitz (1977). *Fishes and Associated Environmental Data Collected in New York Bight, June 1974–June 1975*. NOAA Tech. Rep. No. NMFS SSRF-716. Washington, DC: U.S. Government Printing Office.
- Woodruff, R. S. (1952). Confidence intervals for medians and other position measures. *Journal of the American Statistical Association* 47, 636–646.
- Xie, Y. (2015). *Dynamic Documents with R and knitr, 2nd ed.* Boca Raton, FL: CRC Press.
- Yadav, M. L. and B. Roychoudhury (2018). Handling missing values: A study of popular imputation packages in R. *Knowledge-Based Systems* 160, 104–118.
- Zhang, C., C. Antoun, H. Y. Yan, and F. G. Conrad (2020). Professional respondents in opt-in online panels: What do we really know? *Social Science Computer Review* 38(6), 703–719.
- Zhang, G., F. Christensen, and W. Zheng (2015). Nonparametric regression estimators in complex surveys. *Journal of Statistical Computation and Simulation* 85(5), 1026–1034.

Index

- Allocation, stratified sampling, 27–29
- Analysis of variance (ANOVA), 37, 52
- Balanced repeated replication (BRR), 113, 116–124, 144
- Balanced sampling, 151
- Bootstrap, 113, 116–124
 - capture-recapture estimation, 158–159
- Boxplot
 - complex survey, 92–94, 100
 - domains, 148
 - stratified random sample, 32
- Capture-recapture estimation, 157–159
- Categorical variable, 23
- Chi-square test, 129–132
- Clopper-Pearson confidence interval, 24, 37
- Cluster sample
 - equal probabilities, 57–68
 - estimating means, 57–63
 - estimating proportions, 61
 - finite population correction, 62–63
 - selection, 69–77
 - unequal probabilities, 69–83
- Combined ratio estimator, 49
- Contingency table, 129–131
- Conventions used in book, 1
- Cumulative distribution function, empirical, 103–105
- Degrees of freedom (df)
 - bootstrap SRS, 123
 - bootstrap stratified random sample, 124
 - chi-square test, 129
 - domain estimation, 47, 119
 - jackknife cluster sampling, 122
 - jackknife SRS, 121
 - one-stage cluster sample, 57–59, 78
 - random group, 116
 - regression, 45, 141, 143
 - replicated samples, 115
 - simple random sample, 21
 - stratified random sample, 35
 - two-stage cluster sample, 60, 79
- Density estimate, 93–94
- Design effect (deff), 92, 132–133
- Domain, 46
 - comparison, 145–148
 - estimation, 46–48, 90, 148
- Empirical cumulative distribution function, 88, 103–105
- Empirical probability mass function, 103–105
- Examples in SDA
 - Example 02.05, 15–17
 - Example 02.06, 17, 19–23
 - Example 02.07, 19–23
 - Example 02.11, 19–23
 - Example 03.02, 30–37
 - Example 03.06, 32–37
 - Example 04.02, 41–42
 - Example 04.03, 41–42
 - Example 04.05, 42–44
 - Example 04.07, 44–45
 - Example 04.08, 46–47
 - Example 04.09, 48, 124–125
 - Example 04.11, 50–52
 - Example 04.12, 52–54
 - Example 05.02, 57–58
 - Example 05.06, 58–59
 - Example 05.07, 61–63
 - Example 05.08, 59–60
 - Example 05.12, 63
 - Example 05.14, 64
 - Example 06.02, 69–70
 - Example 06.04, 77–78
 - Example 06.06, 78–79
 - Example 06.08, 81
 - Example 06.09, 81–82
 - Example 06.10, 82–83
 - Example 06.11, 72–77, 79–80
 - Example 07.05, 88–89, 103
 - Example 07.06, 89
 - Example 07.09, 90–92

- Example 07.10, 93–95
- Example 07.11, 93–95
- Example 07.12, 93–95
- Example 08.09, 111
- Example 09.03, 113–115
- Example 09.04, 115–116
- Example 09.05, 117–119, 126
- Example 09.06, 119–120
- Example 09.07, 120–122
- Example 09.08, 122
- Example 09.09, 122–123
- Example 09.10, 123–124
- Example 09.12, 89
- Example 10.01, 129–130
- Example 10.05, 130–131
- Example 10.06, 131–133
- Example 10.08, 133–135
- Example 10.09, 135
- Example 11.02, 139–140
- Example 11.04, 140
- Example 11.06, 141–142
- Example 11.07, 142–145
- Example 11.08, 146–147
- Example 11.09, 147–148
- Example 11.12, 149–150
- Example 12.01, 155–157
- Example 12.04, 155–157
- Example 13.01, 157–159
- Example 13.03, 159–161
- Exercises in SDA
 - Exercise 02.27, 24
 - Exercise 02.32, 24
 - Exercise 02.34, 24
 - Exercise 05.40, 65–67
 - Exercise 06.36, 82
 - Exercise 06.45, 70
 - Exercise 07.19, 88
 - Exercise 11.37, 151
- Finite population correction (fpc)
 - chi-square test, 131
 - one-stage cluster sampling, 58
 - regression coefficients, 141
 - simple random sampling, 19, 21
 - stratified random sampling, 34–35
 - two-stage cluster sampling, 60–63
 - unequal-probability sampling, 77
- Functions in R
 - abline, 42
 - anova, 135–137
 - apply, 81, 158–159
 - as.data.frame, 4
 - as.svrepdesign, 116–125, 141
 - axis, 150
 - binom.test, 24
 - box, 150
 - boxplot, 9, 33, 94
 - calibrate, 110
 - captureci, 158, 160–161
 - chisq.test, 129, 133–134
 - cluster, 13, 70–71
 - coef, 22, 42, 114–115
 - complete.cases, 107–108
 - confint, 21–23, 44–45, 57–59, 117–125
 - cor, 41–44
 - cut, 100
 - data, 7
 - data.frame, 78, 79, 117–119, 124–125, 133–134
 - degf, 33–35, 120–122, 140–141
 - density, 92–94
 - ecdf, 104–105
 - emppmf, 103–104
 - factor, 23, 37, 47, 62–64, 91, 110, 147
 - getdata, 17–18, 32, 75–76
 - glm, 44, 50–159
 - help, 4
 - help.search, 4
 - hist, 19, 158–159
 - inclusionprobabilities, 72, 75–76
 - install.packages, 3
 - intervals_ex40, 65–67
 - is.na, 12, 107–108
 - jpeg, 9
 - length, 144–145
 - library, 3
 - lines, 92–95, 101–102, 144–145, 150
 - lm, 52, 54, 63, 111, 139–140
 - lme, 64
 - lmer, 64
 - locpoly, 101
 - matrix, 4, 129
 - mean, 107–108
 - mstage, 71–75, 85–88
 - nobs, 143
 - order, 85
 - par, 92–94
 - pdf, 9
 - plot, 42, 50–54, 96–97, 150
 - postStratify, 48, 109, 124–125
 - predict, 41–42, 44–45, 50–54, 111, 144–145

- qt, 20
 - rake, 109–110
 - read.csv, 6–7
 - read.table, 6–7
 - regTermTest, 149–150
 - round, 100
 - rq, 102
 - sample, 15, 97
 - bootstrap for capture-recapture, 158–159
 - SRS, 15–16
 - stratified random sample, 30
 - unequal-probability sample, 69–70
 - seq, 144–145, 150
 - set.seed, 16, 25, 30, 65–66, 69–70, 114–115, 123
 - sink (saving output), 7–8
 - source, 8
 - srswor, 15–18, 72, 114–115
 - srswor1, 24
 - srswr, 15–18, 72
 - stargazer, 8–9
 - strata, 13, 31–32, 76
 - subset, 46–48, 91–92, 94, 99, 108, 150
 - summary, 130–131, 140–143, 146–150
 - svrepdesign, 126–127
 - svyboxplot, 92, 94–95, 100
 - svyby, 35–36, 47, 115–116, 145, 146
 - svycdf, 95, 104–105
 - svychisq, 130–134
 - svyciprop, 24, 37
 - svydesign, 15, 20, 100, 124–125
 - complex sample, 89–132, 142–143
 - one-stage cluster sample, 57–59
 - one-stage unequal probability sample, 78
 - pps sampling without replacement, 82
 - random group method, 115–116
 - replicated sample design, 114–115
 - SRS, 21–22, 140–141, 157–158
 - two-stage cluster sample, 59
 - use in calculating replicate weight variances, 141
 - use in creating replicate weights, 117–120, 122–124
 - svyglm, 44–45, 139–143, 146–150
 - svyhist, 92–95
 - svyloglin, 133–137
 - svymean, 15, 20, 92–95, 107–108, 114–125, 131–133, 139, 155–157
 - complex sample, 89
 - one-stage cluster sample, 57–59
 - proportion, 22–23
 - proportions with stratified random sample, 36–37
 - SRS, 21–22
 - stratified random sample, 33–35
 - two-stage cluster sample, 59
 - with replicate weights, 126
 - svyplot, 95, 98–99, 144–145
 - svyquantile, 88–92, 104, 119–120, 123
 - svyratio, 120–122, 157–158
 - one-stage unequal probability sample, 78
 - SRS, 41–44
 - stratified sampling, 49–50
 - svysmooth, 92–95, 101
 - svytable, 107–108, 130–132, 135–137
 - svytotal, 15, 20, 124–125
 - one-stage cluster sample, 57–59
 - SRS, 21–22
 - stratified random sample, 33–37
 - t.test, 114–115
 - table, 27–28, 32, 107–108, 145–146
 - tapply, 30, 35–36, 57
 - trimWeights, 110
 - twophase, 155–157
 - unique, 5, 27, 32
 - UPbrewer, 72
 - UPpoisson, 72
 - UPsampford, 72, 75–76
 - UPsampfordpi2, 80
 - UPsystematic, 72
 - write.csv, 9
 - write.table, 9
- Histogram
- complex survey, 92–94, 150
 - SRS, 19
- Imputation, 110–112
- Intraclass correlation coefficient, 65
- Jackknife, 113, 116–124
- Lists, 73
- Logistic regression, 149–150
- Loglinear model, 133–137
- Masking of R functions, 13
- Missing data, 12–13, 90
 - regression analyses, 143–144

- Mixed model, 64–65, 151, 162
- Multivariate imputation by chained equations, 111
- National Health and Nutrition Examination Survey (NHANES), 85, 89–92
 - graphs, 95–102
 - regression models, 142–150
- Nonresponse, 107–112
- Numeric variable, 23
- Odds ratio, 129–130, 150
- Packages in R, 2
 - BalancedSampling, 151
 - emdi, 162
 - FHDI, 111
 - ggplot2, 102
 - Hmisc, 111
 - installr, 2
 - kernSmooth, 101
 - knitr, 10
 - lme4, 64, 151
 - mice, 111
 - mme, 162
 - nlme, 64, 151
 - pps, 71
 - PracTools, 29
 - quantreg, 102
 - Rcapture, 161
 - sae, 161
 - sampling, xi, 3, 15, 27, 69, 85, 113, 151
 - SamplingStrata, 29
 - SDAResources, 3, 7, 15, 27, 65, 103, 158
 - SpadeR, 161
 - stargazer, 8
 - stratification, 29
 - survey, xi, 3, 15, 27, 41, 57, 77, 80, 88, 109, 113, 129, 139, 155, 162
 - VIM, 111
- Poststratification, 48, 108–110
- Probability proportional to size (pps) sampling, 69
- Quantiles, 88–89, 119, 123
- R chunk, 10–11
- R software, obtaining, 2
- R-squared, 143
- Raking, 108–110
- Random effects model, 64–65
- Random group method for estimating variances, 113–116
- Rao–Scott test, 131–132, 143
- Ratio estimation, 41–44
 - combined, 49
 - separate, 49–50
 - stratified sampling, 49–50
- Regression
 - design-based, 140–145
 - model-based, 139–140, 151
- Regression estimation, 44–45
- Replicate weights
 - constructing, 116–124
 - from survey data file, 126–127
 - nonresponse adjustments, 124–125
- Replication variance estimation, 80, 113–127
- Reproducible research, 10
- RStudio, 2, 10
- Scatterplot
 - complex survey, 95–102, 144
 - SRS, 42–43
- SDA, acronym for Sampling Design and Analysis, xi, 1
- SDAResources
 - agpop, 7, 17–18, 27–32
 - agpps, 82
 - agsrs, 19–23, 41–48, 50–52, 124–125
 - agstrat, 32–37, 49–50
 - algebra, 58–59
 - anthrs, 139–141
 - classes, 9, 69–75, 85–88
 - classeslong, 9, 85–88
 - classpps, 80
 - college, 114–115
 - collegerg, 120–122
 - coots, 59–60, 122
 - deadtrees, 44–45, 52–54
 - gpa, 57–58
 - htpop, 104
 - htsrs, 88–89, 123
 - htstrat, 89, 92–94, 103–105, 123–124
 - impute, 107–108, 110–111
 - intervals_ex40, 65–67
 - nhanes, 89–92, 95, 119–120, 142–150
 - santacruz, 42–44
 - schools, 61–65
 - syc, 115–116, 130–133, 135–137
 - vietnam, 155–157
- Separate ratio estimator, 49–50

- Simple random sample
 - computing estimates, 18–23
 - selection, 15–18
 - weights, 18
- Small area estimation, 161–162
- Stratified sample
 - allocation, 27–29
 - computing estimates, 32–37
 - selection, 30–32
- Sweave, 10–11
- Trend line
 - quantiles, 102
 - smoothed for mean, 101
- Wald test, 131, 136, 143, 150
- Website for book, xi
- Weight
 - cluster sample, 71, 75
 - SRS, 18
 - stratified random sample, 32
- Weighted least squares, 50
- Weighting class adjustments for
 - nonresponse, 108

